

# Polaris

*Fixus inter mutabilia*

## The map of the system

A pre-pilot, issuer-unlinkable, duress-aware identity-token system, signed with ML-DSA-65 under an audited algorithm-migration path, read from the centre outward

Egor Khaklin

Seton Hill University · September 2026

# Before you read

Polaris is an attempt to build identity infrastructure in which important claims do not have to be trusted merely because a central application says they are true. Authority, state, history, privacy boundaries and the system's own behaviour are progressively exposed to independent verification and constrained by layers that do not share each other's assumptions. Each layer exists because the layer inside it could be entirely correct and still fail to prove what a society needs to know. That progression is the spine of the paper.

The rule the project follows is that the unit of progress is a new external dependency survived: a named wallet, a named conformance profile, a named relying party, a defect filed by somebody who is not the author. The count of invariants, versions and lines is not progress and the repository's scoreboard refuses to record it. This paper follows the rule. Where it reports internal evidence, it says so, and it keeps that evidence out of the sentences that report the other kind.

This document is a map of that progression, drawn at version `1.0.0-rc.7` of the system (20 September 2026), and revised on 28 October and 2 November 2026. The system has kept moving since. A later event that changes what a reader should believe is dated where it appears; everything else later work corrected or learned is recorded in the companion *Evidence Record* and does not redraw the map. That record also holds the history of this report's own versions, the corrections, and how every count was measured, on the tree and not copied from earlier documents; where the repository's documents disagree with its code, the code wins. Every term of art is in the glossary, Appendix C.

## What this is, for whom, and what it asks

In plain terms: Polaris is software for issuing and checking identity credentials, built so that the people who run it cannot quietly rewrite what it says, watch the people it describes, or turn it against them. It is for the engineers, reviewers and institutions who have to decide how such a system should be built. Polaris is pre-pilot software on notional data. What it asks of a reader is a test: walk the stranger's path in Section 9, run a verifier against it, or attack it, and report what happened. The first wallet written by strangers to try it refused it before it accepted it; Section 20 tells how.

## Five marks

Every capability carries one of five marks, so that code that runs is never confused with an intention, and so that a thing the author has tested is never confused with a thing somebody else has. In the running text an implemented path carries no mark, since most of this map is exactly that; a mark appears there only where a claim is something else. Tables mark every row.

**IMPLEMENTED** A path that runs at `1.0.0-rc.7`, exercised by a test or a drill in continuous integration, and pinned by an invariant check that fails the build if it stops being true.

**EXPERIMENTAL** A path that runs but is limited in a way the repository itself names: not covered by CI, notional in scale, or dependent on a backend that CI simulates.

**EXTERNAL VALIDATION REQUIRED** A property the repository implements and tests internally but that no external party has audited, attacked, deployed or independently re-implemented.

**EXTERNALLY EXERCISED** A path that a named party outside the repository has actually exercised, on a date, with a result that can be pointed at: a wallet, a conformance service, a published test corpus. It is given only for a filled row of the repository's scoreboard, and never means audited, certified or piloted. Three things carry it in this paper.

**PROPOSED** A proposal: a roadmap row, a design note, or an idea that exists only in this paper. It is not code.

# Contents


Before you read i
1 The problem 1
2 How to read this map 2
3 The core: one credential, one signature, one person 6
4 The walls: the schema as the security boundary 11
5 Authenticating truth: signatures, custody, two witnesses and one migration 14
6 Verification without the issuer: the packaged verifiers, the conformance contract and the first strangers 18
7 Verification is not authorization: status, revocation and the offline trade 21
8 The privacy boundary: what a verifier is allowed to learn 25
9 Using the credential: holders, relying parties and the login that keeps no record 30
10 Federation: independent authorities that recognise one another 32
11 Watching the watchers: logs, witnesses, the split view and the public report 36
12 Institutions: exchange, time and signatures without a message log 39
13 Surviving reality: the stack, failure, load, the standing witness, and the day a pilot ends 43
14 How Polaris observes and understands itself 46
15 The assurance layer: how the system notices its own errors 49
16 What Polaris refuses to become 51
17 AI agents, proof of personhood and the future internet 53
18 Future research and engineering, under a rule that forbids most of it 57
19 The design's boundaries, and the tests that would break its claims 60
20 The last ring: other people's software 62
21 Conclusion: the whole town 64
A Capability status 69
B Lineage: what is inherited, and what is not 75
C Glossary 76

# 1 The problem

The problem is worldwide, and it takes two forms. Some people have no official credential at all: the World Bank's 2025 estimate is that about 800 million people have no official ID, and that at least 2.8 billion have no government-recognised digital identity with which to transact online [9]. Others hold several credentials that do not talk to each other. In the United States, for example, a person carries six to eight: a driver's licence, a passport, a Social Security card, a Real ID, a voter registration, an insurance card. Each is a separate artifact, signed by a separate authority to a separate standard, with no shared revocation path and no shared audit trail. The shift to digital identity wallets is under way: in the European Union, every member state is now obliged to provide at least one European Digital Identity Wallet [5]. Consolidating a person's credentials into one active credential record, verified through context-scoped events, is the easy half of the problem, and it is still the problem's shape. For a person with no documents at all, the record begins with enrolment through a trusted referee (Section 3); Polaris implements that path, and no proofing has been performed with it.

The hard half is what happens when the system works. An identity system that succeeds becomes the doorway to everything, and a doorway is a place where power concentrates. Three failure modes follow, and every layer of Polaris exists to answer one of them.

**The system lies about the past.** An operator, or an insider with database access, revokes a token and later erases the record; issues a credential under procedures the public was never told of; or shows one history to an auditor and another to a court. A national identity system cannot be allowed to retroactively rewrite what it did.

**The system watches.** Every verification is a fact about where a person was and what they were doing. A log of those facts is a surveillance backbone whether or not anyone intended one, and it stays one after the operator's intentions change.

**The system coerces.** A holder can be compelled: a border officer, an employer, or a thief can force a credential to be presented, and a doorway that every service requires can force a population to enrol. An identity system is also the most efficient instrument of exclusion ever built, if the list of who is not in it can be produced on demand.

The conventional answer to all three is policy: rules about retention, access, audit and consent, enforced by the people who run the system. Polaris's constitution (`MISSION.md`) takes the position that policy is worthless as the only thing holding a promise up, because the people who run the system are exactly the ones the promise is meant to bind. The promise has to be enforced where it cannot be talked around: in the schema, in triggers, in unique indexes, in check constraints, in signatures that anyone can verify without asking the system, and in logs that other people watch. The application is not the security boundary. The database is, and the layers around the database exist to make even the database's claims checkable by strangers.

## 1.1 Why post-quantum from the first day, and what the phrase can claim

Identity data has a secrecy horizon measured in generations, and migrating a national system takes a decade. By Mosca's inequality, which compares the years data must stay secret plus the years a migration takes against the years until a quantum computer can break today's signatures, a credential system deployed on classical cryptography in 2026 is a deferred compromise rather than a working system (the companion *Evidence Record*). At `1.0.0-rc.7` the default signing algorithm is ML-DSA-65 (the lattice-based signature standard FIPS 204), ML-DSA-87 is accepted beside it, the zero-knowledge proof system (which proves a statement without revealing the data behind it) is hash-based with no trusted setup, and the public TLS edge and one of the two internal hops negotiate a hybrid post-quantum key exchange.

*Post-quantum* is accurate about the algorithms. It is not a claim the repository can make about security, because ML-DSA's security rests on lattice assumptions that could fall to mathematics rather than to hardware, and because the default development path signs with a labelled placeholder rather than a lattice signature at all. The defensible claim is *algorithm agility under an audited migration path*: the algorithm is a row in the schema and never a constant, a population can be re-signed under a new one on a path that runs on every push, and what a break would cost is written down. Section 5 gives the argument and the cost; an invariant check refuses any outward surface that says more.

### 1.2 The progression this paper follows

Each of the following sentences names a layer, and each is true of the layer before it.

- A credential's authenticity is not its current authorization. A signature stays genuine forever; the authority under it can be withdrawn this afternoon.
- Current authorization is not privacy. A verifier can learn that a credential is valid and still learn far more than it needed to.
- Federation is not institutional independence. Two instances of the same code run by the same author prove that the protocol works, not that independent institutions run it.
- A transparency log is not protection from a split view. One observer can be shown a private fork of history and never know.
- A green test is not proof that the test asked the right question.
- Privacy is not freedom from coercion. A system can know almost nothing about a person and still be the doorway every service demands.
- A system that has checked itself is not a system that anyone else has checked. Every instrument in the repository passed over a defect that the first outside implementation refused on sight.

Two of them the project learned most expensively: the green test, which Section 15 is about, and the last, which the last ring, Section 20, carries.

## 2 How to read this map

Polaris is not one enormous table or one enormous server. It is a very small credential core with deliberately bounded structures arranged around it, each of which constrains, protects, observes, verifies, records or limits something about the structure inside it. The paper reads outward from the core, ring by ring (Figure 2), and the reader should be able to zoom: from the person, to the credential, to the cryptography that authenticates it, to the schema rules that bound it, to the authority that issued it, to the parties that rely on it, to the federation of authorities, to the witnesses that watch them, to institutional exchange, to the operations that keep it alive, to the society it sits inside, to the environment of software agents it now serves, and finally to the outside: the software written by strangers whose acceptance or refusal is the only evidence the repository did not produce about itself.

### 2.1 The questions asked of every layer

For each major component the paper answers the same questions, in a card of the form used throughout. What is this? Why does it exist? What problem does it solve? What does it trust? What does it refuse to trust? What can it see? What is it deliberately prevented from seeing? What happens if it fails? What part of Polaris checks it? Is it needed now, or is it preparation for a future environment? How does it connect to the layer inside it and the layer outside it? The answer to what checks it is the only one kept out of the cards: the checks and drills behind each

card are listed in Appendix A, so the body reads without them. A reader who can answer those questions for every ring has the mental model this paper exists to build.

## 2.2 The town

IN THE TOWN The credential record is the register kept in the town hall: one entry per person, the claim that everything else is organised around. The wall is the schema: triggers, check constraints and unique indexes that decide which states may exist at all, and which bind every visitor, every official and every restore from backup alike. The gates are the endpoints through which a claim enters or leaves: issuance, verification, login, exchange. The watchtowers are the transparency logs, and the witnesses stand outside the wall, cosigning what the towers publish and shouting when two towers disagree. Other towns are other authorities, each with its own hall and its own key, joined by roads that run in one direction and for one purpose. Above everything stand laws that the town itself cannot repeal by a vote of its officials. And beyond the walls are travellers from towns nobody here built, who owe the town nothing; what they find at its gate is the last thing this map shows.

The map of the town, with the technical name beside every civic one, is Figure 1 below. The analogy describes arrangement and sight lines, not political centralisation: Polaris is federated per authority, with no central database, root or verification service. It will recur in a shaded box like the one above, and every recurrence is followed by the mechanism it stands for. Each ring also opens with one line in italics: the paper's single idea, restated for that ring, that the ring inside it was correct and still not enough.

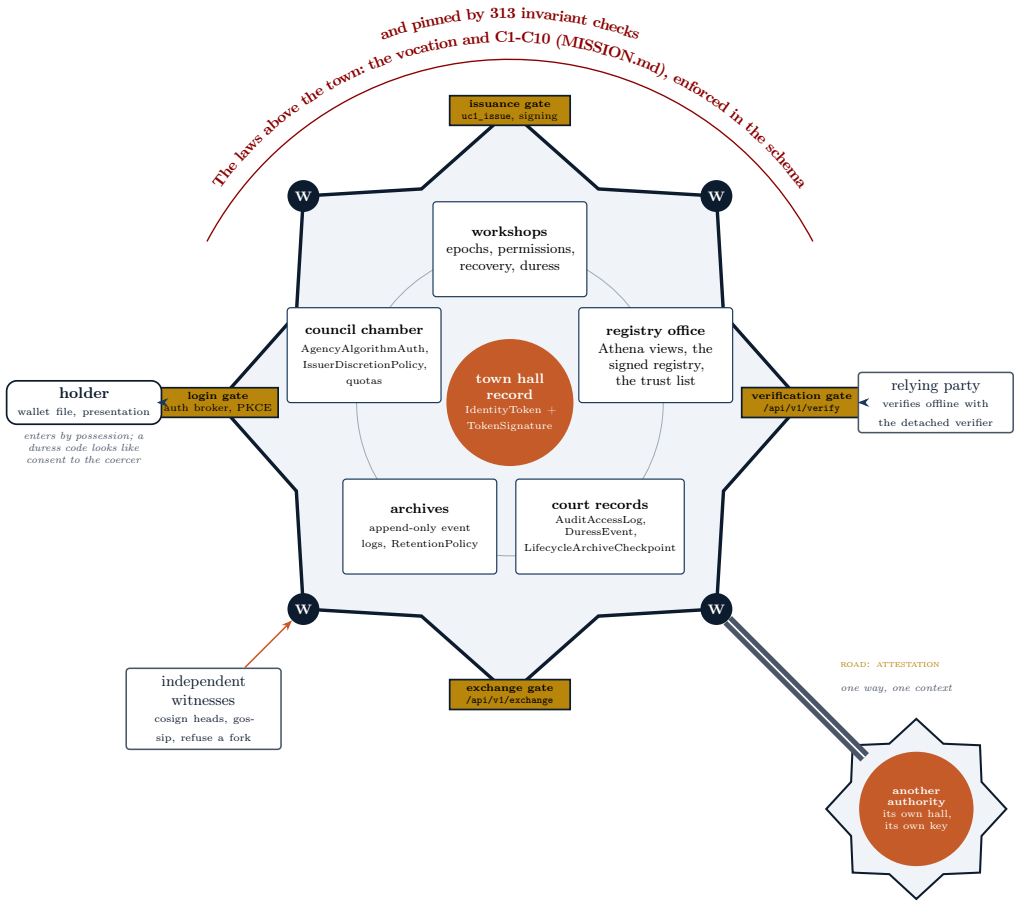
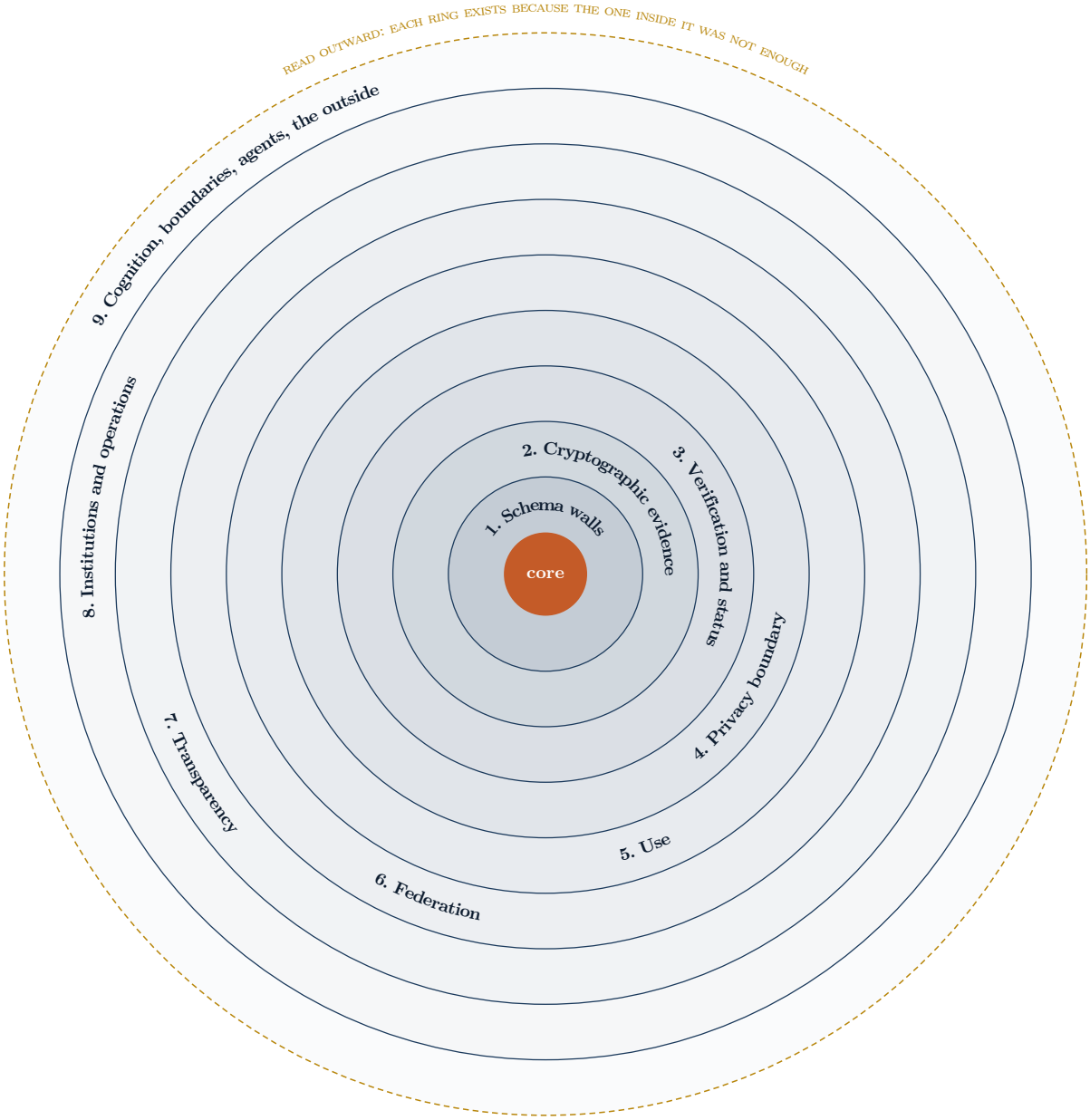



**Figure 1:** The walled town, with the real component beside every civic name. The register in the hall is the credential core; the wall is the schema; the four gates are the routes through which a claim enters or leaves; the watchtowers are the transparency logs and the witnesses outside the wall watch them; the road to the second town is a directional, per-context attestation; the laws above the town bind the town itself. The plan describes structure. Polaris is federated: every authority is its own town.




**Core.** one `IdentityToken` row and its `TokenSignature`: a signed claim

**1. Schema walls**
triggers, CHECK constraints, the partial unique index; C1–C10

**2. Cryptographic evidence**
ML-DSA signatures, custody, two witnesses, canonical signing

**3. Verification and status**
detached verifier, SDKs, conformance, status assertions, epochs

**4. Privacy boundary**
disclosure levels, unlinkable records, membership proofs

**5. Use**
wallet, presentations, relying-party API, auth broker

**6. Federation**
per-authority roots, manifests, directional attestations, feeds

**7. Transparency**
logs, signed heads, witnesses, gossip, equivocation proofs

**8. Institutions and operations**
registry, gateway, receipts, timestamps, signing; HA, DR, chaos, CI

**9. Cognition, boundaries, agents, the outside**
Atlas, Athena; what is refused; agents and personhood (built); strangers' software (dashed: not ours)

**Figure 2:** The concentric map of Polaris. The core is one credential record and its signature. Each ring names the actual components on it, and the reading order of the paper follows the rings outward. The outermost ring is dashed because most of it is not code: it is what parties outside the repository have done with the rings inside it, and at `1.0.0-rc.7` that is one wallet, one hosted conformance run and one published test corpus.

### 2.3 Trust and refusal

The most useful habit for reading Polaris is to ask, of every layer, what it refuses to take on faith from the layer below. The schema refuses to trust the application. The detached verifier refuses to trust the issuer's server. The second witness refuses to trust the first implementation. The relying party refuses to trust a transitive chain of authorities. The witness refuses to trust the log's own head. The check layer refuses to trust a check that cannot detect its own violation. The mutation drills refuse to trust a test that has never been shown to fail. And the scoreboard refuses to count anything the author ran against the author's own code. The conclusion draws the refusals as a ladder (Figure 15); Sections 3 through 13 explain each one.

## 3 The core: one credential, one signature, one person

*A record of a person is not the person.*

### 3.1 What the credential is

At the centre of Polaris is one row of the table `IdentityToken` and at least one row of `TokenSignature`. The token row records a unique `token_value`, a physical serial, the person it belongs to, the agency that issued it, the algorithm it was issued under, its predecessor if it replaced an earlier token, and its status. The signature row records the issuing agency's ML-DSA signature over the SHA3-256 digest of the token value, together with the public key that produced it, so that verification is self-contained and survives the retirement of the signing key. A token without a signature row cannot exist: a database trigger refuses any write that would leave one without a non-deprecated signature. The credential is therefore not a document but a signed claim: *this authority issued this token to this person.* The claim may also bind a key the holder controls, recorded in `HolderKeyEvent` under the same append-only discipline as an authority key, so that possession of the file stops being possession of the credential (Section 17).

What the holder holds is the authenticity pack: the token value, the signature and the issuer's public key, exported by the application and kept as a file by the reference wallet (`scripts/polaris-wallet.py`). Anyone can check the pack with a standard ML-DSA library and no Polaris code (Section 6).

### 3.2 The physical token, as an object somebody else can implement

The silicon is modelled and not manufactured; everything around it exists. The card is an encoding with a reference codec and published vectors (`polaris_card/card_profile.py`), a software token that speaks ISO 7816-4 with real status words, two PIN slots, a retry counter and a PUK (`emulator.py`), a personalization step in which the card generates its own keys so that the authority never holds them, and a reference verifier device that reads a card over NFC or a presentation over QR and reports three findings separately, possession, authenticity and authorization, because a revoked card still signs a challenge and a device that says only *accepted* teaches its operator nothing. The profile states its one shaping constraint plainly: certified secure elements that sign FIPS 204 in hardware are not something an authority can buy in volume today, so a card carries two issuer signatures over one body, a classical one that today's silicon can produce and an ML-DSA one, and the reader decides which it trusts. Every one of these runs in a drill on every push. IMPLEMENTED for the profile, the emulator, the vectors and the device; PROPOSED for a manufactured card, which no drill can establish.

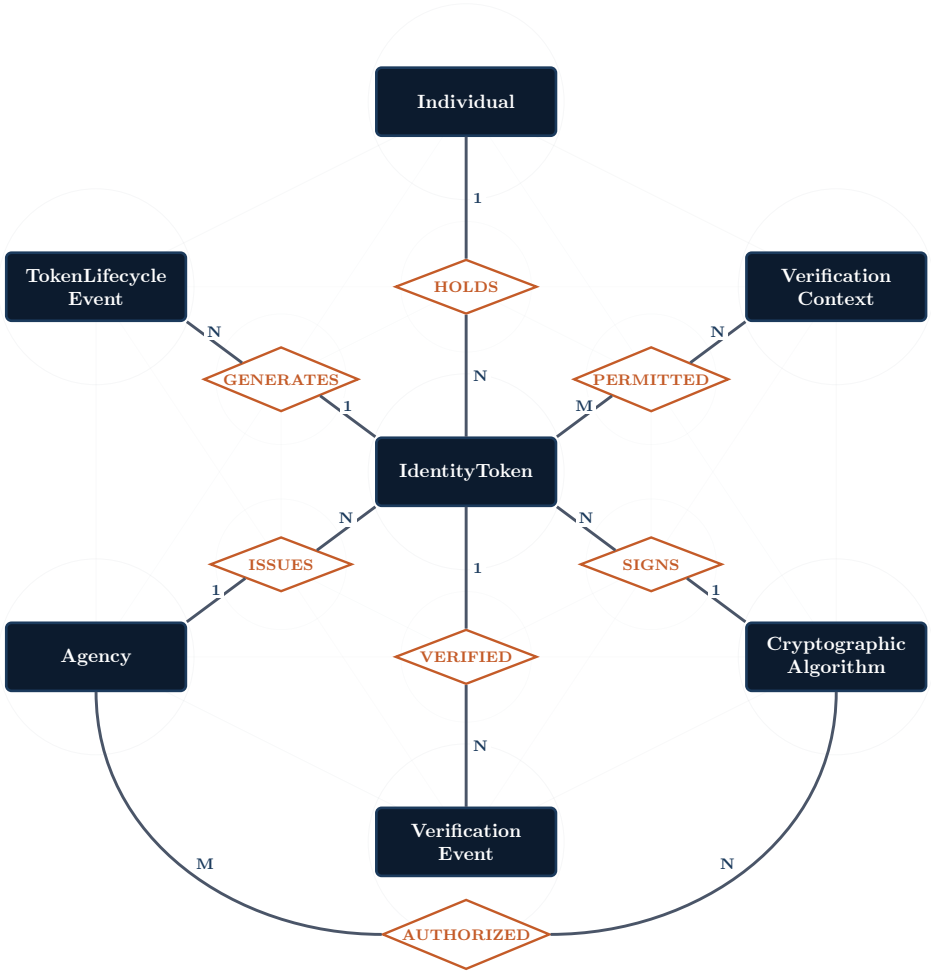



**Figure 3:** The core entity geometry, unchanged from Version 1. Seven entities around `IdentityToken`: the person who holds it, the agency that issues it, the algorithm that signs it, the contexts it may be verified in, and the two append-only event logs it generates. The many-to-many relationships resolve to the junction tables `AgencyAlgorithmAuth` and `TokenPermission`. This geometry is the constant of the paper: Section 4 draws the other thirty-eight tables as rings around it.

## 3.3 What the credential is not

The constitution's tenth constraint, C10, states that identity attestation never carries spending authority: there is no monetary claim in the schema, and a check fails the build if one appears. The reason is stated as a failure mode, not a preference. A credential that is also money inherits programmability: every constraint anyone wants to place on spending accretes onto the identity token, until an administrative paperwork error becomes an existential bank-balance error, and until the system can be told that a person may not buy fuel. The same refusal covers reputation, behaviour and profiles. Polaris answers one question, *is this token valid for this context right now*, and it declines to answer *should this person be allowed to do X*. That second question belongs to the relying party, outside the boundary.

### 3.4 One active token per person

A person may hold several provisioned tokens but only one may be `ACTIVE`. Two active tokens for one person would let one token authorise a transaction and the other repudiate it. The rule is C3, and it is enforced by a partial unique index, `uq_one_active_per_person`, on the person's identifier where the status is active. Two operators activating two reserve tokens for the same holder at the same moment find that exactly one of them succeeds, and the test that proves it runs with real threads against a live database (C9). The rule is also modelled in TLA+ and the model is checked on every push, under every interleaving of concurrent issue and revoke, with a counterpart configuration it is required to fail. One more thing is true of it, recorded in the companion *Evidence Record*: when the index was dropped to see what would notice, the suite that exists to prove C3 turned out to be violating it, committing a second active token before failing.

What C3 calls a person is a row, `Individual.individual_id`, not a human being. The index makes a second active credential for the same row impossible; it does nothing about the same human enrolled as two rows, and no mechanism in the repository does. Uniqueness of humans rests on enrolment, where an operator records the evidence and the assurance level is derived from it.

### 3.5 The lifecycle

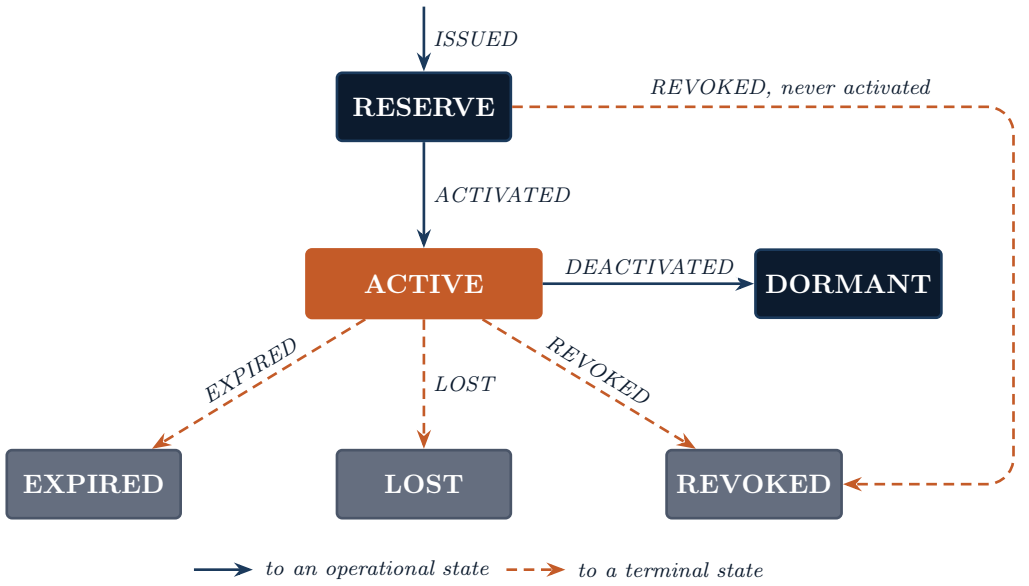



**Figure 4:** The token lifecycle machine as the trigger `trg_token_state_machine` enforces it: six legal transitions, each labelled with the lifecycle event it writes automatically. `ACTIVE` is the only operational state. A token enters at `RESERVE` and is activated, or is revoked before it ever is; an active token leaves through one of three terminal states or into `DORMANT` when a successor takes over.

A trigger enforces the transitions, and a second trigger writes a `TokenLifecycleEvent` row for every status change, including a change made by hand from a database shell. Succession is by reference, never by overwrite: a replacement token points at its predecessor and the old token stays in the database in its terminal state. Lost tokens stay lost, and their data is not erased.

### 3.6 Entering, leaving and returning

Three mechanisms surround the core so that the schema cannot be turned against the holder at the edges of the lifecycle, and each keeps a record of what it rested on.

**Entering: tiered enrolment, and what an enrolment rests on.** Every person has an enrolment status drawn from five values, and `EXEMPT` is a first-class one: a recognised way to take part in civic life without a token, whether for biometric incompatibility, religious exemption or an alternative path. Recording an exemption is one row; enumerating the not-enrolled is possible but unhelped by any function, and it is audited when done. A credential records how the person was proven to be who they claimed: `EnrollmentProofing` records one proofing event and `EnrollmentEvidence` what it rested on, both append-only, and the assurance level is *derived* from the evidence rather than typed by an operator; there is deliberately no column for the document itself, and a named set of fields is refused outright, so the record cannot reconstruct a passport. Document authenticity is the operator's attestation.

Every combination of evidence a standard names starts from documents, and a person with none, a care leaver, a refugee, an adult who never held a passport, fails all of them by omission. The trusted referee (`RefereeVouching`) is the answer to that, and it is also the easiest way to mint an assurance level out of nothing, so the repository treats the anti-exclusion mechanism and the forgery channel as the same table: a vouching is bounded, co-signed past a threshold, and invisible on the credential itself, because a person who needed one should not carry a mark for it at every counter. An enrolment code, the last piece that needs no hardware, establishes only that somebody who could reach a channel returned a secret, and the level it can contribute is capped accordingly.

**Leaving: issuer discretion, as a decision that survives being changed.** The worst failure of an identity system is not a forged token but an authority revoking a population's tokens at scale. A trigger bounds how fast one agency can revoke its own tokens (five percent of its outstanding base in thirty days by default) and above that a co-signer from a different agency is required, recorded in the audit trail. The five percent is a deployment default and a governance parameter, not a security invariant: an authority may set its own (one seeded agency runs at seven), and no property of the system follows from the number itself. The bound converts a surprise into a trend that reporting can see; it does not stop slow abuse under the ceiling, and says so. The settings around this power are records: the ceiling itself (`IssuerDiscretionPolicy`) can only be changed with a stated justification and leaves a row saying who changed it and from what; a quota on how much of the population an agency may touch is a decision with a reason rather than a value that can be overwritten; the row at the root of the hierarchy, `Agency`, carries a trigger that records every change to an authority's level and marks whether it widened; and a relying party's bar is recorded with the same discipline, weakenings flagged and reasons required.

**Returning: the recovery ceremony.** A fire takes the wallet and the reserve together. Recovery is mandatory and is also an impersonation path, so it is made expensive without being made impossible: a two-phase ceremony with a 48-hour cool-down, three independent out-of-band channels, an approver who cannot be the requester, and admin-only completion. The decision fields move one way under a trigger, and the four-eyes rule, the cool-down and the three channels are refusals a stored procedure makes, each of which a mutation drill shows the tests would notice losing (Section 4). The three channels are recorded one at a time, each by someone other than the requester and each attributed; what the record shows is who attests a check, not that it happened. The holder is without identity for the cool-down. IMPLEMENTED for the ceremony, PROPOSED for an interim credential for that window.

### 3.7 Duress: an alert the coercer cannot see

The vocation above the ten constraints is anti-coercion: no person may be compelled to renounce, transfer or surrender their identity against their will. The mechanism that gives that sentence something to stand on is old and comes from banking: a second secret that looks exactly like success to whoever is watching, while an alert goes out through a channel the watcher cannot see. A holder may enrol a duress code. When it is presented, the verification proceeds identically on every operator-visible surface, the same page, the same flash message, the same event row, and a `DuressEvent` row is appended to a table the operator's screens never join, from which a pager fires at the highest severity. The comparison is constant-time and the work is paid on both paths. On the card the same mechanism is a second PIN of the same length as the first, entered the same way, unlocking a second key slot whose public key the card object does not carry, and a drill measures the response in time as well as in bytes, because a coercer at a reader observes how long the card took. IMPLEMENTED as a tested property of the modelled flow.

The duress mechanism is *duress-aware*, not compulsion-resistant: it can keep an alert from a coercer who does not know it exists, and it does not protect against an informed coercer or against institutional access. A check refuses the stronger word on every outward surface. Against a casual coercer, one who does not know duress codes exist, the mechanism works: the screen is identical and the alert is silent. Against an informed coercer standing beside the holder it does not: the code is typed on the verifier's terminal, keystrokes are not constant-time, and because enrolment is opt-in, *I have no duress code* is a claim a coercer can press on and the holder cannot prove. Against post-hoc lawful or institutional access it is worse than nothing: `DuressEvent` is append-only with no purge path, which is C1 working as intended, so where the coercing party is the institution, or can later compel it, the mechanism manufactures indelible evidence that the holder resisted. The assessment proposes no mechanism change and this paper proposes none; it records the boundary. The side-channel claim, that nothing observable distinguishes the two paths, rests on the repository's own measurement. EXTERNAL VALIDATION REQUIRED

**Layer card: the credential core**

| | |
|---|---|
| **What it is** | One `IdentityToken` row and its `TokenSignature` rows, optionally bound to a holder key: a signed claim that an authority issued this token to this person, in a lifecycle with one active state, with the proofing it rested on recorded beside it. |
| **Why it exists** | Every other structure in Polaris organises itself around this claim. It is kept as small as possible so that the rest can be bounded. |
| **What it trusts** | The issuing agency's signing key, held behind a custody interface (Section 5); the schema's constraints; the operator's word that presented evidence was genuine. |
| **What it refuses** | Application code as a source of truth: the state machine, uniqueness and audit are decided by the database. An assurance level typed rather than derived. Money, reputation and profiles: C10. |
| **What it can see** | The holder's legal name, date of birth and country; the issuing agency; the algorithm; the lifecycle; the kind of evidence an enrolment rested on. |
| **What it cannot see** | A biometric template (never stored), the document behind an enrolment (no column exists for it), a verification graph (Section 8), a spending history (never modelled). |
| **If it fails** | A forged core is a forged signature, which the two-witness verification and the detached verifier refuse; a duplicated core is refused by the unique index; a rewritten core is refused by the append-only triggers; a coercer who is the institution is not stopped, and the record says so. |
| **Now or later** | Now. It is the part of Polaris that has existed since Version 1; the card is an object now. |
| **Inside and outside** | Inside: the person, who exists outside the software. Outside: the wall of constraints (Section 4). |

IN THE TOWN The town hall keeps one register. Each entry says who a person is, which official entered it, under which seal, and what papers the official was shown, though never the papers themselves. The register cannot be edited, only appended to; a second entry for the same person is refused at the desk; and the seal is one that every reader can check against the official's published mark without asking the hall.

# 4 The walls: the schema as the security boundary

*A promise enforced only by the people who run the system is not a promise.*

IN THE TOWN The wall does not ask who is passing. It holds against the mayor, the clerk and the stranger alike, and it holds again after the town is rebuilt from its plans. A rule written into the wall does not depend on anyone remembering it.

## 4.1 Why the database is not plumbing

In most systems the database stores what the application decided. In Polaris the database decides, and the application is one client among several: the operator's web application, the command-line tool, a bulk loader, a database shell, a restore from backup. A rule that lives in application code binds only the clients that run that code. A rule enforced by a trigger, a check constraint or a unique index binds every client, survives every restore, and cannot be bypassed by the next caller, because the application role has no data-definition privilege with which to remove it. The governing sentence of the architecture is *by construction*: the system cannot be operated in a way that violates the constraint, because the constraint is enforced before any application code runs.

The design principle behind it is older than Polaris: make invalid states impossible to represent rather than asking developers not to create them. A verification event that records a token identifier at the zero-knowledge disclosure level is not an application bug to be caught in review. It is a privacy violation that must never be storable, so it is a check constraint on the row.

## 4.2 The ten constraints and where each lives

The constitution names ten hard constraints in two tiers. Five are constitutional, rights guarantees that may not be relaxed without an amendment recorded in the changelog; five are engineering invariants, the disciplines that keep the first five true under load and attack. The hierarchy is one line deep, vocation, then constitutional constraints, then engineering invariants, and the constitution states that no deeper apparatus governs them.

**Table 1:** The ten constraints at `1.0.0-rc.7`, their tier, and the object that enforces each. The table is kept honest against the code by `check_c1c10_objects_resolve`, which fails the build if a named object no longer exists.

| # | Constraint | Tier | Enforced by |
|---|---|---|---|
| **C1** | The audit trail is append-only; history is never rewritten or deleted | Constitutional | Triggers reject UPDATE and DELETE on every audit table (`reject_audit_modification`) |
| **C2** | A zero-knowledge verification stores no token identifier | Constitutional | Bidirectional CHECK `chk_disclosure_token_consistency` |
| **C3** | One person holds at most one ACTIVE token | Constitutional | Partial unique index `uq_one_active_per_person` |
| **C4** | Failed-login counting is atomic | Engineering | Single-statement `UPDATE ...RETURNING` |
| **C5** | No inline scripts; the content security policy is `script-src 'self'` | Engineering | Response header, verified per route |
| **C6** | Disclosure level is enforced server-side; a client cannot upgrade what it learns | Constitutional | Server code paired with redaction tests; the C2 CHECK behind it |
| **C7** | No hardcoded cryptography; algorithms are rows in a registry | Engineering | Foreign key to `CryptographicAlgorithm` |
| **C8** | Every map and API aggregate is bounded | Engineering | Route-level caps, passed into the SQL as its limit |
| **C9** | Concurrency claims are tested with real threads | Engineering | Threaded suites against a live database |
| **C10** | Identity is not money; the schema carries no monetary claim | Constitutional | Structural absence, pinned by a check |

The repository also records a structural claim about the ten: that the set is closed, so that an eleventh constraint requires either replacing one or extending the framework, and that the constraints depend on one another, so that removing any one ripples through the rest. The dependency walk (remove C10 and every other constraint protects a financial-surveillance system instead; remove C1 and every other constraint's enforcement becomes retroactively deniable) is the argument for treating them as a graph rather than a list. Athena, the read-only layer that describes the structure of authority (Section 14), holds the map from each constraint to the exact live mechanism that enforces it, and a check fails the build if a mechanism drifts. Three of the ten now also have a formal model checked on every push: C1's purge coverage, C2's unlinkability across two pooling relying parties, and C3 under concurrent issue and revoke, each with a counterpart configuration it must fail, so none of the three is vacuous.

## 4.3 The audit of record

Fourteen tables follow one named pattern, the audit of record: the row and its history are the same object. There is no separate event log beside the table that could be edited while the table is locked. Mutation is either forbidden outright or bounded to one field that moves one way, such as a signature's deprecation date or an attestation's revocation date. Every one of the fourteen is enforced by a trigger that raises `insufficient_privilege`, and a check fails the build if any trigger stops doing so. Beyond the fourteen the design record names, thirty-one tables in all carry an append-only trigger at `1.0.0-rc.7`, the new ones being the records that decisions about authorities, operators, relying parties and enrolments now leave behind, and C1 is tested once per table that claims it. Of the thirty-one, twenty-four refuse every edit, and a table-driven suite tests each of them for both UPDATE and DELETE, cross-checked against the catalogue; the other seven permit bounded mutation, one field moving one way, and the trigger mutation drill is what answers for them. The cross-check finds tables by their guard function, and the list of those functions was written by hand, so a new table under a new guard would have gone untested

without anything saying so. Since 23 September 2026 every such guard must be declared strict or bounded, so that each has a named test that answers for it, and an undeclared one fails the build.

The corollary is that no foreign key anywhere in the schema cascades a delete. A cascade would let the lifecycle events of a deleted token vanish with it. Every reference defaults to `NO ACTION`, so a principal referenced by any audit row is effectively undeletable, and a check scans every SQL file for both cascade forms with no allowlist. The one record that deliberately carries no foreign key at all, so that ending a relying party's contract cannot erase its history, taught the repository something about its own test database at `1.0.0-rc.7`: the seed that reloads the sample world restarted the party's identifiers from one and reached that record with neither, so the first party of each reload inherited a dead party's events. The seed names the record now, and a check computes from the schema's own foreign keys which records a reload would leave behind.

The application role, `polaris_app`, is revoked UPDATE and DELETE on the append-only tables and holds no data-definition privilege. The only path that deletes audit rows is the archive-then-purge procedure, which verifies the archive against its manifest first, refuses any cutoff inside the retention window, and appends its own append-only checkpoint row. The retention window itself is data with a floor: no policy row can express a retention shorter than a year, because a check constraint refuses it, and shortening the floor is a schema change answered for in review. The vocation refuses unbounded retention; it equally refuses retention short enough to erase the record.

## 4.4 The check layer, and the drills that attack it

Above the schema sits a flat layer of 313 plain functions, each of the form `check_*(repo_root)`, that read the repository and return a finding. They gate continuous integration: any failure fails the build. Each check is paired with a detection test that breaks a fixture in the specific way the check claims to catch and asserts that the check turns red, and also asserts that the check passes on the same fixture unbroken, because a test that only ever asserts failure has not established that it detects anything. A check that cannot detect its own violation is treated as broken. This is the mechanism by which prose claims are turned into gates: a count stated in the README, a route that must return a header, a migration that must not cascade, a drill that CI must run.

The walls are tested as well as trusted. A mutation drill removes each check, each CHECK constraint, each trigger, each unique index and each refusal a stored procedure makes, one at a time, and requires something to go red; it runs on every push, with a negative control so that zero survivors cannot be confused with a harness that ran nothing. Section 15 describes that layer, and the companion *Evidence Record* records what it found, including the suite that exists to prove C3 violating C3.

**Layer card: the walls**

| | |
|---|---|
| **What it is** | PostgreSQL 16: 45 tables, 46 triggers (42 of them in the trigger file), sixteen stored procedures and four use-case functions that implement every state-changing use case, the append-only audit tables, the retention policy as data. |
| **Why it exists** | A promise enforced only by the people who run the system is not a promise. The schema binds every client, every restore and every insider with a database connection short of the owner. |
| **What it trusts** | The database engine and its owner; the correctness of the constraints as written, which is why each is now mutation-tested. Above the owner there is nothing: an owner can disable a trigger, drop a policy or rewrite a row, and every guarantee in this section falls with it. Against the owner the only defence is evidence held outside the database, the transparency logs and their witnesses, and that defence is as independent as the witnesses a deployment recruits: with witnesses its own operators run, the schema binds every actor except its sovereign. It also binds only the roles that actually run: several of the corrections recorded in the companion *Evidence Record* since this map was drawn are guarantees that held for the owner, whom the suites connected as, and not for the application role production uses. And the routines that record a governance act trust the acting user they are given: they run with the owner's rights and check the named administrator's role, not that the caller is that administrator, so a compromised application can act as any administrator within their rules. |
| **What it refuses** | Application code, review process and operator discipline as the last line: the application never enforces a constitutional guarantee on its own. A test that has never been shown to fail. |
| **What it can see** | Everything the system stores. This is why the privacy layer bounds what is stored, not who may read it. |
| **What it cannot see** | Biometric templates, holder-side keys, enrolment documents, the content of exchanged messages or timestamped documents: none enter the database. |
| **If it fails** | A dropped trigger fails the build: deleting each of the 42 in the trigger file in turn, a check notices 38 by reading the schema, and the trigger mutation drill notices all 42 by running the suites; a lowered retention floor or a cascade fails the checks; a constraint whose loss the suite would not notice fails the mutation drill; a database owner with a shell has larger options than any trigger, which the threat model states rather than hides. |
| **Now or later** | Now. |
| **Inside and outside** | Inside: the credential core. Outside: cryptographic evidence, because a constraint binds only the database that holds it. |

> *A constraint binds the database, not a copy of the claim that has left it. The moment a credential is handed to a stranger, or an authority's history is read by an auditor, nothing in the schema travels with it. The next layer makes claims carry their own proof.*

# 5 Authenticating truth: signatures, custody, two witnesses and one migration

*An entry in the register is not proof of who made it.*

IN THE TOWN Every entry in the register carries the seal of the official who made it, pressed with a die nobody else holds. Two assayers, trained in different schools, examine each new seal, and a seal that only one of them recognises is not accepted.

## 5.1 What a signature is for

A digital signature is a seal that only the holder of a private key can make and that anyone with the matching public key can check. Polaris signs the SHA3-256 digest of a token value under the issuing agency's ML-DSA key and stores the signature beside the public key that made it. The claim a signature carries is narrow and permanent: *this key signed these bytes.* It says nothing about whether the token is still authorized, which is the next section's problem, and nothing about who holds the key, which is custody's problem.

The algorithm is ML-DSA-65 (FIPS 204, security category 3) by default and ML-DSA-87 (category 5) beside it; ML-DSA-44 is refused everywhere as below the floor. No signed body hardcodes its algorithm: the custodied key decides it before signing, because `algorithm` is itself a signed field, and the registry advertises the accepted set. SLH-DSA-128s and SLH-DSA-256s, the hash-based alternatives that do not rest on lattice assumptions, are registered in the algorithm table precisely because they rest on hash functions alone, so that a rotation away from lattices is a row update; no SLH-DSA signer is wired, and the posture ledger carries the gap. IMPLEMENTED for ML-DSA; PROPOSED for a hash-based signer.

The migration path does not depend on the algorithm's shape. In the lab, one population moves through seven signature families, from ML-DSA through FN-DSA and SLH-DSA to four candidates of NIST's additional-signature process (MAYO, UOV, SNOVA, CROSS), and no credential stands without a signature in force at any hop. What the shapes do change is cost: a public key copied beside every signature is affordable for ML-DSA and not for UOV, whose key alone is 447 kilobytes per row, so the signature row's design, not the migration, is where the next algorithm will press. None of the candidates is accepted by the product. EXPERIMENTAL

## 5.2 Custody

The private key never sits in application code. It sits behind a custody interface with three drivers: a file (development), PKCS#11 (a hardware security module or a software token; the key is generated inside the token and never leaves it), and a cloud key-management service. Continuous integration exercises the PKCS#11 driver against a software token that implements ML-DSA, not a hardware module, and rotates a key inside it: a token signed under the old in-token key still verifies after the switch while the new key signs. The KMS driver's envelope cryptography is real and its service is a stand-in. One production profile makes the module the sole signer: the application refuses to boot if a file key is present or real signing is unavailable. Which driver, which module, and who holds the key are decisions the readiness ledger assigns to a deploying organisation.

The placeholder deserves a paragraph, because it is where an honest system is most easily misread. Without the real-signing flag and a lattice library present, the signing module produces a deterministic SHA3-256 binding labelled `DETERMINISTIC-PLACEHOLDER-SHA3-256`: thirty-two bytes that verify against no key, where a real ML-DSA-65 signature is 3,309. That is the default in development and in most of continuous integration. It is guarded: a production boot fails closed unless real signing is actually available, the placeholder is a named development profile, and it warns loudly when used unnamed, and it is still the reason the repository no longer calls itself post-quantum without saying so in the same breath.

## 5.3 Two witnesses at issuance, one at use, sampled, and one outside corpus

The two-witness principle states that any cryptographic verdict Polaris ships must be checkable by a second, independent implementation, written separately, differently represented, sharing no code, and agreeing on the verdict or abstaining explicitly. A verifier that agrees with itself has established nothing. At issuance the signature just produced is verified by two independent FIPS

204 implementations, a lattice library and an OpenSSL-backed one, and is refused persistence unless both accept it; a missing witness at issuance is a refusal, not a downgrade.

Verification at use runs one witness, because a stored signature is already known to verify under both, and one witness detects tampering at about ten times the rate: on the reference machine roughly 7,950 verifications per second per core against roughly 740 two-witness. The fast path earns its speed by being continuously checked: a mandatory random fraction of successful checks is replayed through the second witness, a disagreement increments a counter and pages at the highest severity, and every response names which witness set ran.

One thing about the witnesses is not self-referential. On every push, the Project Wycheproof ML-DSA-65 verification vectors, a third-party corpus pinned to a commit in which accepting a signature the corpus calls invalid is a failure, run under both witnesses: 58 of 58 agree. That is evidence that Polaris's verification of the primitive agrees with an outside authority on 58 cases. It is not a wallet, not an interoperability result, and it says nothing about the credential protocol; the repository's scoreboard records it in exactly those words. EXTERNALLY EXERCISED for the primitive.

The same principle governs the zero-knowledge verdict (Section 8): a Python re-implementation with plain integers modulo a prime, sharing no code with the Rust prover, re-derives the membership statement and must agree, and it abstains, in the open, on the one axis it cannot model, the integrity of the proof bytes. What the principle catches is implementation divergence. What it does not catch is a shared misreading of the specification, and the repository says so beside every two-witness claim. The witnesses are also mutation-tested, so the claim that two implementations must agree is a claim about the tests as well as the code.

## 5.4 Canonical signing: the same bytes on both sides

Every signed artifact in Polaris is a JSON statement with sorted keys and no whitespace, digested with SHA3-256, then signed. The application builds the statement; the detached verifier (Section 6) rebuilds it from the artifact to check the signature. The two constructions must be byte-identical or a genuine artifact fails to verify, and a test, the canonical-equivalence oracle, pins every signed type on both sides. The one bug the oracle is there to catch was found in review: an artifact whose `algorithm` field was set after signing carried a signed field the signature did not cover.

## 5.5 Keys have a lifecycle

The key register is an append-only table of events, registered, retired, or compromised from an instant that may predate the discovery, from which a signed trust list is published. A verifier holding a trust list decides a key's status at any instant, so a credential under a compromised issuer key is rejected independently of the issuer's own manifest, which a thief holding the key could forge, and a document signed and timestamped before the compromise stays valid while one signed after does not. The two-instance drill records a compromise on one authority and watches the other's decision flip over the wire, with no cooperation from the compromised party.

## 5.6 Agility under an audited migration path, and what a break would cost

ML-DSA-65's security does not rest on a prediction about quantum hardware; it rests on Module-LWE and Module-SIS, the lattice problems the scheme is built on, being hard against the best known classical and quantum algorithms, which could be wrong for mathematical reasons. The two real risks are a cryptanalytic advance against the lattice assumptions and an implementation flaw, and the design already answers both, the hash-based hedge in the registry and the two witnesses at issuance. What the repository can demonstrate, and so what it claims, is *algorithm agility under an audited migration path*: C7 makes the algorithm a row, `AgencyAlgorithmAuth`

authorizes it per authority, and a population can be re-signed under a new parameter set on a path that runs on every push.

That path is measured rather than planned. `polaris migrate-population` re-signs the whole active population under a new parameter set, built out of the same constraints the per-token procedure relies on: the unique constraint is the serialization point, so two runners racing one credential produce one row, and sixty-four runners divide the population without coordinating. The number that matters is not throughput but how many holders hold a credential that verifies under nothing, and the drill samples it at every batch boundary rather than at the ends, because a gap that opens and closes between two endpoints is invisible to a before-and-after check. It is zero throughout. The old signature keeps verifying until its deprecation date, and that interval *is* the migration: deprecating the superseded algorithm is refused while any active credential is unmigrated, because a holder would otherwise discover the gap at a border while the console reported progress. Under real ML-DSA-87 with one custodied key the drill re-signs about 345 credentials per second on one runner, 91 percent of it signing, so a population of 350 million is about twelve days on one runner and four and a half hours on sixty-four; buying database capacity for it buys almost nothing.

What a break in Module-LWE would cost is written down in the readiness ledger, and this paper repeats it. It would not require a schema change, a change on the verification path, or a new trust model. It would cost every credential already issued under the broken algorithm, and one thing no agility buys back: during a cutover a credential carries a classical signature beside the post-quantum one, and the classical half is protected by nothing this design provides: once the classical algorithm falls, anyone can forge that half, and a verifier that still accepts it accepts forgeries. The risk is forgery, not disclosure; a signature encrypts nothing, so harvest-now-decrypt-later does not apply to the credential. That is why signing post-quantum now is worth doing for credentials that outlive the classical assumption, and why migration protects only the verifiers that stop accepting the classical half.

The repository's own lab then asked which side carries the agility. Relying parties take a new or retired algorithm by a software release: the detached verifier's accepted set is in its code, the registry publishes names, and the trust list carries per-key status. So a deployment plans a spare algorithm: a token never returns to an algorithm it already used (`one_signature_per_algorithm_per_token`), so a migration moves to an algorithm that already exists, is keyed and is not deprecated.

## 5.7 What remains classical

The token signature, the digests, the Merkle commitments, the zero-knowledge proof, the client-to-edge key exchange for modern clients and the hop from the application to the connection pooler are post-quantum, each of the last two read off a real handshake rather than inferred from a base image. The hop from the pooler to the database, the certificate signatures, and the operator's WebAuthn credentials (hardware-key sign-in) remain classical, each gated on a third party (a PostgreSQL release, the public certificate ecosystem, authenticator hardware) and each stated in the posture ledger with its threat and its exposure.

**Layer card: cryptographic evidence**

| | |
|---|---|
| **What it is** | ML-DSA signatures over SHA3-256 digests, stored with their public keys; a custody interface with file, PKCS#11 and KMS drivers; two independent verifying implementations and one outside corpus; a canonical statement format; an append-only key register and a signed trust list; a population migration that runs. |
| **Why it exists** | A claim that leaves the database needs to carry its own proof, a proof only one program can check is a promise, and an algorithm that cannot be replaced is a deadline. |
| **What it trusts** | FIPS 204 and 202 as standardised; the two libraries not sharing a bug; the custody backend the operator chose. |
| **What it refuses** | A verdict from a single implementation at issuance; an algorithm named in code rather than by the key; a signature stored without its key; a placeholder passing as a signature; a deprecation while any credential would be left verifying under nothing. |
| **What it can see** | The bytes it signs, which are digests: the signing path never sees a person. |
| **What it cannot see** | Whether the signed claim is still authorized (Section 7). |
| **If it fails** | A forged signature fails both witnesses; a compromised key is retired from an instant by the register and every verifier holding the trust list rejects what it signed afterwards; a broken algorithm costs every credential under it and is survived by re-signing, at a measured rate. |
| **Now or later** | Now. |
| **Inside and outside** | Inside: the schema, which stores the signature rows. Outside: the independent verifiers that check them without the issuer. |

*A signature the issuer's own server checks proves nothing to a stranger, because the stranger has to trust the server. The next layer takes the verification out of the issuer's hands entirely, and, for the first time, into the hands of software the issuer's author did not write.*

# 6 Verification without the issuer: the packaged verifiers, the conformance contract and the first strangers

*A verdict from the issuer is not a verdict anyone can check.*

IN THE TOWN A townsperson's papers can be checked at any inn on any road, by anyone who holds the town's published seal, without sending a messenger to the hall. The town is never asked to vouch for its own seal.

## 6.1 The detached verifier, as a thing a stranger installs

`polaris-verify` verifies a credential's authenticity pack with only a standard ML-DSA library: no Polaris code, no database, no network unless asked to fetch status. It is a package on a public registry, at `1.0.0-rc.3` (each package carries its own version, separate from the system's), a release candidate, with `0.1.0` beneath it since 15 September 2026, published by the repository's own workflow under trusted publishing so that no long-lived credential exists anywhere, and the script of that name in the repository is a shim onto it. It refuses to start until the run says what cryptography it is doing: `-pqc-provider oqs`, `cryptography` or `auto`, or `-dev-placeholder`, with no default and no environment variable, because a verifier that quietly stopped verifying still answers every question. Every machine-readable verdict carries a `crypto` field, and under the development flag the real backends are turned off, so a development run cannot produce a true verdict at all; building that gate found that the first version of the flag declared the mode without entering it, which is a label, not a mode.

It is the one Polaris component whose input is chosen by an adversary, because a relying party runs it on whatever a stranger presented, and it is the component through which every later protocol artifact is checked: manifests, epoch checkpoints, revocation feeds, status bundles,

status assertions, transparency heads, receipts, timestamps, signed documents, registries, trust lists, presentations, holder proofs, agent grants. Three independent ML-DSA implementations agree on the published vectors: a lattice library, OpenSSL, and a third in the TypeScript SDK. It is held *total* against hostile input by a metamorphic fuzzer, and an attack suite of executable adversaries runs beside it; a grep proves a line exists, an attack proves a defence holds.

Whether a stranger can actually install it is measured, not assumed. On every push, and before every publish, a drill builds the wheel, installs it into a throwaway environment with none of Polaris present, and runs it from a directory that is not the repository with every `POLARIS_*` variable stripped: the forbidden runtime surface (the web application, the database driver, the check layer) must be absent, the command must refuse to start without a declared crypto mode, a genuine ML-DSA-87 credential must verify and its tampered twin must not. It carries a negative control, a second wheel with a database import bolted onto the verifier, which the harness has to catch. This matters because every other suite in the repository runs from inside the repository with the repository on the path; an import that would have stayed green there fails on the first stranger's machine.

## 6.2 The SDKs and the conformance contract

A relying party installs one of two verify SDKs, `polaris-sdk-python` and `polaris-sdk-ts`, each answering one narrow question about a presented credential, is it authentic and is it authoritative right now, and never returning a person's data. Both are on their registries. The TypeScript one is a lesson in what a registry teaches: its first packaging pointed `exports` at TypeScript source, which Node refuses to strip inside `node_modules`, so the package could not have been imported by a single consumer on any Node version; the repository's tests ran inside the package directory, where stripping is allowed, and never noticed. It compiles to a distribution now, and the drill installs the packed tarball into a bare project.

Both are held to a language-agnostic conformance suite: a runner drives any verifier over standard input and output through the published cases, 118 at `1.0.0-rc.7` across twenty-five artifact types, including the composite federation trust decision, the holder proof, the agent grant and its revocation. Passing the suite is what *conformant* means, and the normative wire specification (RFC 2119 language, twenty-seven format strings, the exact signed-field list of every artifact) is what an implementation importing no Polaris code would build against.

The contract was then asked what it did not ask, by measurement, and the answer bounds what conformance means. Three artifact types had no case in which the signature was bad, so a verifier that never checked theirs conformed. Five of the published valid vectors had expired months earlier and no case noticed, because none asserted freshness. Asking about freshness surfaced two divergences between the shipped reference verifiers: neither bounded a holder proof's age, so an integrator following one got no replay protection on presentations, and neither could report whether an artifact's issuer was trusted at all, so a genuine signature by a stranger passed. All are fixed and the three verifiers agree on every case. What the contract still does not prove is stated where an integrator will read it: 44 verdict fields are unconstrained by any published case, and none can be closed by writing one, because each needs a conforming verifier to compute something it does not. One of them is not a decision at all: the window a trust attestation carries, echoed back beside the `expired` verdict that is the decision. The contract constrains what its weakest conforming implementation computes. Then the reference verifiers themselves were mutated: every refusal in each was inverted in turn, and eighteen of eighteen in the Python implementation and nine of fourteen in the TypeScript one accepted what they exist to reject with both that SDK's tests and the conformance runner green, including an agent grant's exhausted use count and, in the TypeScript SDK, a constant-time comparison whose length guard inverted made a five-byte value compare equal to a 32-byte root. All are closed in the SDKs' own tests,

and the drill inverts all 32 refusals on every push with a per-SDK negative control. The detached verifier, the package a stranger installs first, was the one verifier no drill inverted. On 23 September 2026 each of its 45 refusals was inverted against every instrument CI runs on it, the self-test, the published vectors, the attack suite, the fuzzer and twenty-four drills: 32 survived, among them the consistency and inclusion checks of the transparency log and the `cryptography` witness's refusal of a forged signature, which survived only because the second witness was installed beside it, where a stranger following the README has one witness. Each now has a test that drives it; seven are declared, each unreachable or refused again downstream, and the same drill now inverts this verifier on every change to it.

## 6.3 Versioning, and the frozen version 1

Protocol majors live in the format string (`polaris-registry/1`), minors are advertised by the registry and are never needed to verify, and the negotiation rule is normative: reject an unknown major, accept any minor, never emit an unadvertised version, answer an unsupported one with a specific error and the supported list. Version 1 of the protocol is frozen under checksums, with the detached verifier of `v9.317` vendored beside it, and a compatibility suite runs both directions on every push: today's verifiers must accept everything version 1 published (44 of 44 cases hold), and the pinned older verifier must never accept what today's suite rejects.

## 6.4 A verifier for wallets Polaris has never met

Everything above is Polaris verifying Polaris. One more verifier faces the other way: a door for wallets Polaris has never met. It is an OpenID4VP 1.0 verifier (OpenID4VP is the standard protocol by which a digital wallet presents a credential) under the High Assurance Interoperability Profile (HAIP), a role for which the OpenID Foundation's conformance suite has a test plan, `oid4vp-1final-verifier-haip-test-plan`, in which *the suite signs the credential*, so Polaris's own ML-DSA issuance is not involved. `polaris-oid4vp` is that verifier: it checks an SD-JWT VC (a signed credential whose fields can be disclosed one by one) signed with ES256 (a classical elliptic-curve signature), with holder key binding, delivered as an encrypted response the wallet posts back to the verifier (`direct_post.jwt`), and it refuses with a named reason, one per negative module of the plan, `issuer_signature`, `sd_hash`, `kb_signature`, `nonce`, `audience`, `kb_freshness`. It is a separate package because `polaris-verify` promises to open no socket and this one must listen. One of its fixtures is not self-referential: a complete response produced by the conformance suite's own wallet, built by a Java library this project does not use, is committed, decrypts, and verifies, and is refused a year later, under another nonce, under another audience, and with one disclosure rewritten.

This verifier is the one door through which software written outside the project has come: a conformance service somebody else runs, and a wallet somebody else wrote. What they found there is the last ring of this map (Section 20).

Two format bridges make the boundary explicit. A Polaris credential renders in the ISO/IEC 18013-5 mobile-document structure, and a reader that speaks the standard parses it, walks its namespaces and verifies every disclosed element's digest against the signed security object, with an independent implementation of CBOR, the standard's binary encoding, proving it rather than the one that wrote the bytes; the reader cannot verify the issuer signature, because it is ML-DSA-65 and the standard mandates classical algorithms, and the repository calls that the design rather than a limitation, since a post-quantum credential carrying a classical signature to earn a green tick would be a classical credential. A verification result also renders as a W3C Verifiable Credential, deliberately not an identity credential: its subject vocabulary is closed and refuses identity fields by name, and its subject identifier is the per-verifier handle or absent. Both are formats, not trust models.

**Layer card: independent verification**

| | |
|---|---|
| **What it is** | Four packages on public registries: a standalone verifier, two SDKs and an OpenID4VP verifier; a conformance runner over 118 published cases; a normative wire specification; a frozen version-1 set with a cross-version suite; a fuzzer, an attack suite and three mutation drills; two format bridges. |
| **Why it exists** | A verification the issuer performs asks the relying party to trust the issuer's server. Authenticity has to be decidable by anyone, anywhere, with the network off, and by software the author did not write. |
| **What it trusts** | The published issuer keys the relying party chose to trust (its anchor set); a standard ML-DSA library, or for the OpenID4VP path, a registered X.509 trust anchor and ES256. |
| **What it refuses** | Any Polaris code or database; any input it cannot parse; any object presented to the wrong type's verifier; any version it was not told about; to start without a declared crypto mode; a nonce or audience it did not itself ask for. |
| **What it can see** | The artifact and the anchor set. For an SD-JWT VC, exactly the disclosed claims. Nothing else. |
| **What it cannot see** | Whether the credential is still authorized, unless a status artifact is presented with it. |
| **If it fails** | A wrongly accepted mutation fails the fuzzer; a refusal that stops refusing fails the SDK mutation drill; a drift between specification and signer fails `check_wire_spec_matches_code`; a package that cannot be installed by a stranger fails the boundary drill; a defect every internal instrument missed is what the outside is for. |
| **Now or later** | Now, for the software and for one outside wallet on one path. |
| **Inside and outside** | Inside: the signatures. Outside: the status layer, which answers the question a signature cannot. |

> *Signing a credential proves who signed it. It does not prove whether that credential is still authorized: the seal on a revoked passport is as genuine as the seal on a valid one. Polaris therefore needs status.*

# 7 Verification is not authorization: status, revocation and the offline trade

*A credential's authenticity is not its current authorization.*

IN THE TOWN A seal can be genuine and the traveller still unwelcome. The board by the gate, not the seal, says whose papers are honoured today, and a copy of the board carried along the road is trusted only until it goes stale.

## 7.1 Two results, never one

The whole product is visible in one object. A relying party asks the issuer about a credential and receives, for a revoked token:

```
POST /api/v1/verify  ->  {
  "authentic": true,                 // the ML-DSA signature is genuine
  "currently_authoritative": false,  // but this token has been revoked
  "status": "REVOKED",
  "usable": false,                   // authentic AND authoritative -> false
  "decision": "reject"
}
```

Authenticity is permanent and cacheable. Authorization is fresh and revocable. A relying party needs both, and the API refuses to collapse them into one flag, because a signature stays genuine forever while the authority under it can be withdrawn at any moment.

The relying-party endpoint authenticates the caller as an organisation, not a person, by an OAuth2 client-credentials grant; it accepts a possession proof rather than a sequential identifier, so it cannot be used to enumerate tokens; it returns a uniform verdict on a mismatch, so it is not an existence oracle; and it keeps no record of who verified whom.

## 7.2 Fresh state, not a stale replica

The verify endpoint is routed to a read replica for throughput, and the repository's history records the mistake that split the two results permanently. Authenticity is a property of immutable material, the signed value, the signature bytes and the stored key, so a lagging replica can neither forge it nor deny it. Whether the token is usable *now* is a current-authorization question, and a revocation lands on the primary; a replica inside its staleness window could still show a just-revoked token as active. The status that decides `usable` is pinned to the primary, the response names its source and carries `as_of` and a zero staleness bound, and a caller may cache authenticity and must re-read authorization.

## 7.3 Revocation

Revocation is a status transition through one sanctioned procedure that also writes to `RevocationList` in the same transaction, so the verifier-facing list can never diverge from the token's state, and a trigger refuses any other path into `REVOKED`. The list carries a canonical reason vocabulary; the audit trail carries the procedural detail and the co-signer when the issuer-discretion ceiling required one. Revocation looks forward: events recorded while a credential was valid are not invalidated, because history is not rewritten.

## 7.4 Authorization with the network off

An offline verifier cannot ask the issuer. Polaris answers with a short-lived signed status assertion: the issuer's signature, under the same key that signed the credential, over the token's current status and a window. The holder fetches it by presenting the genuine credential, with no bearer and no personal data, and staples it to a presentation. A verifier accepts offline only if the credential is authentic, the assertion is authentic, bound to this credential, fresh on the verifier's own clock, no wider than the window the verifier will accept, and `ACTIVE`. The rule that a verifier never accepts a credential more than one window after its revocation, and that the window is the one the verifier set rather than the one the issuer minted, is now a formal model checked on every push with a configuration it must fail.

The cost is stated as plainly as the benefit. Because verification touches no issuer, the issuer never learns it happened: offline is more private than online. And because it touches no issuer, a revoked credential's last assertion stays valid until it expires. Issuer non-observation, offline availability and revocation freshness cannot all be maximised at once; a deployment picks two,

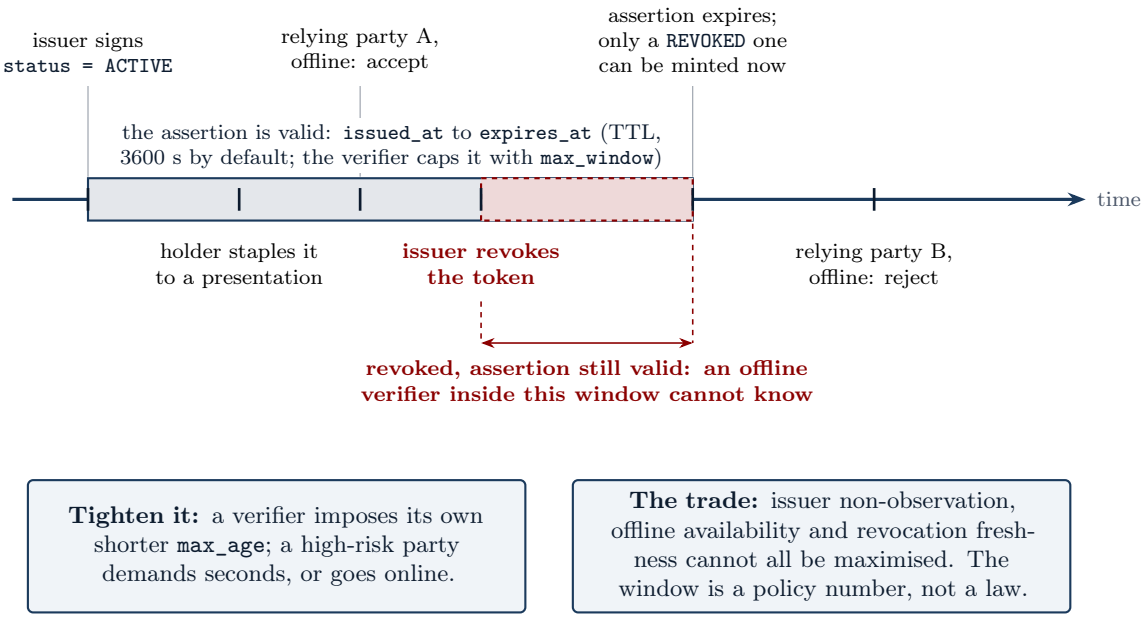



**Figure 5:** The offline trade, drawn as a timeline. The issuer signs a status assertion with a window; the holder staples it; a relying party verifying offline inside the window accepts. If the issuer revokes the credential inside the window, the last `ACTIVE` assertion stays valid until it expires, and an offline verifier cannot know. The window is bounded by the issuer's time-to-live (an hour by default) and by the `max_window` the verifier imposes; a high-risk relying party demands seconds or goes online. Who chooses the window, and who bears it: the issuer sets the time-to-live and each relying party its `max_window`; the harm of a long window falls on whoever relies on a credential revoked inside it, and on the holder whose revocation was meant to protect them. Polaris makes the window explicit and bounded; how long is acceptable is a governance decision it does not make.

and the window is a policy number, not a law. One consequence is a setting for the relying party: the issuer mints a one-hour window, the reference verifier's own default bound is unset, and the assertion check does not read the ceiling the artifact carries, so the relying party's configuration sets part of a status's maximum offline lifetime. A relying party that needs seconds sets seconds or goes online.

## 7.5 Status across authorities, and through networks that are not trusted

Two more signed objects carry status between authorities without a callback the issuer could log. An epoch checkpoint is an authority's commitment to the latest point on its epoch chain, the epoch number, its Merkle root and the prior epoch it extends; two checkpoints prove monotonicity, and two different roots signed at one epoch number are cryptographic proof that the authority equivocated about its own history. A revocation feed is the sorted set of revoked-credential leaves, each the SHA3-256 of a token value, derivable only by the holder; a newer feed that drops a published revocation is a caught rollback, because the underlying table is append-only. At federation scale an aggregator mirrors many authorities' feeds and checkpoints, verbatim and still under each member's own signature, into one short-lived bundle. The aggregator is untrusted for correctness: in full control of the bundle, recomputing the commitment and re-signing the envelope, it still cannot forge a member's status, because it cannot sign as the member. The bundle adds availability, not trust.

Signing a status artifact is also what lets an untrusted cache carry it, and a cache introduces the one failure signing does not prevent, which is time. A cache directive is never a constant: every windowed artifact's `max-age` is its own remaining life, computed from the `expires_at` the issuer signed, so a cache cannot outlive the window the issuer committed to; every artifact that names one credential is `no-store`, because a shared cache holding one would serve one holder's credential to another; and an expired or unparseable window is not cached at all. Distribution at content-delivery scale is thereby specified and drilled; no deployment has it. IMPLEMENTED for the artifacts, drills and cache rules; PROPOSED for a distribution deployment.

The epoch cadence is the single number that sets three unrelated properties, and a design record now says which: revocation freshness, since a revoked credential drops out of the *next* epoch; the size of the crowd, since the anonymity set is the epoch; and how often a relying party's one-human-once ledger resets. Closing an epoch was once the expensive part, and it was expensive for a bad reason: a fixed-depth tree padded to the national depth of 24 materialised sixteen million leaves whatever the population was, so a root over a thousand members took 10.8 seconds and 2.9 GB. The padding is one repeated value hashed once per level, and a root over a million members takes 2.1 seconds in tens of megabytes; the binding constraint on cadence is the anonymity floor per context, not the compute.

**Layer card: verification and status**

| | |
|---|---|
| **What it is** | The relying-party verify endpoint, the revocation list, the signed status assertion, epoch checkpoints, revocation feeds, the aggregate status bundle, and the cache rules that let an untrusted network carry them. |
| **Why it exists** | Authenticity is not authorization. A verifier needs to know not only that a claim was made but that it still stands, and sometimes needs to know it with the network off. |
| **What it trusts** | The issuer's signing key, for the assertion; its own clock, for freshness; the primary database, for the online verdict. |
| **What it refuses** | A replica's opinion of current status; an assertion wider than the verifier's ceiling; an aggregator's word for a member's status; a feed not bound to the issuer's key; a cache entry that would outlive the window the issuer signed. |
| **What it can see** | The status of one credential at one instant; the set of revoked leaves. |
| **What it cannot see** | A revocation that happened after the assertion it holds was signed; the active population, which no feed publishes. |
| **If it fails** | A stale verdict is bounded by the window, and the bound is the verifier's; a forked epoch chain or a rolled-back feed is non-repudiable evidence against the authority; a lying aggregator is caught at the member's signature. |
| **Now or later** | Now, for the artifacts and the rules. |
| **Inside and outside** | Inside: the verifiers. Outside: the privacy boundary, which decides what a verifier may learn while it checks. |

*Status proves current authority, but proving it can reveal far more than the verifier needed, and every verification is a fact about a person's day. The next layer bounds what anyone learns, and this version reports what was measured about that bound rather than what was intended.*

# 8 The privacy boundary: what a verifier is allowed to learn

*Current authorization is not privacy.*

IN THE TOWN At some gates the guard learns only that the traveller belongs to the town, and not which townsperson is standing there; what the guard may write down is decided by the town, not by the guard.

Two things in Polaris use zero knowledge, one thing deliberately does not exist, and the distinction between them is the most important sentence in this section. Version 3 adds a fourth thing: an adversary, written to falsify the claim, and what it found.

## 8.1 Three disclosure levels

Every verification event is recorded at one of three levels. `ZERO_KNOWLEDGE` proves that a valid token exists without recording which: the event's token identifier is null, and a check constraint refuses any row that says otherwise (C2). `SELECTIVE` records named attributes only. `FULL` records the identity and is logged and rate-limited. The level is decided by the server and cannot be upgraded by the client (C6): a relying party cannot turn a zero-knowledge check into a full one by asking differently, and the Atlas, the operators' bounded console (Section 14), redacts a zero-knowledge event's location server-side. No other page of the application returns that location except the admin and auditor SQL console, which runs the caller's own query; that console and the owner-role archive export return it as the row stores it. The default level is zero-knowledge.

## 8.2 Issuer-side unlinkable verification records

The first use of zero knowledge is a property of the issuer's database. A default-mode verification stores no token identifier, so the verification graph, who was verified where and when, cannot be reconstructed from the database even by someone holding all of it. This is a structural claim, not a promise: the repository carries a redaction proof that the redacted field is non-derivable under a stated adversary model, property tests run a read-only adversary that may query every stored

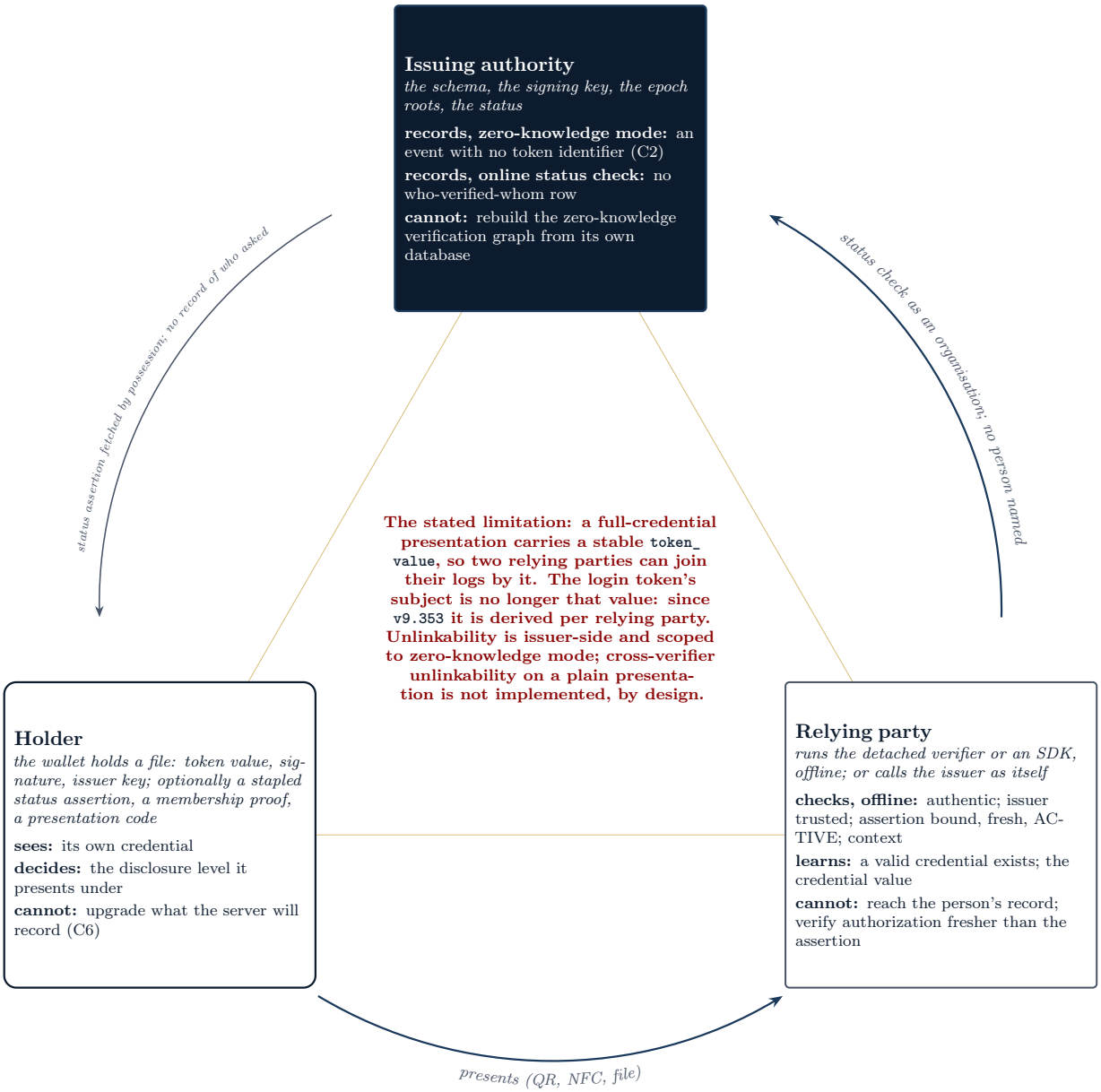



**Figure 6:** Holder, relying party and issuer, with what each sees and what each is prevented from seeing. The holder presents a file; the relying party decides offline with the detached verifier or an SDK and learns that a valid credential exists and its value; the issuer records, in zero-knowledge mode, an event with no token identifier, and keeps no who-verified-whom row for the online status check. The three flows form a circle turning one way; at its centre, in red, is the stated limitation: on a plain presentation the credential value is a stable handle across the verifiers it is shown to.

row, and the claim that two relying parties pooling everything they hold still cannot link a holder is a formal model checked on every push. The property test that measures reconstruction risk runs an adversary against isolated zero-knowledge events and asserts its success rate is no better than chance, and the assertion is shaped so that a weak adversary cannot flatter the result. The claim is scoped precisely. It covers zero-knowledge-mode events. Selective and full events do carry a token identifier, because that is what those levels mean.

## 8.3 The membership proof

The second use of zero knowledge is a proof object. A Merkle tree behaves like a cryptographic fingerprint of an entire set: instead of handing a verifier the whole set, one can hand it a short proof that a particular item belongs to the set with the committed fingerprint. Polaris commits, per epoch, a Merkle root over the active-token set, and a holder can produce a Plonky2 proof (a hash-based zero-knowledge proof system) on their own device, against a published anonymity set, that their token is a leaf under that root, bound to the epoch, a context and a nonce, without revealing which leaf. The verifier sees the proof bytes and the public inputs and cannot recover the leaf; that is the proof system's property, not an API rule. The setup is transparent, with no ceremony, and the commitments are hash-based. The default tree depth of fourteen covers the schema's ten-thousand-leaf epoch cap, so the anonymity set is the epoch, and the epoch has a floor: an epoch smaller than the minimum anonymity set (a database setting, twenty unless set; the notional sample sets one and says so) is refused rather than closed smaller, and a proof against a smaller one answers *privacy unavailable* rather than verifying. The authority waits for members or lengthens its cadence; epochs are not merged. The detached verifier applies the same floor offline to a foreign authority's epoch, from the size its signed checkpoint states. On the reference machine a proof takes about 25 ms to make and about 12 ms to check, and the verdict is two-witnessed at the statement level. That second witness is independent in implementation only: it is a separate program, written by the same author from the same reading of the circuit, so it catches an implementation slip and not a misreading of the statement. The two signature witnesses (Section 5) are independent in implementation and in authorship, and share only the standard. Neither pair is independent in its interpretation of the specification. The proof carries a per-verifier nullifier, a value that repeats if the same holder proves twice in the same scope and reveals nothing else, `Poseidon(secret || scope || epoch)`, Poseidon being a hash function designed for proof circuits, so a relying party can enforce *one enrolled identity, once* (not one human: C3 does not see a human enrolled twice, Section 3) within its own scope while another cannot correlate its nullifiers.

What the proof does not do is stated with the same care. It proves membership and binding, nothing else: whether a token is active, permitted for the context and unrevoked is decided when the epoch is composed, not inside the circuit. The nonce prevents substituting a captured proof into another context; online, a single-use nonce store refuses a replayed bundle, and offline the verifier keeps no state, so a relying party that needs single use issues a challenge. The nullifier does not hide the holder from the *issuer*, which derives every leaf in order to build the tree and could compute any member's nullifier in any scope; the property is between relying parties. And the repository's soundness ledger says not to protect real identities with the circuit over the proof library as shipped. The proof library has been evaluated against its successor and kept, with the triggers that would reopen the question recorded. The circuit is drilled on every push to main: the Plonky2 crate's tests under an 85 per cent line-coverage floor, a prove and verify round trip checked against a second witness written in Python, and a mutation drill that switches each witness off and requires a suite to fail.

## 8.4 Bounded, not permanent, and what bounded was found to mean

Polaris is a full-credential presentation system, and the consequence would be flat if the login token's subject were the credential's hash: the same value at every relying party, so that two of them could join their user tables exactly and forever. The subject and the presentation handle are derived under the relying party's own scope, stable where an account needs it and unrecognisable at the next verifier; since the subject is the value a relying party *writes down*, and written-down values are the ones pooled, sold, subpoenaed and breached, this is the half that matters most in practice. The other half did not change: a plain presentation still shows the verifier a stable

`token_value`, the issuer's signature and the holder's public key, so the guarantee is about what a verifier should *store*, not about what it is *shown*, and the verifier reports which it is giving: `exposed` for a plain presentation, `bounded` for the zero-knowledge path.

Version 3 can say what `bounded` was measured to mean, because under the operating contract (the rule, Section 18, that counts only what outsiders do as progress) the repository's lab asked the question directly: given complete transcripts from two presentations, what advantage does a colluding pair of verifiers have at deciding *same holder*? Three findings, in the order they were made.

**The verdict did not mean what it said.** `bounded` was reported whenever a scoped nullifier was present. It never checked the premise its own documentation stated. A presentation carrying the nullifier *and* the credential's stable token value, issuer key and signature was reported bounded while two verifiers could link it with a single string comparison. Fixed the same day: `bounded` now requires the nullifier and the absence of every field that is identical across verifiers, including the holder's public key, which is stable by construction and is exactly why the pairwise handle is derived from it rather than shown. The fix corrected a verdict; it made no presentation more private than it was.

**No advantage beyond the anonymity set, and the set has a floor.** An adversary that flattens every scalar in a transcript, drops what is constant across the population and matches on the rest, with a positive control that links the exposed population at an accuracy of one, achieves on the bounded population an accuracy that tracks $1/N$ at every size from two holders to two hundred. Within an epoch, scoping does what the word implies against this adversary. And $N$ is the epoch's membership. The schema constrains an epoch's committed count to between one and ten thousand, and the minimum anonymity set floors it: an epoch below the floor, twenty unless a deployment sets another, is not closed, and a proof against a smaller one answers *privacy unavailable*. Below a floor the measurement shows what is at stake: an epoch of three members gives a colluding pair a one-in-three guess, and an epoch of one identifies the holder outright, while the verifier still, correctly, reports the transcript as bounded. Above the floor the count is readable off the signed checkpoint, and no verdict reports it. Reporting it would be a new product guarantee, which lab work does not get to create, so it is recorded as a limitation for a deploying organisation.

**What the adversary cannot see.** It matches on exact equality and nothing else. Shown a population that is perfectly linkable with no equal field in it, it stays at chance. So the null result means no field is *identical* across verifiers, not that none is *correlated*. The one channel it can read is transcript length, which carried the entire above-chance residue in that control; nothing in a bounded Polaris presentation varies in size per holder today, and a deployment where disclosed attribute sets or optional elements made it vary would open it. Presentation size, timing, issuer metadata, status artifacts and transcript structure are unmeasured, and the lab says so rather than extrapolating.

The path a stranger's wallet actually used cuts the other way. An SD-JWT VC presentation over OpenID4VP, measured the same way, leaves nine values identical across two verifiers, the holder's key, the issuer's signature, the disclosure digests, the hash of the presentation and its length, all issuer-signed so the holder cannot vary them. That is a known property of presenting one such credential repeatedly; the ecosystem's mitigation is batch issuance, many credentials with one holder key each, which the repository does not implement. Adopting the ecosystem's format imported the ecosystem's linkability; it was not visible before it was measured, and it is not a reason to stop interoperating. So cross-verifier correlation is bounded on the native zero-knowledge path, measured within the limits above, and exposed on the interoperable one.

*Unlinkable by default* still means the issuer's zero-knowledge-mode records. IMPLEMENTED for the derivations and the measurement, with the residue stated rather than closed.

## 8.5 Bounded surfaces

The remaining privacy mechanisms bound what a surface can return rather than what is stored. Every map and API aggregate is hard-capped (C8), so the whole population cannot be extracted through the operator's atlas, and a request for six and a half million cluster bins returns at most five thousand. The single-entity investigation pages are built to refuse cross-individual link analysis and are audited: a meta-audit table records who read which audit surface, with what filter, and how many rows came back, and it is append-only. The authority's most invasive power, pulling one named person's whole verification history under a warrant, writes the same row, and a structural rule extends that to any read hidden behind a procedure or a view, where a search for a `SELECT` would not find it. The authority-structure layer, Athena, is forbidden by `check_athena_no_person` from referencing any natural-person table or column (Section 14). And timestamps, the timestamp authority and the exchange gateway are built to learn nothing about what they touch, which Section 12 describes.

**Layer card: the privacy boundary**

| | |
|---|---|
| **What it is** | Three disclosure levels with a CHECK behind the default; server-side enforcement; the redaction proof and its adversary; the epoch membership proof, its nullifier and its second witness; the per-verifier handle; bounded aggregates; an audited single-entity investigation surface. |
| **Why it exists** | A verification log is a surveillance backbone unless it is structurally unable to be one. |
| **What it trusts** | The C2 constraint; the proof library's soundness; the redaction proof's adversary model, and the linkability adversary's stated blind spots. |
| **What it refuses** | A client's claim about its own disclosure level; a token identifier on a zero-knowledge row; an unbounded read; a person node in the authority layer; a verdict of `bounded` over a transcript with any field identical across verifiers. |
| **What it can see** | At zero knowledge: that a valid token was verified at a time in a context. At full: the identity, logged. |
| **What it cannot see** | Which token, in zero-knowledge mode, even from the whole database; and the size of the crowd a given proof hides in, which no verdict reports. |
| **If it fails** | An identifier on a zero-knowledge row is unstorable; a cap removed fails the build; a person reference in Athena fails `check_athena_no_person`; an adversary weakened to flatter the result fails the property suite's own control. Cross-verifier correlation on a plain presentation is not a failure; it is the documented boundary. |
| **Now or later** | Now. Linkability-resistant presentation for the interoperable path, and a crowd-size floor, are a different system and later. |
| **Inside and outside** | Inside: status. Outside: the holders and relying parties who use the credential. |

> *A credential bounded in what it reveals still has to be used: held, presented, accepted and logged in with. The next layer is where the credential meets software that is not the issuer's, and where, for the first time, that software was not the author's either.*

# 9 Using the credential: holders, relying parties and the login that keeps no record

*Being verified is not being followed.*

IN THE TOWN Townspeople carry their own papers. A merchant learns what the merchant needs and no more, from papers the customer chooses to show, and two merchants comparing their ledgers cannot tell that they served the same person.

## 9.1 The wallet is a protocol, not a product

The reference holder agent, `scripts/polaris-wallet.py`, holds a credential as a file, shows it, verifies it offline through the detached verifier, presents it, proves membership in zero knowledge against a published epoch, signs a document on the holder's behalf, delegates a bounded authority to an agent (Section 17) and logs in to a relying party. Every message it emits is specified so that any client can emit the same. Native mobile and desktop clients and a browser bridge are product engineering that a pre-pilot system on notional data does not attempt; they would follow the protocol, and never lead it. The physical presentation path, a card that signs a device's challenge, is the emulator and verifier device of Section 3. IMPLEMENTED for the protocol and the reference script; PROPOSED for native clients.

A presentation is an unsigned wrapper around signed things: the issuer-signed credential, an optional stapled status assertion so that authorization is decidable with no connectivity, an optional membership proof, an optional holder signature over the verifier's nonce, the context and disclosure level presented under, and an opaque presentation code. The verifier decides it offline and reports the code as present or absent without ever interpreting it, because a duress code is matched only by the issuer, out of sight; a duress presentation is byte-for-byte the same shape as a normal one. For transfer by QR or NFC the presentation is compressed and split into frames that each name the digest of the whole; a receiver takes them in any order and refuses, before parsing anything, frames naming different digests, a missing index, a conflicting duplicate, or a reassembled payload whose digest does not match. The decoder is resource-bounded: at most 9,999 frames, 512 KiB compressed and 2 MiB inflated, so a decompression bomb is refused at the limit rather than decompressed. The bounds were added after an outside review named the gap.

## 9.2 The relying party

The reference relying party (`scripts/polaris-relying-party.py`) decides accept or reject for a presentation by combining offline authenticity with online status, refusing a revoked, tampered or foreign-issuer credential and staying blind to duress, and an end-to-end drill runs the whole wallet-to-relying-party matrix under real signatures every release. A relying party registers with an issuer as an organisation with a scope that a schema constraint bounds to exactly `verify`, `authenticate`, or both, so that a fourth kind of access is a constitutional change rather than a configuration one. Identity never becomes a login product by accident. A relying party is also an object about which decisions are recorded: registering one, turning off the zero-knowledge step-up its holders had to satisfy, disabling or re-enabling it, each writes an append-only event, a change that weakens what the party must satisfy is flagged as such and refused without a stated reason, and a change made from a database shell is recorded on the same terms as one made through the tool, because the writer is a trigger and not the tool. The first version of the rule that decides what counts as a weakening ranked a capability by the length of its name, so granting a capability the party did not hold read as harmless; it was found by running it against a loaded database, not by reading it.

## 9.3 Logging in without a login record

The auth broker lets a relying party offer *log in with a Polaris credential* through an authorization code with PKCE. The holder presents the credential at its issuing authority's instance by possession, and the relying party exchanges the resulting code for an identity token signed by the issuing agency, which it verifies offline. Four guards keep this from becoming a login record. The token's subject is the per-relying-party handle of Section 8, never a person identifier and not the same value at every relying party. The token carries no claim beyond the vocabulary: context, disclosure level, the assurance reached, enrolment status, instants. The authorize route writes nothing, and the token endpoint's only write is the code's hash into a register that holds no subject and no relying party. And duress is served identically.

A relying party's demands, a required zero-knowledge step-up or a required enrolment status, bind the authority and do not travel only in the holder-side request: the policy is registered on the relying party's row and applied first, a request may add a requirement and never remove one, and a context other than the registered one is refused. The credential must be permitted in the context the token names, and a requested enrolment status applies alongside the registered one. The authorization code is encrypted as well as signed. And the conformance contract was later found to verify an identity token's signature and never its audience or nonce, so a verifier that ignored both, and therefore accepted a token minted for one relying party at another, passed every case; four cases fixed it.

## 9.4 The stranger's path

A reader can reach one accepted presentation from software nobody here wrote, on a clean machine, in about ten minutes, without cloning the repository: install `polaris-oid4vp` from the registry, start the published wallet image at its pinned digest, generate the verifier's keys, give the wallet a credential bound to a key it generated itself, run the verifier, present. The page that says how was executed from an empty directory before it was written down, which caught its own defect on the way (a buffered pipe that left the verifier's log empty), and it ends with the line that matters: `<- 200 authentic, claims ['cnf', 'family_name', 'given_name', 'iat', 'iss', 'vct']`. If it fails for a reader, the page says, that is the most useful bug the project can receive, and the repository's issue tracker is where a walker reports it.

**Layer card: use**

| | |
|---|---|
| **What it is** | The wallet protocol and reference script, presentations and QR framing, the reference relying party, relying-party registration with a bounded scope and a recorded history, the auth broker, and the stranger's path to an accepted presentation from an outside wallet. |
| **Why it exists** | A credential nobody can hold, present or accept is a database row. This layer is where the credential meets software the issuer does not run. |
| **What it trusts** | The signed objects inside a presentation; the issuer's key for the identity token; the relying party's registered policy. |
| **What it refuses** | To interpret the presentation code; to write a login record; to accept a scope outside the CHECK; to let a holder's request weaken the relying party's registered demand; to lower a relying party's bar without a reason on record; to decompress an unbounded payload. |
| **What it can see** | A relying party sees the credential value and the verdicts, or on the zero-knowledge path a proof and a scoped handle. The authority sees a possession proof and consumes a code hash. |
| **What it cannot see** | The authority cannot see who logged in where; the relying party cannot see the person's record. |
| **If it fails** | A replayed code hits the consumed register; a wrong presentation, an unmet step-up, a verify-only bearer at the broker, a stale or tampered frame are each refused in the drills; a weakened relying-party bar with no reason is refused by the database. |
| **Now or later** | Now, as a protocol, and now for one outside wallet on one path. |
| **Inside and outside** | Inside: the privacy boundary. Outside: federation, because a relying party rarely trusts only one authority. |

*A relying party that trusts one authority is easy. A society has many authorities, none of which should be able to speak for the others, and none of which should hold every person's record. The next layer is how independent authorities recognise one another without collapsing into one.*

# 10 Federation: independent authorities that recognise one another

*Federation is not institutional independence.*

IN THE TOWN Other towns keep their own halls and their own dies. A road between two towns runs one way and for one purpose, a town honours another's traveller only because it has said so in writing, and no town's trust passes on to a third.

## 10.1 Why there is no centre

Two shapes were possible. A central instance holds every person's identity and every agency's tokens under one trust root, with agencies as tenants. A federated shape gives each issuing authority its own instance and its own signing key, records cross-authority trust explicitly, and lets a relying party verify against published keys with no central service in the path. The decision record chooses the second and states that it is not a free choice: the constitution names the federation trust graph as load-bearing, says no agency holds a monopoly, and refuses population-scale aggregation structurally. A central instance is a monopoly by construction and a single store to seize, subpoena or mine. The federated shape is deliberately the more expensive one; its cost is the inter-authority protocol, and the cost is accepted.

## 10.2 Trust is a row, not a chain

An attestation is one row of `AgencyTrustAttestation`: *verifying agency V accepts issuing agency I in context C, until a date.* Verification across agencies succeeds if and only if exactly one active row matches the verifier, the issuer and the context. The lookup is a single non-recursive query. It never computes a closure, so V trusting I never implies V trusts what I trusts; cycles cannot arise because nothing walks the graph; and an operator who wants multi-hop trust declares every hop, so the audit trail holds every decision rather than the inputs to an inference. Self-attestation is refused by a check constraint. Revocation sets a date and a reason together, once, and looks forward: events recorded while the edge was active are not invalidated.

The row records which Polaris operator created the attestation, and under real signing it also carries the attesting authority's own signature over the canonical `polaris-trust-attestation/1` statement, so the trust graph rests on a signature rather than on an operator's word.

## 10.3 The three signed objects an authority publishes

A relying party needs three things from a foreign authority, and all three are signed views over append-only tables, published at short-lived endpoints and consumed offline by the detached verifier. The federation manifest carries the authority's own anchors, every key it has ever used with its real status, and the attestations it has made, each with the attested key. The epoch checkpoint is its commitment to the latest point on its epoch chain. The revocation feed is the set of revoked leaves. A manifest must be self-consistent, signed by one of the active anchors it declares, so a stranger's key cannot sign one; authentic, under two witnesses; fresh, within a window the consumer caps; and trusted, from an authority whose anchor the relying party chose. The cross-authority decision then reads: accept if and only if the credential's signature is genuine, the trusted manifest attests to that signing key in the presented context, and the issuer's feed, fail-closed, does not list the credential.

A holder can also prove membership in a foreign authority's epoch in zero knowledge. The proof already carries the epoch root as a public input; the root already travels the federation signed in the checkpoint. The decision composes the two: trust the foreign checkpoint through the graph to obtain a trusted root, require the proof to bind to that root, epoch and context, then check the proof by shelling to the prover binary as a local subprocess, with no network. If the binary is absent the verifier abstains, which is never a false accept. The verdict names an authority and a decision, never a token.

## 10.4 One instance, many authorities, and holder isolation

A Polaris instance can hold many agencies; the schema permits it and the shipped seed demonstrates six. Inside such an instance an operator is bound to one authority, and the database, not the query, keeps the others' credentials and holders out of that operator's sight: row-level security hides them from a bound session, the views run with the caller's rights, and the routines that must see past the binding to enforce a rule do so as the owner and return nothing the binding would hide. A deployment that needs holder isolation runs one instance per authority.

## 10.5 Two instances, and what they prove

A dedicated CI job boots two separate instances as two authorities, each with its own database and its own real ML-DSA root, talking only over HTTP, and drives the whole matrix: cross-verification from a fetched manifest, attestation revocation flipping the decision, epoch checkpoints and revocation feeds fetched and verified over the wire, a key compromise recorded on one side reversing the other's acceptance, the exchange gateway, the timestamp service read out of a fetched registry, the broker's login, document signing through the wallet, and a mixed pair of parameter sets (ML-DSA-87 on one side, ML-DSA-65 on the other). It is red on any wrong decision.

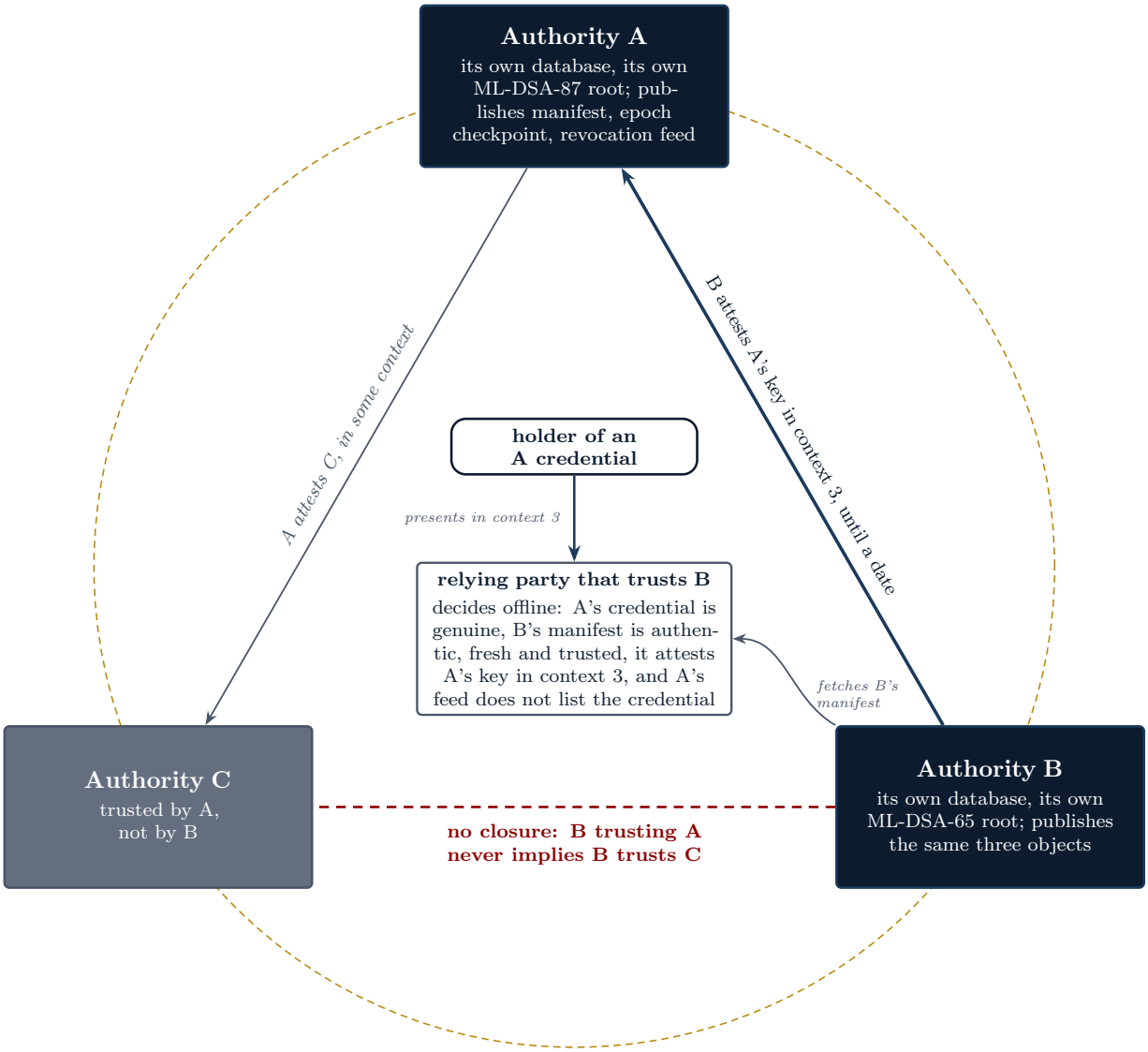



**Figure 7:** Two authorities and one directional attestation. B attests A's key in one context; a relying party that trusts B accepts A's credential in that context, offline, from B's manifest and A's revocation feed. A also trusts C, and that gives B nothing: trust is never a closure. In CI this runs as two separate instances over HTTP under a mixed pair of parameter sets.

What the job shows is that two instances, each with its own root, exchange attestations and verify each other.

**Layer card: federation**

| | |
|---|---|
| **What it is** | Per-authority roots, directional per-context attestations, the signed manifest, epoch checkpoint and revocation feed, the aggregate status bundle, cross-authority zero-knowledge membership, per-authority operator isolation, and the two-instance drill. |
| **Why it exists** | No agency may hold a monopoly, and no single store of every person may exist; yet a relying party must be able to accept a credential issued elsewhere. |
| **What it trusts** | The anchors the relying party itself chose; the append-only tables the signed objects are views over. |
| **What it refuses** | Transitive trust; a central service in the verification path; a manifest not signed by its own declared anchor; an aggregator's word for a member's status; a callback the issuer could log; an operator reading another authority's holders. |
| **What it can see** | Published trust data: keys, attestations, epoch roots, revoked leaves. No person. |
| **What it cannot see** | The active population of any authority; who verified whom. |
| **If it fails** | A forged manifest fails self-consistency; a forked epoch chain or a rolled-back feed is non-repudiable evidence; a compromised issuer key is retired from an instant by a trust list the issuer does not control; a shared instance's owner is outside what any filter can bound. |
| **Now or later** | Now, as a protocol between instances. |
| **Inside and outside** | Inside: use. Outside: transparency, because a signed history can still be told differently to different people. |

*Signed federation objects prove what an authority said. A dishonest authority can still say different things to different observers, publishing one epoch chain to an auditor and another to a relying party. Polaris therefore needs transparency: a way for observers to prove that history only ever grew, and to compare notes.*

# 11 Watching the watchers: logs, witnesses, the split view and the public report

*A transparency log is not protection from a split view.*

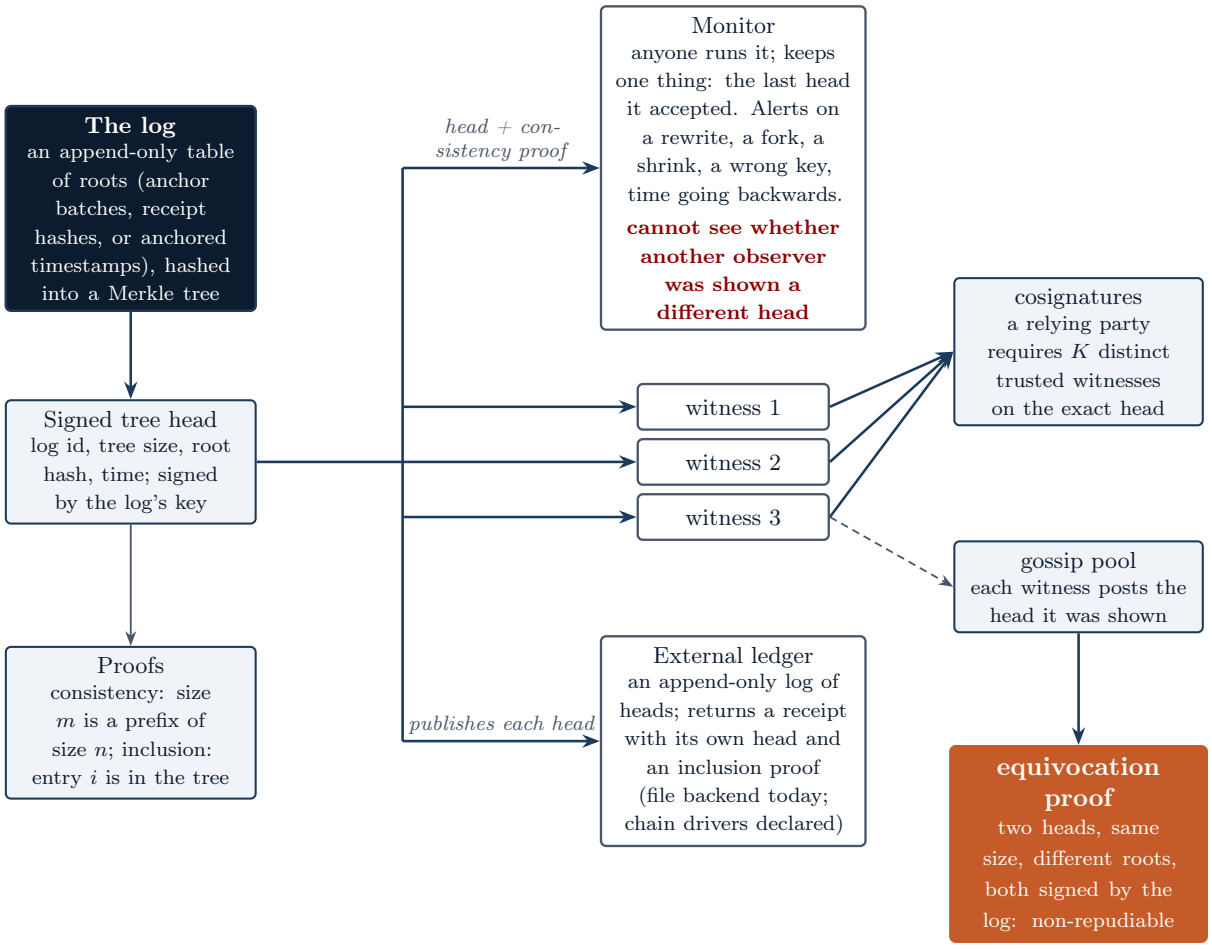
IN THE TOWN Each evening the watchtowers publish what the hall recorded that day, and cannot quietly take it back. Witnesses outside the wall copy every page and compare notes, and a tower that shows two visitors two different pages is caught when the witnesses meet.

## 11.1 The gap after anchoring

Polaris periodically commits a batch of audit rows to a Merkle root and records the root in an append-only table, so that an outside verifier can prove that the history under a root has not been revised since. That leaves one gap. A single root proves the rows under it; it does not prove that the operator did not later discard a root and publish a different one in its place. A transparency log closes the gap by committing to the whole sequence of roots and letting anyone prove that the sequence only ever grew.

## 11.2 The log

The log's entries are the anchor roots in append order; nothing new is stored, because the underlying table is already append-only. Over those entries the application builds a Merkle tree in the style of Certificate Transparency (RFC 6962), with SHA3-256 and distinct prefixes for leaves and interior nodes so that one can never be presented as the other. It publishes a signed tree head, the log's commitment to its entire history at a size; a consistency proof that the size-$m$ tree is a prefix of the size-$n$ tree, which exists only if the first $m$ entries were never reordered, rewritten or dropped; an inclusion proof for any entry; and the entries themselves, so that a monitor can replicate the log and recompute everything. The same machinery publishes two more logs: the




*The protocol for independent witnesses is built and drilled under real signatures.*

**Figure 8:** The log, the monitor, the witnesses, the gossip pool and the equivocation proof. The log publishes a signed head and proofs. A monitor proves the log only appended. Witnesses cosign heads they found consistent, a relying party requires a threshold of them on the exact head, and witnesses gossip the heads they were shown; two log-signed heads of the same size with different roots are a non-repudiable proof that the log lied. Heads can also be published into an external append-only ledger that returns a receipt.

hashes of minted exchange receipts, and the hashes of anchored timestamps. There is no personal data in any of it.

## 11.3 The monitor, and what it cannot see

A monitor is a standalone program anyone runs. It keeps one piece of state, the last head it accepted; each cycle it fetches the current head, verifies the signature against a key it was given out of band, fetches a consistency proof from its last head to the new one, and either records the new head or alerts on a rewritten entry, a fork, a shrunken tree, a head signed by the wrong key, or a timestamp that went backwards. What it proves is consistency: the operator cannot rewrite history without every monitor that saw the old head detecting it. What it cannot prove on its own is that two monitors were shown the same head at the same size. A log can show one observer a private fork of history.

## 11.4 Witnesses, gossip and the equivocation proof

A witness is a monitor that does two more things. It cosigns a head with its own key, only when the head is an append-only extension of the last one it cosigned, and it gossips the log-signed head it saw into a shared pool. Two uses follow, both verified offline. A witnessed checkpoint is a head carrying cosignatures from at least $K$ distinct trusted witnesses, so a split view needs $K$ independent witnesses to equivocate, not just the log. An equivocation proof is two heads for one log, both validly signed by the log's key, at the same size with different roots; the log signed both, so it cannot repudiate the lie, and two gossiping witnesses produce the proof the moment they compare notes. A witness that is itself shown a second head at a size it already cosigned refuses and writes the proof.

An external ledger completes the picture: the log publishes each head into an independent append-only ledger and receives a receipt, the ledger's own signed head plus an inclusion proof, so the log cannot use a head it has not publicly committed and the ledger cannot later drop it. A ledger is itself an append-only log, so this reuses the same machinery and is monitored the same way. The file-backed ledger is implemented and tested; the chain-backed drivers are declared and wait on their interfaces. IMPLEMENTED for the file backend; PROPOSED for a chain.

## 11.5 The public report: saying out loud how often the power was used

The logs prove that history was not rewritten. They do not tell the public what the history contains, and one part of it matters more than the rest: an identity authority asks to be trusted with the power to pull one named person's entire verification history under a warrant, and every control around that power, the warrant, the role check, the disclosure model, is invisible from outside. The authority can publish a transparency report that says how often it used that power, built from the audit-access record the read now leaves (Section 8). Building it began by asking what was recordable and finding the answer was nothing, because the read had left no trace of itself. Small counts are withheld rather than published as zero, because a report with a cumulative total lets a reader subtract one period from the next and recover the cell, so the report carries no running total to subtract from; a drill on every push attempts exactly that recovery and must fail. The report is a surface where the authority makes its own most invasive act countable by strangers.

## 11.6 Who runs the witnesses

The strength of this defence in practice is exactly the independence of whoever runs the witnesses. What is built is the protocol: the cosignature format, the threshold rule, the equivocation proof, and a witness daemon anyone can run, all drilled under real signatures every release with three witnesses catching a fork two ways.

**Layer card: transparency**

| | |
|---|---|
| **What it is** | Three RFC-6962-style logs over append-only tables; signed heads; consistency and inclusion proofs; a monitor; witnesses with cosignatures, a threshold and gossip; the equivocation proof; external-ledger publication with receipts; a public report of the authority's own warrant reads. |
| **Why it exists** | A signed object proves what was said; only a watched sequence proves that what was said was not later unsaid; only compared notes prove it was said the same way to everyone; and only a published count lets the public ask how often the watched thing happened. |
| **What it trusts** | The log's key for the head; the witnesses' keys and their independence, which is a deployment fact; the authority's own record of its reads. |
| **What it refuses** | A head with no consistency proof from the last one; a head not cosigned by the threshold; a receipt the ledger did not record; a second head at a size already seen; a report from which a small count can be recovered. |
| **What it can see** | Roots and hashes, and counts of warrant reads. Never a person, never a payload. |
| **What it cannot see** | On its own, a split view: that is what the witnesses are for. |
| **If it fails** | A rewrite fails every monitor; a fork is a non-repudiable proof; a dishonest witness is one of $K$; a dropped published head fails the ledger's own consistency; a recoverable small count fails the report drill. |
| **Now or later** | Now, as a protocol with a drill. |
| **Inside and outside** | Inside: federation. Outside: the institutions that exchange, sign and timestamp under this watch. |

*With authority, status and history checkable by strangers, institutions can build on the credential: exchange requests, sign documents, attach time. The danger is that an exchange fabric becomes the very message log the vocation forbids. The next layer is built as its inversion.*

# 12 Institutions: exchange, time and signatures without a message log

*A receipt for an exchange is not a copy of the letter.*

IN THE TOWN When one town's hall answers another's request, it hands back a receipt saying that a lawful exchange took place, under whose authority and when, without saying what the letter contained. The receipt proves the exchange, not its contents.

## 12.1 The inversion

A conventional exchange fabric between institutions makes an exchange provable by logging the exchanged messages: a mediating node signs and timestamps the traffic so that a receipt can later be shown to a third party. The evidence is real, and so is its cost: a store of who asked what about whom, which is exactly the population-scale record the vocation forbids. The Polaris exchange fabric is the inversion. It proves the same three things a message log proves, that an exchange occurred, between these parties, and was authorized, while retaining none of the payload, by committing to the request and the response by hash and never by content. Every primitive in it, signatures, the detached verifier, the transparency logs, the trust graph, already existed; the fabric composes them.

## 12.2 The registry: discovery, not configuration

A consumer learns what an instance offers and trusts from one signed artifact: the protocol formats and versions it speaks and its accepted algorithms, each service's kind, path template and authentication, the transparency logs, every federated authority with its key register, the contexts and the proof each requires, the in-context trust graph, and the relying parties by name and scope only. Every fact is a view over the authority layer; the registry adds no truth of its

own. It must be signed by the active key it lists for its own publisher, so an impostor cannot publish one in an authority's name, and it is non-transitive: it describes its publisher's instance, never another's. The two-instance drill reads a service's path out of a fetched registry and calls it. No token, no holder, no verification record appears: a map of the institutions, never of the people. One thing the registry does not carry is recorded in Section 5: the accepted algorithms appear as a bare list of names with no status, so a verifier reading it cannot learn that one has been deprecated.

## 12.3 The gateway and the receipt

The receipt commits to the requester's key, the responder, the context, the two body hashes, which attestation authorized the requester, and the instant. The mint endpoint accepts only hashes, so the application cannot retain what it is never given. A third party verifies offline that the receipt is authentic and that the requester was authorized in that context, given the manifests it trusts. A party that holds a body can confirm the hash binds; a party that does not still holds the responder's signed account. One correction from the outside review belongs here: a receipt alone cannot prove the requester took part, because a dishonest responder can sign any receipt it likes; the requester's own signed envelope beside it can, and the wire specification names the pair as the evidence. The set of receipts is itself a transparency log, so an institution cannot later deny a receipt existed and anyone can see how many it minted, while each receipt stays in the hands of its parties.

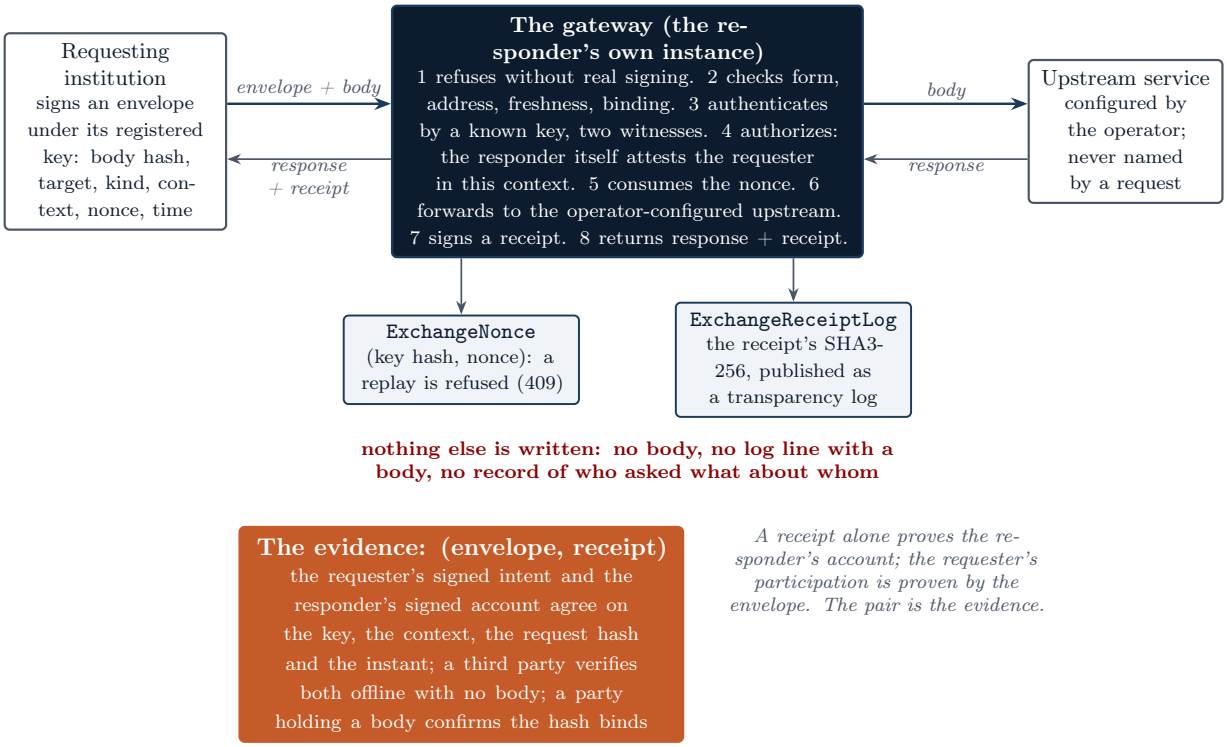



**Figure 9:** An institutional exchange through the gateway. The requester signs an envelope under its registered key; the responder's own instance authenticates it by a key it already knows, authorizes it through its own attestation in the envelope's context before anything leaves the process, consumes the nonce in an append-only replay register, forwards the body only to an upstream the operator configured, and returns the response with a signed receipt. Nothing but the receipt's hash and the consumed nonce is written. The evidence of the exchange is the pair of envelope and receipt, verifiable offline by a third party with no body.

Authorization at the gateway is directional. The responding agency itself must attest the requester's key in the envelope's context; an attestation by any other agency on the instance authorizes nothing there. Before that fix the gateway accepted an attestation from anyone on the instance, which was the transitive trust the federation layer refuses, reappearing one layer up.

## 12.4 Time that learns nothing

The timestamp authority binds an arbitrary SHA3-256 digest to an instant under an authority's registered key. It accepts a digest, never content, and keeps no per-request record, because a timestamp authority that logs every request is a store of who timestamped what and when. The evidence is the signed statement in the requester's hands. A caller may choose to anchor: the timestamp's own hash joins an append-only, witnessed log and the inclusion evidence comes back stapled, so that a timestamp backdated under a stolen authority key is one absent from every witnessed head of its claimed era. What the authority then retains is one digest and one instant, a record of volume and timing, never of content, and the default request still leaves nothing.

## 12.5 Signing a document so that it outlives the key

A holder in Polaris may hold a key (Section 17), and a holder need not, so the document-signing model stays notarial: the issuing authority signs on behalf of a holder who has just proved possession, exactly as it asserts status for one, and records who by hash. The authority's signature says that a holder of this credential, active at this instant, asked it to sign this digest, and the embedded evidence lets anyone check that claim later.

Long-term validation is where the outside review and the repository's own drills sharpened the claim. An authentic timestamp is not a trusted one, since anyone can sign one, and a signer's own timestamp is backdatable by whoever holds the key. So the strong claim requires the timestamp to be trusted by the verifier, from a key distinct from the signer's, active at the instant per the trust list, and, if the verifier asks, anchored in a witnessed log or corroborated by a quorum of independent authorities. A verifier given no timestamp anchors reports the facts and claims nothing. Anchor verification is in the detached verifier and both SDKs, so an implementation that imports no Polaris code can decide long-term validity.

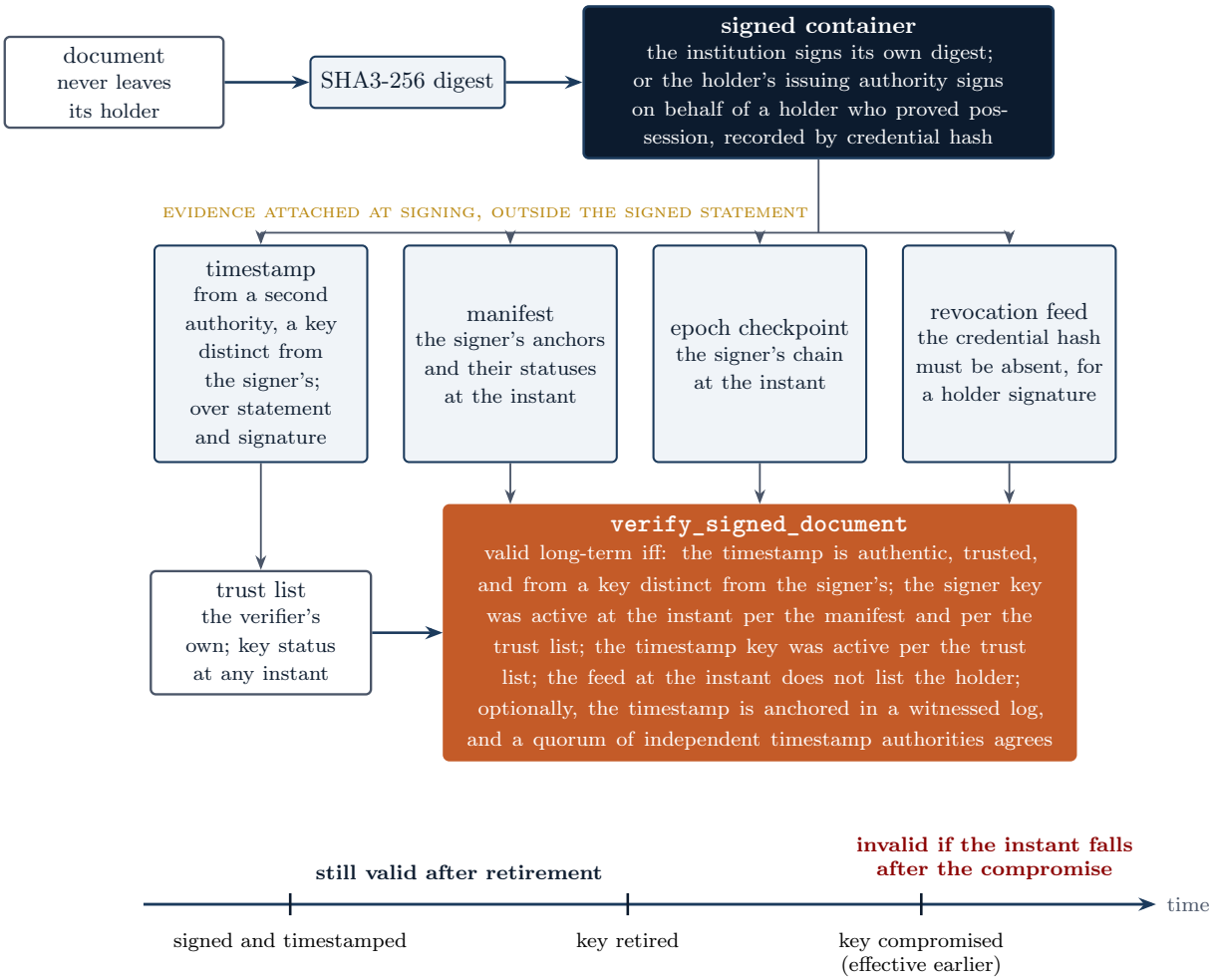



**Figure 10:** A signed document and its long-term evidence. The document never leaves its holder; its digest is signed by an institution, or by the holder's issuing authority on the holder's behalf after a possession proof, recorded by credential hash. Evidence fixed at the instant of signing, a timestamp from a distinct key, the signer's manifest, checkpoint and feed, together with the verifier's own trust list, lets a verifier decide validity at that instant long after the signing key has been retired. A signature whose instant falls after a compromise is refused even though its embedded manifest says the key was active.

**Layer card: institutions**

| | |
|---|---|
| **What it is** | The signed registry, the exchange gateway, the exchange receipt and its log, the timestamp authority with optional anchoring, document signing with long-term validation, the trust list. |
| **Why it exists** | Real software ecosystems need discovery, mediated requests, signatures and time. Each of those is conventionally built on a log of content, and the vocation forbids that log. |
| **What it trusts** | Registered institutional keys; the in-context trust graph; the verifier's own timestamp anchors and trust list. |
| **What it refuses** | A body at the mint endpoint; a URL named by a request; a replayed nonce; an attestation not made by the responder itself; a timestamp from the signer's own key as the strong evidence; a placeholder signature as authentication. |
| **What it can see** | Digests, keys, contexts, instants. |
| **What it cannot see** | The exchanged messages, the timestamped content, the signed document: none are ever given to it. |
| **If it fails** | A forged receipt fails the responder's signature; a dishonest responder's receipt is contradicted by the missing envelope; a stolen timestamp key is caught by the witnessed anchor or the quorum; a compromised signer key invalidates only what it signed after the instant. |
| **Now or later** | Now, between two instances. |
| **Inside and outside** | Inside: transparency, which watches the receipt and timestamp logs. Outside: the operations that keep all of it running. |

*Every layer so far is a claim about correctness. None of it matters if the system falls over under load, loses a database, or cannot be deployed by anyone but its author. The next layer is survival.*

# 13 Surviving reality: the stack, failure, load, the standing witness, and the day a pilot ends

*A correct system is not a system that survives.*

IN THE TOWN A town that is right about everything and falls down in the first storm protects no one. So it rehearses its fires, keeps a second hall in another valley that takes no decisions until the first is gone, and measures how long its gates stay shut.

## 13.1 The stack

The production topology is five services: a self-built TLS edge that negotiates a hybrid post-quantum key exchange with modern clients, the application under gunicorn, a connection pooler, PostgreSQL with continuous archiving, and Redis for the shared rate limiter. Every service runs as a non-root user with every capability dropped, third-party images are pinned by digest, and every production container drops the test framework. Four deployment paths share one secrets discipline, one backup path and one threat model: a laptop launcher, a single-host compose profile with a blue-green rolling deploy, a scripted Linux install under systemd exercised on two distributions, and a Helm reference profile with default-deny network policies and the restricted pod security standard, booted in CI on a one-node cluster.

## 13.2 Failure, drilled and measured

The repository's rule is to exercise rather than read: ten of ten defects found in one sweep were invisible to reading. Every operational claim is a drill that runs in CI and commits a number: high availability under a live write stream, the loss of a region, replicas, partitions, backup and restore, chaos under traffic and rolling deploys, each against a ceiling. The drills and their numbers are listed in Appendix A.

## 13.3 Load: what was measured, what was projected, and what a number rests on

Three kinds of number appear in the repository, and the paper keeps them apart. *Measured* numbers come from a named machine, an Apple Silicon laptop with eight cores, and carry a version stamp. *Simulated* numbers come from the national simulation harness, a synthetic country driven through the real procedures at a stated scale factor. *Projected* numbers are arithmetic over measured ones and are labelled as such by the repository itself, under a named assumption. No production cluster has been measured.

Table 6, in Appendix A, lists every performance number with its kind and the version that measured it.

The roadmap's planning targets (ten million persons for a state; 350 million, five thousand sustained and fifty thousand peak verifications per second nationally) were checked against a computational throughput model, and both halves of the answer are recorded. The model is built from single-machine measurements under stated assumptions; nothing has run at national scale, and the first bottleneck no measurement reaches is operational (replication, pooling and partition maintenance under a real multi-instance load). Every throughput target clears by more than an order of magnitude: the peak needs about six and a half cores of single-witness verification, the sustained rate under one, and a surge of two hundred thousand enrolments a day is nine minutes of one signer. And the system could not have run for a week at the sustained rate, because the verification event's identifier was a 32-bit sequence: two billion rows at five thousand a second is five days, twelve hours at the peak, and nothing is slow when a sequence is exhausted, every insert on the path simply fails. Five surrogate identifiers are 64-bit now, a check recomputes the arithmetic from the live schema on every push, and the rule the repository drew from it is that a capacity question answered in cores and seconds is the easy half. The model also refuses to call a core count *measured*: the per-core rate is measured, the national figure is a model, and every extrapolation carries its assumption by name. A separate cost model finds that verification throughput is not the cost driver at any realistic national scale, since a hundred million people verified twelve times a year is thirty-eight verifications per second on average, and states which of its inputs are facts about the code and which are guesses about a deployment. A national workload has been simulated at a scale factor and driven through the real constraints. EXPERIMENTAL

## 13.4 Abuse, secrets, supply chain, and decisions that leave a record

Per-agency quotas on issuance, revocation and verification are opt-in rows enforced by a trigger on every write path, exact under concurrency by an advisory lock, refused loudly on every surface, and keyed by agency identifiers, never by a person; a quota is a decision with a reason that survives being changed rather than a setting that can be overwritten. Velocity alerts compare each agency to its own trailing week. The per-IP rate limiter shares one Redis bucket across workers. Operators authenticate with WebAuthn as a second factor after an enrolment deadline, sessions are registered server-side with per-role caps and origin checks, and a per-role network policy applies. Creating an operator account leaves a record, and every audit event type the schema names has a writer, pinned by a check. Secrets are file-mounted, sealed and rotated through one lifecycle, and a database password reaches the database before its file changes, or is not rotated at all; the application refuses to boot in production on a default key, and the application log records no form body, cookie value or token value, which a check holds by reading the logging calls. Every release ships bills of materials with signed provenance, and CVE scans of the dependency surface and of every self-built image gate the build.

## 13.5 The standing witness, and whether it watches

Twenty CI jobs run on every push, and most carry a comment naming the past failure they exist to prevent. They run the product suite and the invariant layer; drive the Atlas console in a headless browser; build and boot the images; build the edge and prove the post-quantum handshake against a real certificate; page a duress alert through a real alert manager to a webhook; validate the tracing wire and the dashboards; kill and restore the primary; install the Linux profile on two distributions; sign in-token through a software PKCS#11 module with two witnesses; roll a deploy under traffic; evacuate a region; fail the HA profile's leader, lease store and etcd member under a live write stream; boot the Kubernetes profile and fail its database over; sign and verify with real ML-DSA-65; federate two instances over HTTP; install `polaris-verify` as a stranger would; type-check, test and conform the TypeScript SDK; scan dependencies and every image for vulnerabilities; and boot the full production stack end to end.

The drills are themselves watched. A drill refuses to report a guarantee it did not test, and a check holds every drill to counting its cases; every CI job installs what it runs, so a job fails on a finding rather than on its own setup; and the local gate a maintainer runs before pushing runs what CI runs, names the verification a change needs from the paths and route handlers that moved, and repeats the web suite as the production role and under the real signer.

## 13.6 The day a pilot ends, and the day the old credential stops being accepted

Two operations were written down as procedures that run rather than as paragraphs, because each is the promise an institution actually says yes on. A pilot's real promise is not that it will work but that it can be wound back, and that promise fails quietly: erasure becomes a paragraph in a consent form nobody executes. The wind-down revokes every pilot credential and replaces every name with a meaningless marker under the append-only erasure record, the consent language a participant is shown is generated from what the code does rather than written from what would be reassuring, and the whole procedure runs against a real database on every push. The pilot pack that an institution owes before it starts is closed with it.

A new national credential arrives beside a driving licence and a passport, and for years it is the second thing somebody carries. The dangerous moment in that period is not the cutover but the sunset, the day the old credential stops being accepted, because that is the moment the new one becomes compulsory without anybody having decided to make it so. The coexistence plan treats the sunset as a decision with a floor under it rather than a phase that arrives, and its verdict refuses to be computed from what the issuer can see: whether relying parties accept the credential and whether an alternate path exists and is usable are attestations an operator makes, and the system records them rather than guessing.

Two mappings sit beside these. The NIST SP 800-63 mapping states, row by row, what an assurance claim would rest on, every citation resolved and run by a drill so that a row cannot claim evidence that is not there; it is explicitly not a conformance claim and no assessment has been performed. And accessibility is enforced on every push for the third that automation covers, and named as not done for the rest, because an identity system a person cannot operate is one that excludes them from identity.

**Layer card: operations**

| | |
|---|---|
| **What it is** | The five-service stack on four substrates; HA, a standby region, replicas, partitioning, backup and disaster recovery, chaos, rolling deploys; abuse controls, secrets, supply-chain gates; twenty CI jobs and two scheduled drills; the pilot wind-down and the coexistence plan. |
| **Why it exists** | A correct system that cannot survive a lost disk, a lost region, a bad deploy or a hostile client is not infrastructure, and one that cannot be wound back is not one an institution should start. |
| **What it trusts** | The drills' ephemeral hosts as a lower bound on production behaviour; the operator's placement of hosts and regions and the custody choice. |
| **What it refuses** | A claim without a drill; a number without a stamp; a core count called measured; a drill that reports a guarantee it did not test; a root-running container; an unpinned image; a fixable critical CVE. |
| **What it can see** | Metrics, traces and alerts about the system. The correlation id on every request never touches the audit of record. |
| **What it cannot see** | A person: quotas, alerts, metrics and logs are keyed by agency, route and worker. |
| **If it fails** | A drill that misses its ceiling fails the build; a chaos or DR finding becomes a check; a region evacuation that invents or skips a row fails; a production incident is what the readiness ledger's operator decisions exist to prepare for. |
| **Now or later** | Now, on drills. |
| **Inside and outside** | Inside: everything above. Outside: the system's view of itself, which an operator needs without a view of people. |

*An operator running all of this needs to see what the system is doing and to understand where authority comes from, and must be able to do both without the system becoming a map of its people. The next layer is how Polaris observes and understands itself.*

# 14 How Polaris observes and understands itself

*Watching the system is not watching the people in it.*

IN THE TOWN The town keeps a clerk who counts what passes through its gates and a map of who answers to whom; neither keeps a list of the townspeople.

Two cognitive layers exist, two are proposed, and one line separates all of them from the people the system serves. The design rule is stated in the repository in one sentence: make power legible, not people.

## 14.1 Atlas sees

The Atlas is the operator's investigation surface: an analytical console that opens on a bounded, non-geographic overview, a volume time-series with failures overlaid, top-K breakdowns by context, agency, disclosure and outcome, a two-dimensional pivot, a records grid, and a map kept as a tab for what a map is good at, one region's drill or one subject's journey. Every view is a bounded server-side aggregate, never the raw events: at most five thousand cluster summaries for a viewport, however fine a grid the client asks for, at most 240 time buckets, at most fifty categories, keyset pagination so that a deep page stays cheap. A zero-knowledge verification is counted in volume and in the disclosure tallies and is never located, because it carries no token to attribute it by. The single-entity investigation pages are built to refuse cross-individual link analysis, and every read of an audit surface is itself recorded. A live-simulation mode is driven in a headless browser in CI and the console is asserted to actually stream. The benchmark times every bounded aggregate at a tenth of the event stream and at all of it, minutes apart on the same hardware, and grades each against the row-count growth, because a timing at one scale says an aggregate was fast once and cannot tell a roll-up that tracks the rows from one going quadratic;

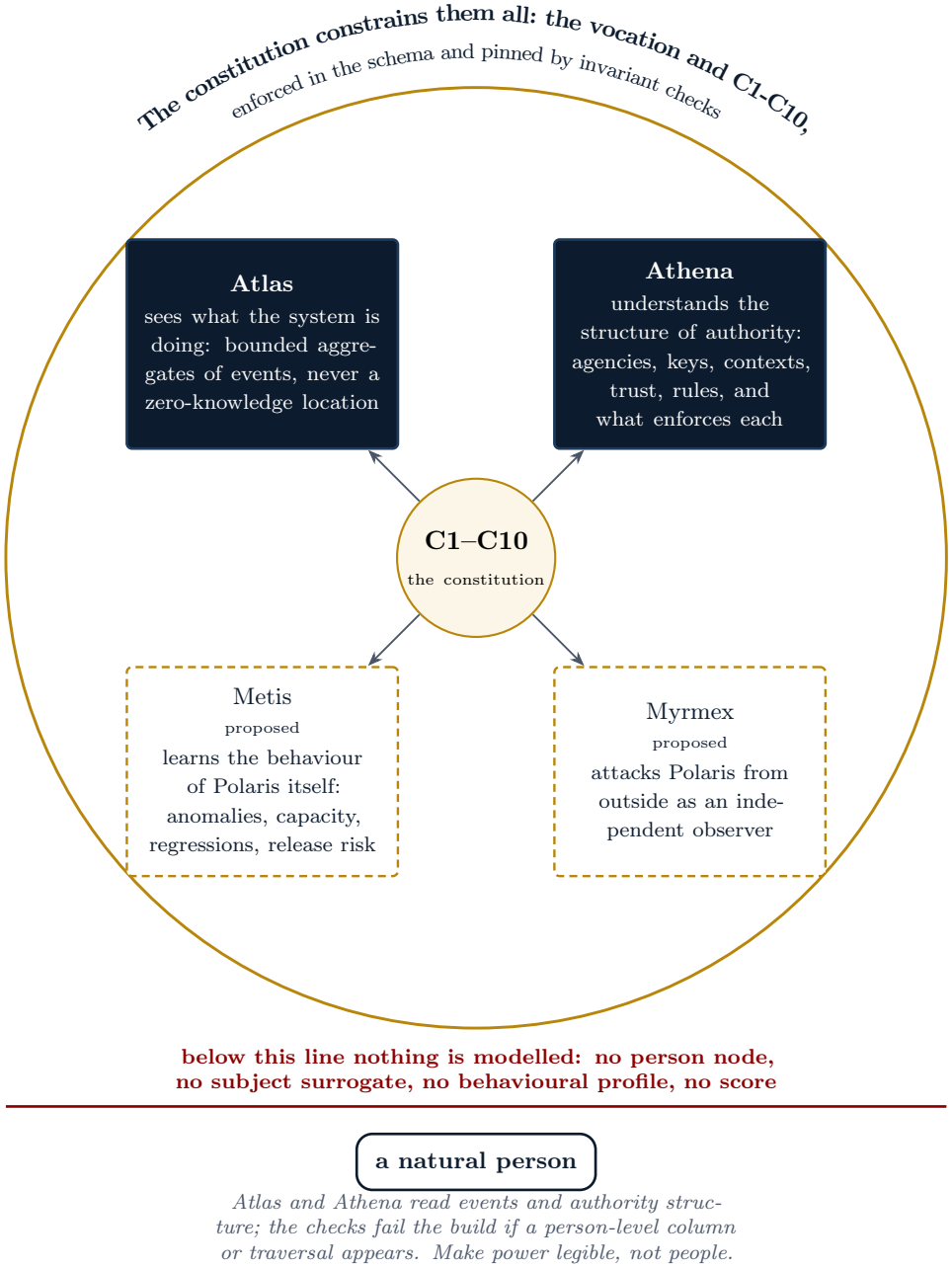



**Figure 11:** The cognitive layers. Atlas sees what the system is doing; Athena understands the structure of authority; Metis, proposed, would learn the behaviour of Polaris itself; Myrmex, proposed in this paper, would attack it from outside. The constitution constrains all of them. The red line is the person-prohibition: nothing above it models a natural person, and `check_athena_no_person` fails the build if a person-level column or traversal appears in the authority layer.

every aggregate grows slower than its data, and the benchmark exits non-zero if one ever outruns it.

## 14.2 Athena understands structure

Athena is a read-only semantic and provenance layer over the authority tables: agencies, algorithms, contexts, trust attestations and retention policies, as ten object views and eight edge views, with four functions that answer the governance questions the tree could otherwise answer only by hand. Why may this agency issue under this algorithm? What proof and disclosure policy bounds this context? If this algorithm is deprecated, who is affected? Which mechanism enforces this constitutional rule? The constitution itself is data: C1 to C10 and the vocation are rows, and a table maps each rule to the exact live mechanism that enforces it, a trigger, a partial unique index, a named check constraint, a check function or a procedure. A check fails the build if any named mechanism no longer exists, and a database test proves each is present in the running catalogue, which closes the drift between prose and code that the constraint lattice alone could not. The signed registry of Section 12 is a view over Athena, so the map of the institutions that an outsider fetches is the same map the operator inspects.

Athena is safe to exist only because person-legibility is made structurally impossible, not merely discouraged. Five invariants gate it, each with an adversarial detection test: no Athena view, function or table references a natural-person table or column, and no column is a stable per-person surrogate; every function is read-only, so *the graph says revoke, therefore revoked* is impossible; every function bounds its output; every authority edge resolves to an existing authority table, and the current views exclude revoked authority, so Athena describes and never owns; and every rule maps to a mechanism that exists. The same ship that created Athena removed the earlier person-aggregating views. If the prohibition ever obstructs a feature, the repository's own assessment says that is the signal to stop, not to relax it.

## 14.3 What must never be built here

No person, household or relationship object. No subject handle, opaque or otherwise, stable across contexts. No traversal from a zero-knowledge event to a subject. No population segmentation or scoring. No action: the layer reads and never writes. Those are the kill criteria of the design study that endorsed a constrained Athena, and they are the boundary any future cognitive layer inherits.

## 14.4 What a learning layer may never learn

The roadmap carries a captured brief for a future, advisory-only learning subsystem that would learn Polaris itself: infrastructure anomaly detection, capacity forecasting with uncertainty, regression and release-risk intelligence, chaos learning that proposes experiments and never injects them, root-cause assistance over Athena's dependency graph, and aggregate institutional anomalies of the form *this agency normally revokes 0.8 percent a month and now revokes 4.9*. Its central hypothesis is to be falsified before any code: machine learning may infer properties of the system and must never infer, rank, score or predict the rights of natural persons, and renaming a person to a subject does not satisfy that if the data stays linkable. Its design law is advisory only: observe, model, predict, explain, recommend, and never predict then revoke, deny, issue or change a constitution or a production configuration. The brief proposes the name Metis, because the obvious alternative collides with the metrics stack, and records that a rule that works as well as a model should be preferred to the model. Nothing of it is built; under the operating contract of Section 18 nothing of it will be until an outside party needs it. PROPOSED

Over all of these layers stands the constitution: the vocation and C1 to C10, enforced in the schema and pinned by the checks. It is not a component; it is what every component is constrained by.

**Layer card: system cognition**

| | |
|---|---|
| **What it is** | Atlas, the bounded operational console; Athena, the read-only authority-and-constitution layer; and two proposals, Metis and Myrmex. |
| **Why it exists** | An operator must see what the system is doing and understand where authority comes from; a system this large cannot be governed by hand. |
| **What it trusts** | The append-only events and the authority tables it reads. |
| **What it refuses** | Any person-level object, handle, traversal, score or action; an unbounded read; an aggregate that grows faster than its data; a model where a rule would do. |
| **What it can see** | Aggregates of events and the structure of authority, keys, contexts, trust and rules. |
| **What it cannot see** | A person. The single-entity investigation page exists outside these layers, gated by role and audited on every read. |
| **If it fails** | A person reference anywhere in Athena's SQL fails `check_athena_no_person` and stops the build, measured by adding one (the other Athena checks guard other properties and stay green); an unbounded aggregate fails C8; a learning system that scored people would violate the vocation and is refused before it is designed. |
| **Now or later** | Atlas and Athena now. Metis after an assessment, and only if an outside party needs it; Myrmex is a proposal in this paper. |
| **Inside and outside** | Inside: operations. Outside: the loop by which the system corrects its own understanding. |

> *A system that can observe itself still has to learn from what it observes, and the hardest lesson Polaris has learned is that its own instruments can be green while measuring the wrong thing. The next section is that lesson, and this version has a great deal more of it to tell.*

# 15 The assurance layer: how the system notices its own errors

*A green test is not proof that the test asked the right question.*

IN THE TOWN The town's own inspectors test the wall by pulling out a stone to see whether the alarm rings; an alarm nobody has ever heard ring is not trusted.

## 15.1 Formal truth is not intended truth

A check can be green while proving the wrong proposition. A signature can exist without being verified; a benchmark can count database writes as verifications; a two-witness guarantee can rest on one witness that happened to be absent; a suite that proves one active credential per person can be the thing violating it. The assurance layer exists because every layer of this map is believed on the word of a check, and the checks are written by the same hand as the code they judge. Its question is not *does it pass* but *would it notice*.

The layer works as a loop (Figure 12): an observation made by running the system rather than reading it, a failure that is a green check proving the wrong thing, a corrected statement of what the mechanism proves, an invariant with a detection test, and continuous integration carrying that invariant on every push. Invariants accumulate and are not deleted.

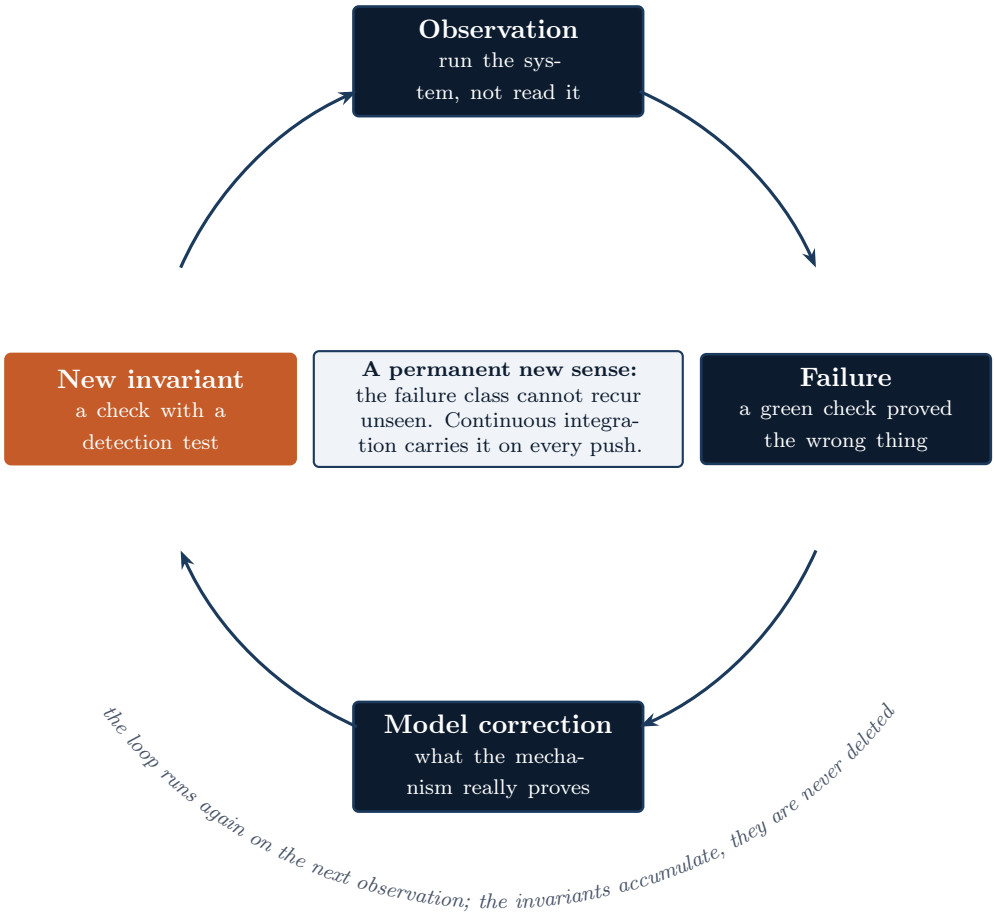



**Figure 12:** The loop by which an error becomes a permanent sense. Observation is running the system, not reading it; the failure is a green check that proved the wrong thing; the correction is a new statement of what the mechanism proves; the invariant is a check with a detection test; and continuous integration carries it on every push from then on.

## 15.2 The observers

No single observer decides whether a claim holds. Each is built differently, so that a mistake one of them shares with the code is unlikely to be shared by all of them.

- *Invariant checks.* Plain functions over the tree, each paired with a detection test that shows it passing on a good fixture and failing on a broken one, so a check that cannot see its own violation is itself a failure.
- *Mutation drills.* Twelve drills remove or invert what a layer relies on (a check, a constraint, a trigger, a procedure's refusal, a verifier's rejection, a witness's comparison) and require something to go red. A survivor is a guarantee nothing was testing; the few that are tolerated are declared with a reason.
- *Independent witnesses.* A cryptographic verdict has two implementations and a published corpus; the zero-knowledge proof has a second witness written separately. Their independence is in implementation, not in their reading of the specification (Section 8).
- *Drills and attacks.* Paths are run end to end rather than inspected, and attacks use the real code; a fuzzer explores the inputs the drills did not choose.
- *The suites under a different axis.* The same suites run as the role production connects as rather than the schema owner, under the real signer rather than the development placeholder,

and from sessions in other timezones, because a suite pinned to one condition certifies only that condition.

- *Outside observers.* A wallet someone else wrote and a conformance service someone else runs. They form a separate layer because every observer inside the repository inherits the author's model of what a correct artifact contains; an outside wallet exposed a certificate defect that every internal instrument had passed.

The same layer holds the vocabulary to the evidence. Checks refuse the stronger word on every outward surface: the system is described by algorithm agility under an audited migration path, not by a post-quantum claim; the duress mechanism is duress-aware, not compulsion-resistant; C3 is one active credential per enrolment identity, not one per human.

**Layer card: the assurance layer**

| | |
|---|---|
| **What it is** | Invariant checks with detection tests, twelve mutation drills, independent cryptographic witnesses, end-to-end drills, an attack harness, a fuzzer, the suites run under the production role, the real signer and foreign timezones, and the outside observers on the scoreboard. |
| **Why it exists** | Every other layer is believed because a check says so, and a check can be green while proving the wrong proposition. |
| **What it trusts** | That observers built differently do not share the same mistake; that continuous integration runs what the gate names. |
| **What it refuses** | One observer's verdict as sufficient; a check without a detection test; a skipped security test in continuous integration; a stronger word than the evidence supports. |
| **What it can see** | Whatever its observers exercise: a refusal that can be removed without anything going red, a check that passes on a broken fixture, a suite that holds only for the schema owner. |
| **If it fails** | A guarantee in another layer becomes a belief nobody is testing; the record of each time that happened is the companion *Evidence Record*. |
| **Now or later** | Now, inside the repository; outside, the observers of Section 20. |
| **Inside and outside** | Inside: every other layer of this map. Outside: the external observers on the scoreboard. |

## 16 What Polaris refuses to become

*Privacy is not freedom from coercion.*

IN THE TOWN The laws above the town bind the town. The hall may not mint coin. The register may not be turned into a list of who is absent. The gates may not be closed on a neighbour on the word of a neighbour's neighbour. A name struck from the register stays legible as struck. No clerk, however clever, may be set to guessing which townspeople are likely to cause trouble. And the town does not claim that its walls keep a person safe from a guard who is standing beside them.

Powerful identity infrastructure carries risks that no amount of correctness removes, because they are risks of what a correct system can be used for. The repository names these as design and governance constraints rather than as politics, and this section states them the same way. Each is a way a working Polaris could be turned into something its constitution forbids, and each is stated no more strongly than the evidence supports.

**Identity must not become money.** A credential that carries spending authority inherits programmability, and every constraint anyone wants placed on spending accretes onto the identity token, until the system can be told whom to refuse at a fuel pump. C10 forbids a monetary claim in the schema; if a value system is ever built, it lives in a separate database that consumes verification proofs over a boundary, and the boundary is what is load-bearing. This is the single most consequential architectural decision in the repository, and it is the one that never expires with any phase.

**Privacy is not merely secrecy.** A system can know very little about a person and still be coercive, if every doorway requires its authorization. The zero-knowledge default bounds what is recorded; it does not bound what a society demands. The tiered-enrolment layer exists for this: exemption is a first-class, one-row state, the not-enrolled cannot be enumerated by an accident of schema, and a person with no documents can be enrolled through a referee whose vouching leaves no mark on the credential. The coexistence plan names the sunset of the old credential as the moment the new one becomes compulsory, and refuses to compute that verdict from what the issuer can see. The repository's own threat model names the strongest attack on this layer as making non-enrolment civically impossible from outside, no token, no healthcare, and says plainly that the schema cannot stop it, only make it a named act.

**Anti-coercion is a vocation, not a claim.** The vocation above the ten constraints says no person may be compelled to surrender their identity. The mechanism that serves it, the duress code, resists the coercer it was built for and not the two others the operating contract named, and against lawful or institutional access its indelible record is evidence against the holder (Section 3). So the front door says *duress-aware* and a check refuses *compulsion-resistant*. The boundary is the honest one: the system notices duress; it does not resist compulsion, and a vocation that pointed at the adversary the mechanism is weakest against would have been the opposite of what the word promised.

**Technical federation is not social decentralisation.** Polaris is federated in code: each authority its own key, no central store, no transitive trust, and an operator bound to one authority even inside a shared instance. Two instances, each with its own root, demonstrate the protocol. Whether authorities, witnesses and relying parties are in fact independent is a property of a deployment, not of the repository.

**Revocation must not become civil erasure.** A revoked token is a terminal state in an append-only record, not a deletion; lost tokens stay lost, and their data is not erased. Succession is by reference. Mass revocation is bounded by a ceiling and a co-signer, and the ceiling itself can only be changed with a reason on record. Erasure, when a jurisdiction requires it, is pseudonymisation with an append-only record of who, when and why, never of the prior name; a pilot's wind-down is that erasure, executed rather than promised. An authority can end a person's credential; the schema refuses to let it end the person's history.

**Relying-party correlation is different from issuer unlinkability.** The issuer's database cannot rebuild the zero-knowledge verification graph. Two relying parties can join their logs by whatever stable material a presentation showed them. Both sentences are true at once, and Version 3 can say how much of the second was measured: on the native zero-knowledge path a colluding pair has no advantage beyond the anonymity set against an adversary that matches on equality, and that set is floored at twenty members unless a deployment sets another; on the interoperable path a single presentation leaves nine issuer-signed values identical at every verifier (Section 8). A reader who hears *unlinkable* and imagines either path is stronger than that has been misled, which is why the repository states the boundary positively and the verifier reports which kind of presentation it was shown.

**Interoperating imports the ecosystem's properties.** Speaking the format an outside wallet speaks was the first thing anyone outside used, and it carried the format's linkability with it. The repository records that as a cost of interoperating, not a reason to stop, and it does not let the interoperable path borrow the native path's privacy claims.

**No population graph.** The Atlas is bounded and never locates a zero-knowledge event; Athena is forbidden a person node, a stable subject surrogate and a subject traversal; the schema carries no attribute to filter a population by; quotas and alerts are keyed by agency, never

by person; the transparency report carries no total a small count could be recovered from. The refusal is structural because a population graph, once built, outlives the intentions of whoever built it.

**Machine learning must not turn identity infrastructure into scoring.** The proposed learning layer may learn how the identity system behaves and must never learn how people behave. Fraud or risk scores for individuals, behavioural profiles, relationship inference, eligibility or likelihood-of-revocation scoring, person-level clustering or embeddings, and model-driven denial of rights are presumptively prohibited, and a recommendation is evidence, never authority.

The repository also lists permanent non-goals that do not expire with any roadmap phase: no payment rails, no central biometric database, no population-scale attribute filtering or analytics, no social scoring, no commercial or advertising use of verification data, and no real personal data before a consent framework exists. Every one of them is a way the system could be made more useful to someone, and every one is refused on sight. Since the operating contract, one more refusal governs the repository itself: no new capability is built because it would be elegant, complete or desirable; a product change needs a named outside party that requires it, or an executable counterexample against something already promised.

# 17 AI agents, proof of personhood and the future internet

*Proving a credential is not proving a person.*

IN THE TOWN Some who reach the gate now carry a letter from a townsperson that lets them run one errand on that person's behalf, until a date; the guard reads the letter, and learns nothing else about who wrote it.

This section maps what the holder-side mechanisms establish when the one presenting a credential may be an agent rather than a person: four mechanisms, what each proves, what each refuses, and the residue none of them reaches.

## 17.1 The problem that is arriving

When software agents can generate unlimited accounts, content, transactions, messages and apparent participants, the cheap signal that a request came from a person stops being available for free. Many digital systems will need to distinguish, without learning more than they must, between one accountable human; one authorized agent acting for that human or for an institution; anonymous machine activity; duplicated or synthetic identities; and organisational agents acting under an institution's authority. Every naive way of obtaining that signal leaks identity: accounts, phone numbers, payment instruments, biometrics. The identity question changes shape. It is no longer only *who are you*, but *what kind of actor are you, whose authority are you exercising, what are you allowed to do, and how can I verify all of that without learning anything else.* That is the future-facing question this paper considers the most important one Polaris's core could be extended to answer.

## 17.2 What exists today that points this way

Six ingredients of a privacy-preserving proof of personhood run.

- **Uniqueness at issuance.** C3, one active token per person, is a partial unique index in the database and a model checked on every push. What it establishes is credential uniqueness within an enrolment identity, not human uniqueness: one `Individual` row holds at most one active credential, and no application code can bypass that. Nothing in the database or above

it notices one human enrolled as two rows; that is decided at the enrolment desk, by an operator's judgement recorded as evidence (Section 8). C3 is therefore not Sybil resistance.

- **Membership without identity.** The epoch membership proof proves that a valid token exists in an authority's committed set with public inputs of only the root, the epoch, a context, a nonce and a scoped nullifier; a zero-knowledge verification records no token at all; the anonymity set is a full epoch of at least twenty members unless a deployment sets another floor.
- **Revocation without naming.** A revoked token drops out of the next epoch root, so a membership proof expires with the epoch and no relying party learns who was revoked.
- **A key the holder controls.** A credential may carry a holder public key the issuer binds to it, so possession of the file stops being possession of the credential and a verifier can require a signature from the person rather than a copy of their credential.
- **Proving on the holder's device.** The epoch's leaf set is published, signed and bounded; every requester receives identical bytes; the holder finds their own leaf and builds the path locally. The issuer is never told which member is proving.
- **A per-verifier nullifier.** The proof carries `Poseidon(secret || scope || epoch)`, constant for one person within one relying party's scope and epoch and uncorrelated across scopes.

## 17.3 One human, once: what runs and what it costs

The membership proof stops replay of one proof, but on its own it does not stop the same person producing a thousand fresh proofs to one site and registering a thousand accounts. *One human, once, here* needs a scoped nullifier: one more public input, a hash of the holder's secret, the relying party's scope and the epoch, so a site can refuse a second proof from the same person while two sites cannot link their nullifiers. That is a public input of the circuit, and the claim that two relying parties pooling everything they hold still cannot link a holder is a formal model checked with a configuration it must fail.

The leaf is opened inside the circuit, and the reason is structural. A nullifier beside a leaf the circuit cannot open proves only that the prover knows some number: nothing ties the two to one secret, so a prover could pair any member's leaf with a nullifier of their own choosing, and every property above would be a claim rather than a proof. Constraining both to one secret means opening the leaf in the circuit, where a `SHA3-256` preimage costs thousands of constraints and Poseidon about a hundred. So the leaf is `Poseidon(secret || context)`, a commitment the circuit opens, and the holder's secret is derived with `SHA3-256` outside it.

The two conditions the vocation imposes were carried into the implementation rather than left as intentions. The nullifier's scope is per relying party and per epoch, never global, because a global nullifier is a persistent identifier and therefore the surveillance outcome the constitution refuses; and it rotates each epoch, so membership never becomes a permanent pseudonym. A relying party's ledger is keyed by its own scope and that epoch, and should not outlive it.

Two bounds remain and are stated rather than closed. The nullifier does not hide the holder from the *issuer*, which derives every leaf in order to build the tree and could therefore compute any member's nullifier in any scope; the property holds between relying parties. And the presentation layer is still full-credential on every path but the zero-knowledge one: a verifier looking at a plain credential sees a stable token value, an issuer signature and a holder key, and a verifier on the interoperable path sees nine issuer-signed values that are the same everywhere. What the pairwise derivation bounds is what a verifier *stores*, which is the material that gets pooled, sold, subpoenaed and breached; what the zero-knowledge path bounds is what it is *shown*, and the lab measured that bound and its blind spots (Section 8). The verifier reports which of the two a given presentation gave, and since the lab's first finding it checks the premise before saying so.

The blinded and one-time presentation technologies that would remove the residue are held, not scheduled, in Section 18; none is implemented. PROPOSED

## 17.4 Delegating authority to an agent

The relationship is a chain, and it runs. A human possesses a foundational credential and a key the issuer bound to it. With that key the human signs a grant giving an agent a narrowly scoped authority: named actions, stated limits, an expiry, and a revocation handle belonging to the grant alone. The agent signs each action it performs, naming the action and the service's own nonce. A service verifies, offline, that the grant is genuine and unexpired, that it covers this exact action, that it has not been revoked, and that it chains to a holder key an issuer bound to a real credential.

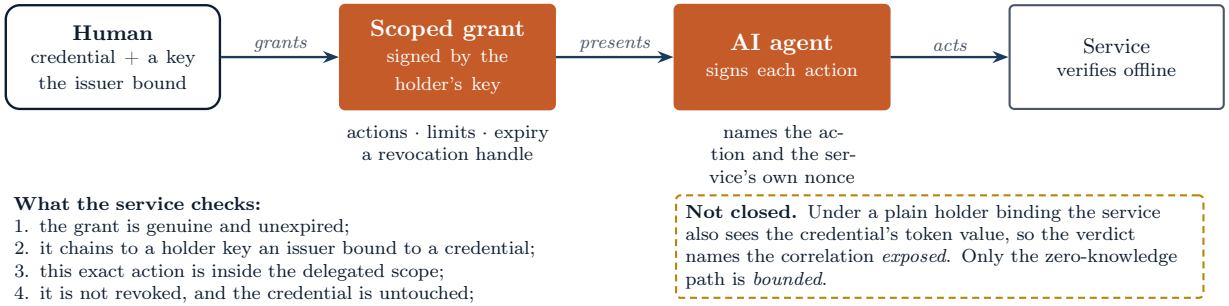



**Figure 13:** The delegation chain, as built. The human holds the foundational credential and a key the issuer bound to it. With that key they sign a grant naming actions, limits, an expiry and a revocation handle; the agent signs each action against the service's own nonce; the service verifies the chain offline. The one link drawn dashed is the residue: under a plain holder binding the service also sees the credential's token value, so the correlation it is left with is bounded rather than absent.

What the design is for is visible in what it refuses. Handing an agent the credential itself gives it everything the person can do, forever, revocable only by revoking the person. The grant is the opposite of each: `actions` and `limits` sit *inside* the signed statement, so a grant widened in transit fails rather than passing invisibly; an empty action list authorises nothing and there is no way to write *everything*; a limit key the service does not recognise is refused rather than ignored, since a grant that says `max_transfers: 3` to a service that has never heard of it must not read as unlimited. And the agent must sign: without that link a grant is a bearer token, and whoever copies it in transit becomes the agent. Two of these refusals, an exhausted use count and an amount over its ceiling, are the bound that makes delegation something other than a bearer token; the conformance contract includes cases in which a grant's or a revocation's signature is bad, and a mutation drill shows the tests notice losing either bound.

Revocation is the property worth naming twice. The revocation is signed by the same key that signed the grant, so anyone may publish bytes but only the holder may end it. The issuer is not contacted and never learns the grant existed, and the human's credential is untouched throughout. A person can end their agent's authority without asking permission from, or being observed by, the authority that issued their identity. The revocation carries the grant identifier and the instant, and nothing else. It will not carry a reason: a field recording *why* a grant ended is a field a coercer can demand be filled in or left empty, and either way it turns a revocation into a signal about the person.

What a service learns is bounded, and the bound is reported rather than assumed. Under a plain holder binding the service also sees the credential's token value, so the verdict names the correlation `exposed`; only a chain carried by the zero-knowledge path, whose handle is the scoped nullifier and whose transcript shows nothing identical across verifiers, is named `bounded`. The grant hides the human from the agent's *actions*. It does not hide the credential from the service.

## 17.5 The future internet, concretely

By a future internet this section means one narrow thing, and it has nothing to do with currency: an internet in which people, autonomous agents, institutions, wallets, distributed services and independently operated networks need portable cryptographic trust that does not route through one company or one state. Against that environment the following table marks what Polaris could provide and what each item's status is today.

**Table 2:** What a future environment would ask of Polaris, and the status of each.

| The environment asks for | What Polaris has | Status |
|---|---|---|
| Portable credentials | A signed file any standard library verifies; four packages on public registries | IMPLEMENTED |
| Independently verifiable proofs | The detached verifier, two SDKs, a conformance suite, and an OpenID4VP verifier an outside wallet has presented to | IMPLEMENTED EXTERNALLY EXERCISED |
| Institutional federation | Per-authority roots, directional attestations, published feeds | IMPLEMENTED |
| Service discovery | The signed registry | IMPLEMENTED |
| Document signatures with long-term validity | Signed containers with evidence fixed at the instant | IMPLEMENTED |
| Transparent authority behaviour | Three logs, witnesses, equivocation proofs, a public report of warrant reads | IMPLEMENTED |
| Offline verification | Status assertions stapled to presentations; a card that signs a device's challenge | IMPLEMENTED |
| Algorithm agility | ML-DSA under an audited migration path, hash-based proofs, a hybrid edge; classical remainders named; what a break would cost written down | IMPLEMENTED |
| Human, account and agent authorization relationships | A login token whose subject is per relying party, plus the agent grant | IMPLEMENTED |
| Privacy-preserving uniqueness | Membership proofs carrying a per-verifier, per-epoch nullifier; measured against a colluding pair | IMPLEMENTED |
| Delegation to agents with bounded scope | A holder-signed grant: actions, limits, expiry, revocation handle; its bounds mutation-tested | IMPLEMENTED |
| Revoking an agent without destroying the human's identity | A holder-signed revocation; the issuer is not contacted and the credential is untouched | IMPLEMENTED |
| Presentation that shows a verifier nothing stable | The zero-knowledge path only, within the anonymity set; a plain or interoperable presentation shows stable, issuer-signed values | IMPLEMENTED PROPOSED |
| Speaking the formats other wallets speak | OpenID4VP with SD-JWT VC verified; an ISO 18013-5 rendering whose issuer signature no standard reader can check; a Verifiable Credential of a verification result | IMPLEMENTED EXTERNALLY EXERCISED |

# 18 Future research and engineering, under a rule that forbids most of it

*An idea worth building is not a reason to build it.*

IN THE TOWN The council keeps a book of what it could build and one rule: nothing in the book is built until someone outside the wall asks for it.

This section is not a list of things to build. The operating contract changes what *future work* means: a product change needs exactly one qualifying reason: a named external implementation requires it, a named non-author operator or holder requires it, a finding originating outside the repository requires it, or an executable counterexample shows behaviour the repository already promises to be false. Cleanliness, completeness, elegance and a desirable new guarantee do not qualify, and the contract says not to invent a fifth reason. Everything else goes to the lab, whose job is to falsify the differentiating claims, not to expand the architecture. So the items below are sorted by who would have to act, and everything is marked PROPOSED unless stated otherwise.

## 18.1 Gates that are not the repository's to pass

The contract was adopted on 13 September 2026, and it dates itself. Its ninety-day objective falls due on 12 December 2026 and names seven things; the scoreboard (`lab/EXTERNAL-NOUNS.md`) says which hold. Read on 5 October 2026, day twenty-two:

**Table 3:** The operating contract's ninety-day objective, item by item, against the scoreboard on day twenty-two.

| The contract asks for | Holds | The evidence, or what is missing |
|---|---|---|
| `polaris-verify` independently installable | yes | on PyPI since 15 September 2026; the boundary drill installs it where no Polaris code exists |
| Real cryptographic behaviour explicit | yes | the verifier refuses to start without a declared crypto mode (Section 6) |
| A named external wallet completes a presentation | yes | walt.id Wallet API v2 first, 15 September 2026; six unmodified wallets and eight libraries and tools by 4 October |
| An external OpenID conformance suite running | yes | the Foundation's hosted suite, from 15 September 2026; `polaris-oid4vp 1.0.0rc7` certified on 24 September |
| A relying party or operator who is not the author has used the verifier | no | none; this is also the one condition between the release candidate and `1.0.0` |
| Repeated outside presentations | no | presentations outside CI: zero |
| External failures entering the development loop | yes | two fixes caused by outside findings (Section 6) |

Five of seven held by day ten and still do on day twenty-two, and the two that do not both need a person who is not the author, which is the point of the contract: nothing left on the list can be done by writing more code. A second wallet and a certification, once rows that no deadline governed, are on the scoreboard now (Section 20).

The two rows that remain can be filled by a reader of this paper, and the repository says how. The stranger's path (`docs/STRANGER-PATH.md`) goes from a clean machine, the published package and an unmodified outside wallet to an accepted presentation in about ten minutes, without cloning the repository, and it is walked end to end before it is changed. Someone who is not the author walking it and reporting the result, in the repository's issue tracker, is the first missing row; the presentation it ends in is the second. A failure counts as much as a success: it is an

external finding, and each of the two recorded so far became a fix. The scoreboard fills a row only with a name and a date, so a report is evidence only when its author is willing to be named. A reader without Docker has a shorter door: each of the other three packages' READMEs, on the registries from their next release, opens with a first run against the repository's published vectors, a genuine credential and a tampered one, that takes a minute from the registry, and each was walked as a stranger on 23 September 2026.

The contract also names the other end. On 12 March 2027, day 180, the external fields are read. If they remain effectively empty while internal versions, invariants and architecture keep growing, the core is to be frozen, the national-scale architecture stops expanding, the verifier is kept only if external use exists, and otherwise the project is archived as a reference implementation and a lab. This paper's subject may be archived by its author's own rule, and the paper records the rule rather than arguing with it.

## 18.2 Questions the lab still owes an answer

Each of these is a measurement, not a feature, and the lab's discipline is to publish the threat model, the adversary, the harness, the result and the limitations together, with a positive control before any null result counts.

**The rest of the linkability threat model.** The measured result covers stable fields only. Presentation size, timing, issuer metadata narrowing the anonymity set, status artifacts stapled to a presentation, and transcript structure are unmeasured, and the one channel the adversary can read, transcript length, is known to be live in any deployment where a presentation's size varies per holder.

**The interoperable path's nine handles.** Batch issuance, many credentials with one holder key each, is the ecosystem's mitigation and is not implemented; whether it would be adopted is an outside party's question, and the cost of not having it is now a number.

**The migration path's other half.** Rollback, and the mixed window during a partial migration, are unmeasured. A verifier-readable algorithm status, in the registry or the trust list, would be a new product guarantee.

**Duress against the informed coercer.** A holder-side panic signal, a deniable enrolment that makes the absence of a duress code unprovable, and a purge path for the duress record are each a mechanism change, and the assessment proposes none of them. What it asks for first is a measurement the repository cannot make: what a coercer standing beside a holder actually observes.

**The anonymity-set size.** Reporting the crowd size beside a `bounded` verdict would change what the system promises; it is recorded for a deploying organisation to decide.

## 18.3 Presentation technologies that would remove the correlator, held

Five families of presentation technology would remove the correlator. One, the pairwise identifier, is built, with the bound Section 8 states. The other four are held rather than scheduled, and the reason each is not built is the contract's, not a technical one.

**Blinded or derived presentations.** The holder proves possession of a signature over attributes without revealing the signature itself. The classical constructions rely on pairing-based assumptions that a quantum computer breaks; post-quantum equivalents are an open research area, and the repository's posture would have to be preserved.

**One-time presentation tokens.** Single-use artifacts minted per presentation, at the cost of issuer contact or a pre-fetched batch, which trades against offline availability and issuer non-observation exactly as the status assertion does.

**Anonymous credentials with selective disclosure.** Revealing that an attribute satisfies a predicate without revealing the attribute. The benchmark against a mature system measured the gap precisely: Polaris as an issuer has no attribute credential to apply a predicate to, and as a verifier it accepts a pre-computed boolean the issuer had to anticipate.

**A native anonymous predicate.** The benchmark's first scenario is satisfied by a disclosed attribute, not by an anonymous predicate.

## 18.4 Metis: learning the system, never the people

The assessment the roadmap requires before any code would have to falsify the central hypothesis (that a model can infer properties of the system without inferring the rights of people) against a synthetic-nation ground truth, compare every model to a deterministic rule, and design the constitutional invariant that would forbid the prohibited uses structurally, in the manner of Athena's six checks. Under the contract the assessment itself waits on an outside party needing what it would produce; nothing about Metis is scheduled, and this paper keeps the brief because the prohibition in it binds any future cognitive layer whether or not Metis is ever built.

## 18.5 Myrmex: an outside observer, proposed

Myrmex is a proposed debugging system that would live outside Polaris and attack it independently, coordinated the way an ant colony is, without a central plan: specialised agents leave machine-readable evidence on a shared landscape, confidence comes only from reproductions and never from agreement between models, and every reproducible failure enters the assurance loop of Section 15 as a test and an invariant. It is held to the opposite discipline of anything inside the tree: outside and adversarial. Part of its purpose is already served without it. The mutation drills are its cheapest members, built inside the tree; and the observer that found what no drill could was one wallet written by strangers (Section 20). The proposal stands because a stranger's wallet exercises one path, and the rings inside it are still watched only by their author's instruments. PROPOSED

## 18.6 Engineering items held under the contract

A hash-based signer behind the same custody interface; the chain-backed ledger drivers whose interfaces are declared; native wallet clients and a browser bridge; status distribution as a deployment rather than a specification; a manufactured card; and the external-implementation path for the native protocol. Each is a roadmap row that the contract has moved from *next* to *when somebody outside needs it*, and the repository's own history is the argument for the move: three hundred and ninety-seven versions were spent building rings that no one outside had asked for, and the first outside party to arrive found a defect in an afternoon.

# 19 The design's boundaries, and the tests that would break its claims

*A limit written down is not a limit removed.*

IN THE TOWN The town posts on its gate what each council that adopts it must decide for itself, and the tests any traveller may put to its walls.

The repository's readiness ledger opens with one line, and this paper repeats it as its own bound: at `1.0.0-rc.7` Polaris is not production-ready for real identity data. Every engineering gap the ledger enumerated is closed and pinned by a check; what remains is not buildable in the repository. This section names the decisions a deployment makes, states what the design keeps if its computational assumptions fail, and gives, for each central claim, the test that would break it.

## 19.1 Decisions only a deploying organisation can make

The readiness ledger lists the decisions to be recorded for a named deployment before real data could be held, among them a legal basis, impact assessment and regulator approval; the signing-key custody driver and who holds the key; the high-availability topology and its hosts, and whether a second region is run; encryption at rest and its key custodian; the off-site backup target and schedule; the alerting backend, the on-call rotation and who receives the duress page; the right-to-erasure policy; the retention schedule per class, with the counsel who says the number satisfies the statute; and an independent penetration test with a threat-model sign-off. Polaris records each decision and its justification; it does not know the law.

## 19.2 If every computational assumption fails

Every signature, hash and proof in this map rests on an assumption that some computation is too expensive. The map is therefore drawn in two layers. The cryptographic layer (credential signatures, the authenticity pack, offline verification, the status assertion, the zero-knowledge proof's soundness, transparency signatures, witness cosignatures, the chain anchors, TLS) holds exactly as long as those assumptions do, and an adversary with unbounded computation removes it: a computer with access to closed timelike curves would, for one, solve every problem in PSPACE [1]. The structural layer does not rest on computation at all. One active credential per person is an index; the audit of record is append-only by trigger; a zero-knowledge verification stores no identifier because a constraint forbids it; identity is not money because no such column exists; recovery needs people at independent institutions. Those hold as long as the database and its operators are not otherwise compromised. When the first layer falls, the authority detects from records the forger cannot rewrite, revokes, and re-signs under whatever replaces the failed assumption, which is why the migration path is drilled rather than planned (Section 5). Polaris does not claim to be safe in that world; it claims to lose its cryptographic layer and keep its structural one.

## 19.3 Each claim, its evidence, and the test that would break it

The front matter asks every reader for a test. This table makes the request specific: for each central claim of this map, the evidence the repository holds and what an outsider would do to break it. A result, a failure as much as a success, reported in the repository's issue tracker by someone willing to be named, becomes a dated row on the repository's scoreboard.

**Table 4:** Each claim, its evidence, and the test that would break it.

| Claim | Evidence today | What an outsider would do |
|---|---|---|
| The schema refuses what the application must not do (C1, C2, C3) | Constraints, triggers and grants; mutation drills; since this map was drawn the suites also run as the application role | Attack the database as the application role with the schema in hand |
| Issuer-unlinkability in zero-knowledge mode | A circuit, a second witness by the same author, a linkability lab | Review the circuit and derive its concrete security; run the linkability lab's adversary with their own strategy |
| The anonymity set is the epoch, floored | Refusal at epoch close and at verification, tested in the suites | Measure what an issuer learns from epoch timing and size in a realistic enrolment pattern |
| Credential uniqueness per enrolment identity | The unique index and its formal model | Enrol one person twice through the documented desk procedure and see what notices |
| The duress mechanism helps against an unaware coercer | The duress lab's measurement | Repeat it with a coercer who has read this paper |
| The native protocol is implementable from the specification | The conformance suite, run only by Polaris's own SDKs | Implement a verifier from `WIRE-SPEC` and the suite, without reading Polaris code |
| Transparency logs let an outsider detect a rewrite | Consistency proofs checked by witnesses the author runs | Run an independent witness for a period and report what it saw |
| The throughput model clears the national targets | Single-machine measurements and a model | Measure a multi-instance deployment under load |
| A stranger can follow the stranger's path | The author walked it on a clean machine | Walk it and report where it failed |

> *Every row of that table waits for a stranger. The last ring is what happened when the first ones came.*

# 20 The last ring: other people's software

*A system that has checked itself is not a system that anyone else has checked.*

IN THE TOWN Every wall, tower and ledger in this town was built by the people who live in it, and checked by them. One day travellers arrived who had built none of it. They carried their own seals, spoke the common language of the road, and owed the town nothing. What they did at the gate is the only report the town did not write about itself.

Every ring in this map exists because the ring inside it could be correct and still unable to prove what a society needs to know. The last ring is the one no amount of internal work can supply: software written by someone else, which does not share the author's assumptions about what a correct artifact contains, meeting Polaris at one of its doors. The door is the OpenID4VP verifier of Section 6, chosen because it speaks the protocol an outside wallet actually speaks.

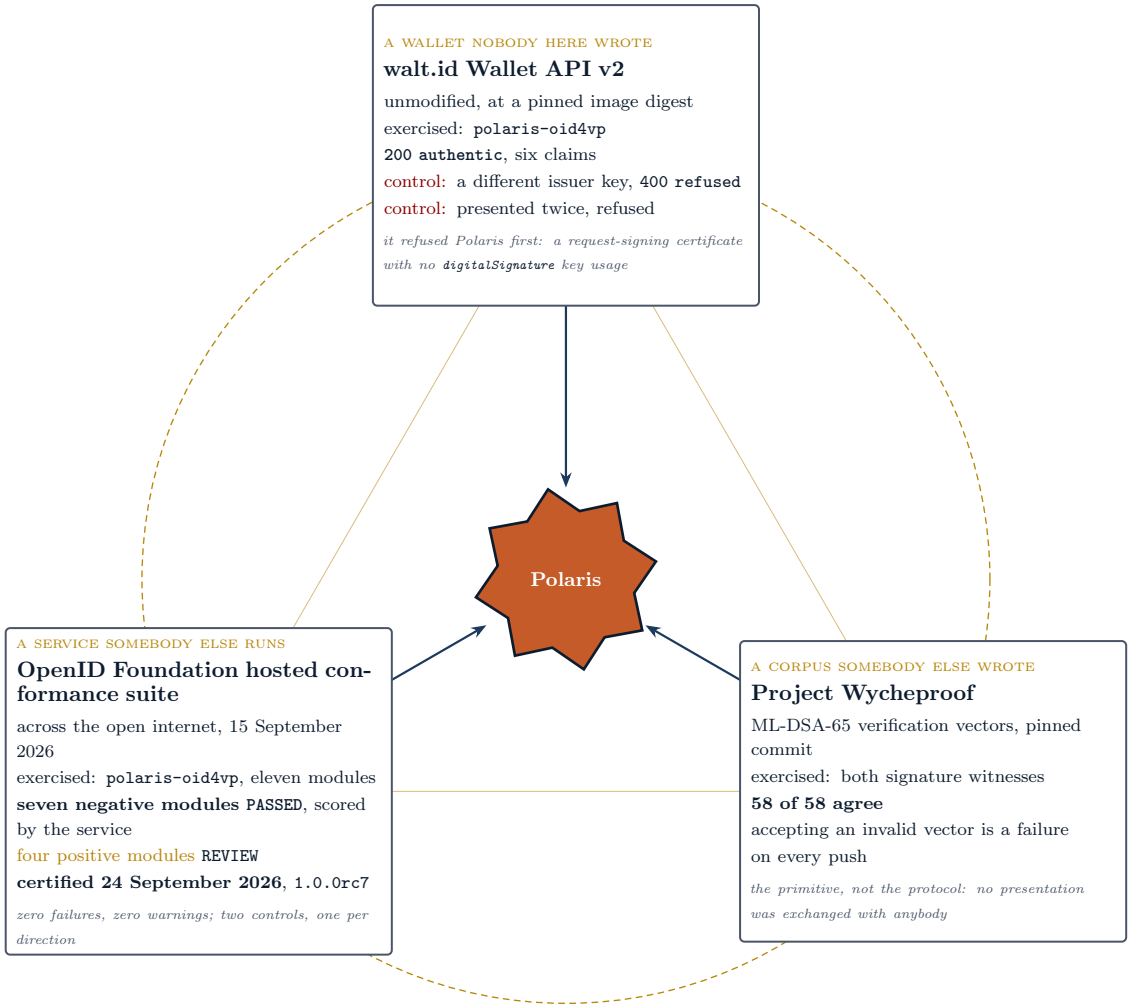



**Figure 14:** The first three things outside the repository that exercised it, and the two controls without which the first would mean nothing. A wallet nobody here wrote presented a credential and was accepted; the same wallet, with the verifier trusting a different issuer key, was refused; and a re-presentation against an answered request was refused. The Foundation's hosted suite scored seven refusals itself and left four acceptances for a human reviewer; a later run of the same plan was certified. A published corpus agrees with both signature witnesses on the primitive.

## 20.1 The first stranger: a conformance service

The OpenID Foundation runs a hosted conformance suite with a test plan for exactly this role, in which the suite itself plays the wallet and signs the credential. It ran all eleven modules of that plan against the verifier across the open internet, fetching the request object and posting the response from outside. The seven negative modules, in which the suite sends a presentation broken in one specific way, are scored by the service itself on the verifier's refusal, which makes them the closest thing to an external adversary this project has been offered. The four positive modules end in a state the suite reserves for a human to attest a successful verification. A later run of the same plan was submitted for certification, and the Foundation lists `polaris-oid4vp 1.0.0rc7` as OpenID Certified to the OpenID4VP 1.0 + HAIP 1.0 Verifier profile. Each run carries two negative controls, one per direction: a verifier patched to refuse everything is noticed by the positive modules, and one patched to accept everything is noticed by all seven negatives. EXTERNALLY EXERCISED

## 20.2 The second stranger: a wallet

Then a wallet arrived. An unmodified walt.id Wallet API, the published image at a pinned digest, fetched the signed request object, verified it against the registered trust anchor, matched its credential to the query, signed a key-binding token with a key it generated and this repository has never held, encrypted the response, and sent it. The verifier checked the whole chain and accepted it.

It refused Polaris first, and it was right.

The verifier's own key generator had produced a request-signing certificate with no key-usage extension, so the certificate whose entire job is signing request objects was not marked as usable for signing. Every observer inside the repository had passed over it: the conformance modules, the package's tests, the mutation drill over every refusal, every check in the invariant layer. Each of them inherits the author's model of what such a certificate must contain. The first observer that did not inherit it found the gap at once. The certificate was fixed, with a test that fails on a leaf without the extension, and the exchange carries the same two controls as the conformance run, because a verifier that accepts everything prints the same success line. Anyone can repeat the exchange from a clean machine in about ten minutes without cloning the repository. EXTERNALLY EXERCISED

This is the argument of the whole map in one event. The rings are the author's; the last ring is everyone else's, and it is the only one that can see what the others share.

## 20.3 The ring as it stands

The last ring is fifteen implementations that Polaris did not write and does not configure beyond trust anchors. Six are wallets run unmodified: walt.id's Wallet API, Credo (the OpenWallet Foundation's TypeScript framework), eudi-dev, OID4VCgo and ProtocolSoup (the last three listed by the Foundation as certified wallets), and Procivis One Core, which issues its own credential over its own OpenID4VCI before presenting it. Nine are libraries, services and test tools: the European Commission's reference OpenID4VP libraries for the JVM and for iOS, its reference PID issuer, vck, Multipaz and irmago, through a short holder built here; Axle's Kotlin OpenID4VP client, driven by a harness written here; and SpruceID's conformance adapter and ERICA, a national wallet programme's verifier testing tool, through their own wallet harnesses. All of them meet the same door, and every exchange carries controls that the verifier or the other side must refuse, because a verifier that accepts everything prints the same success line. The ring is measured on the scoreboard, one dated row per run, not in this map. EXTERNALLY EXERCISED

A published third-party corpus also agrees with both signature witnesses on the ML-DSA-65 primitive; it tests the primitive, not the protocol. EXTERNALLY EXERCISED The door itself is published:

four packages are on public registries, each verified by installing from the registry into a clean environment.

After this map was drawn, on 11 October 2026, an outside project began running the verifier in its own machinery: IDFoundry/OID4VCgo's continuous integration runs `polaris-oid4vp`, daily and on each release, building it from this repository's source at a pinned commit and presenting from the project's own conformance wallet. Its first run, OID4VCgo v0.39.0 against `polaris-oid4vp 1.0.0rc20`, accepted the presentation and refused all four controls. EXTERNALLY EXERCISED

What the ring sees is what the author's tests cannot: the assumptions a correct artifact carries without saying so. A trust manager that finds a certificate authority only through its key identifiers; a library that wants its request object encrypted to its own key; a strict wallet that chains the verifier's own TLS certificate. None of these was a question any internal test knew to ask, and each is now answered in the verifier or its test kit.

Two outside parties also list the verifier: the IETF working group's SD-JWT repository among its implementations, and eudi-dev in its comparison of developer tools. A listing is not use; it sits on the map so that it is not mistaken for use.

### 20.4 The scope of the last ring

`polaris-oid4vp 1.0.0rc7` is OpenID Certified by Egor Khaklin to the OpenID4VP 1.0 + HAIP 1.0 Verifier profile (24 September 2026), and the Foundation lists it: a self-certification for one package, one version and one role, not an endorsement, not an audit and not an independent verification. The exchanges cover fifteen implementations, one credential format, one presentation protocol and the classical signature algorithms the profile pins.

## 21 Conclusion: the whole town

Every ring has now been walked, and the whole can be said in two parts: what Polaris does today, and what it is trying to become.

**What Polaris can do today.** Issue, sign, verify, revoke, present, federate, log, witness, exchange, timestamp and sign documents, re-sign a population, wind a pilot back, be installed from a public registry, and accept a presentation from a wallet it had never met, all on notional data, all drilled in continuous integration (Appendix A).

**What it is ultimately trying to become.** Identity infrastructure in which no important claim has to be taken on the word of a central application, because authority, state, history, privacy and the system's own behaviour are exposed to independent verification and constrained by layers that do not share each other's assumptions, and because the last of those layers is other people's software.




**Figure 15:** The ladder. From the person, who exists outside the software, to the software of strangers, who exist outside it too. The right-hand column is the refusal each layer makes of the layer below it; the last rung's refusal is the operating contract's, and nothing in the tree can discharge it. The ladder counts finer than the rings of Figure 2: L0 is the person outside every ring, L1 the core, and the rings outside it are split into the layers the sections take one at a time.

The town's register is small. The walls around it are what make it safe to keep. And the reason the system has so many rings is not that its builders enjoyed rings: each one was added when the ring inside it was found to be correct and still unable to prove what a society would need to know.

*A record of a person is not the person.*
*A promise enforced only by the people who run the system is not a promise.*
*An entry in the register is not proof of who made it.*
*A verdict from the issuer is not a verdict anyone can check.*
*A credential's authenticity is not its current authorization.*
*Current authorization is not privacy.*

*Being verified is not being followed.*
*Federation is not institutional independence.*
*A transparency log is not protection from a split view.*
*A receipt for an exchange is not a copy of the letter.*
*A correct system is not a system that survives.*

*Watching the system is not watching the people in it.*
*A green test is not proof that the test asked the right question.*
*Privacy is not freedom from coercion.*

*Proving a credential is not proving a person.*
*An idea worth building is not a reason to build it.*
*A limit written down is not a limit removed.*

Polaris is the accumulation of those sentences into mechanisms, and the honest map of where the accumulation stands: at version `1.0.0-rc.7`, a release candidate, which is the number for a system whose every internal condition for 1.0 holds, and which moved to 1.0.0 on 11 October 2026, when IDFoundry/OID4VCgo's continuous integration began running `polaris-oid4vp` (Section 20.3). The rings will keep moving; what the motto on the cover promises is the one thing that does not: *fixus inter mutabilia*, a claim counts only when someone else has checked it.

*And a system that has checked itself is not a system that anyone else has checked.*



# References


[1] S. Aaronson and J. Watrous. Closed timelike curves make quantum and classical computing equivalent. *Proceedings of the Royal Society A*, 465(2102):631–647, 2009.

[2] J. Camenisch and A. Lysyanskaya. An efficient system for non-transferable anonymous credentials with optional anonymity revocation. In *Advances in Cryptology, EUROCRYPT 2001*, LNCS 2045, Springer, 2001.

[3] D. Chaum. Security without identification: transaction systems to make big brother obsolete. *Communications of the ACM*, 28(10), 1985.

[4] R. A. DeMillo, R. J. Lipton and F. G. Sayward. Hints on test data selection: help for the practicing programmer. *Computer*, 11(4), 1978.

[5] European Parliament and Council of the European Union. Regulation (EU) 2024/1183 of 11 April 2024 amending Regulation (EU) No 910/2014 as regards establishing the European Digital Identity Framework, Article 5a(1). *Official Journal of the European Union*, L series, 30 April 2024.

[6] National Institute of Standards and Technology. *Module-Lattice-Based Digital Signature Standard*. FIPS 204, 2024.

[7] National Institute of Standards and Technology. *Stateless Hash-Based Digital Signature Standard*. FIPS 205, 2024.

[8] OpenID Foundation. *OpenID4VC High Assurance Interoperability Profile 1.0*. 2025.

[9] World Bank, Identification for Development (ID4D). *ID4D Global Dataset*, 2025 edition: global ID coverage estimates, as of 2024. 2025.

[10] ISO/IEC 18013-5:2021. *Personal identification: ISO-compliant driving licence, Part 5: Mobile driving licence (mDL) application*. 2021.

[11] Y. Jia and M. Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5), 2011.

[12] R. C. Merkle. A digital signature based on a conventional encryption function. In *Advances in Cryptology, CRYPTO '87*, LNCS 293, Springer, 1988.

[13] OpenID Foundation. *OpenID for Verifiable Presentations 1.0*. 2025.

[14] N. Sakimura, J. Bradley, M. Jones, B. de Medeiros and C. Mortimore. *OpenID Connect Core 1.0*. OpenID Foundation, 2014.

[15] Polygon Zero Team. *Plonky2: fast recursive arguments with PLONK and FRI*. Technical report, 2022.

[16] S. Santesson, M. Myers, R. Ankney, A. Malpani, S. Galperin and C. Adams. *X.509 Internet Public Key Infrastructure Online Certificate Status Protocol (OCSP)*. RFC 6960, IETF, 2013.

[17] B. Laurie, A. Langley and E. Kasper. *Certificate Transparency*. RFC 6962, IETF, 2013.

[18] D. Fett, K. Yasuda and B. Campbell. *Selective Disclosure for JWTs (SD-JWT)*. IETF OAuth Working Group.

[19] O. Terbu, D. Fett and B. Campbell. *SD-JWT-based Verifiable Credentials (SD-JWT VC)*. IETF OAuth Working Group, Internet-Draft.

[20] P. A. Grassi et al. *Digital Identity Guidelines: Enrollment and Identity Proofing*. NIST SP 800-63A, 2017, and its fourth revision.

[21] T. Looker, P. Bastian and C. Bormann. *Token Status List*. IETF OAuth Working Group, Internet-Draft.

[22] E. Syta, I. Tamas, D. Visher, D. I. Wolinsky, P. Jovanovic, L. Gasser, N. Gailly, I. Khoffi and B. Ford. Keeping authorities "honest or bust" with decentralized witness cosigning. In *IEEE Symposium on Security and Privacy*, 2016.

[23] World Wide Web Consortium. *Verifiable Credentials Data Model v2.0*. W3C Recommendation, 2025.

[24] World Wide Web Consortium. *Web Authentication: An API for accessing Public Key Credentials, Level 2*. W3C Recommendation, 2021.

[25] E. Ben-Sasson, A. Chiesa, C. Garman, M. Green, I. Miers, E. Tromer and M. Virza. Zerocash: decentralized anonymous payments from Bitcoin. In *IEEE Symposium on Security and Privacy*, 2014.

# Appendix A Capability status

The table answers, for every major subsystem, whether it runs, what limits it, which invariant check fails the build if it stops being true, and, where the mark says so, who outside the repository has exercised it. The marks are those of the front matter. Every row was measured at `1.0.0-rc.7`. Rows carrying EXTERNALLY EXERCISED point at a filled row of the repository's scoreboard; no row moved from running to proposed since Version 2, and three moved from internal to externally exercised.

**Table 5:** Capability status at `1.0.0-rc.7`. A row carrying two marks runs today; its second mark, from the legend in *Before you read*, applies to the part of the row it names.

| Capability | Status | Limit stated by the repository | Pinned by |
|---|---|---|---|
| Schema constraints, C1–C10 | IMPLEMENTED | The database owner is outside the threat model; three constraints carry a checked formal model | `check_c1c10_objects_resolve`, the TLA+ drill |
| Append-only audit of record (fourteen named; thirty-one tables) | IMPLEMENTED | Tested once per table that claims it; a reload once reached one record with neither its ids nor its rows | `check_aor_append_only_triggers`, `check_append_only_tables_are_tested_exhaustively`, `check_seed_restart_resets_dependent_records` |
| Retention as data with a 365-day floor | IMPLEMENTED | Archive and purge remain deliberate operator steps | `check_retention_engine` |
| Mutation drills over the verification layer (twelve) | IMPLEMENTED | A shared misreading of a specification survives every one of them | `check_constraint_suite_is_mutation_tested`, `check_triggers_are_mutation_tested`, `check_procedure_refusals_are_mutation_tested`, `check_sdk_refusals_are_mutation_tested`, `check_zk_witnesses_are_mutation_tested` |
| ML-DSA-65 and ML-DSA-87 signing | IMPLEMENTED | The default development and CI path signs a labelled placeholder; SLH-DSA registered, not wired | `check_pqc_signing_wired`, `check_real_pqc_default_boot` |
| Two-witness verification, sampled at use | IMPLEMENTED | Catches divergence, not a shared misreading | `check_pqc_second_witness`, `check_verify_witness_sampling` |
| ML-DSA-65 verification against a published corpus | EXTERNALLY EXERCISED | The primitive, not the protocol: 58 of 58 vectors under both witnesses | the `pqc-real` job |
| Key custody: file, PKCS#11, KMS | IMPLEMENTED<br>EXTERNAL VALIDATION REQUIRED | Software token in CI, no hardware module; the KMS is a stand-in | `check_key_custody_abstraction` |
| Key register and signed trust list | IMPLEMENTED | Rotation of a live key file or slot is an operator ceremony | `check_trust_lifecycle` |
| Algorithm agility, population migration | IMPLEMENTED<br>EXTERNAL VALIDATION REQUIRED | A verifier cannot learn a deprecation; rollback and the mixed window unmeasured | `check_algorithm_agility`, `check_quantum_event_readiness`, `check_post_quantum_claims_are_agility` |

*continued on the next page*

| Capability | Status | Limit stated by the repository | Pinned by |
|---|---|---|---|
| Detached verifier, packaged, fuzzed and attacked | IMPLEMENTED | Refuses to start without a declared crypto mode; needs the prover binary for the zero-knowledge path only | `check_detached_verifier`, `check_verifier_fuzz`, `check_attacks_run`, the product-boundary drill |
| Python and TypeScript SDKs, conformance suite (118 cases) | IMPLEMENTED | 44 verdict fields unconstrained by any case | `check_conformance_suite`, `check_typescript_sdk`, `check_conformance_asks_the_relying_party_question` |
| Frozen version 1, cross-version suite | IMPLEMENTED | | `check_protocol_versioning` |
| OpenID4VP 1.0 + HAIP verifier | IMPLEMENTED EXTERNALLY EXERCISED | One wallet, one format, one path, ES256 throughout; `1.0.0rc7` OpenID Certified to the HAIP verifier profile (24 September 2026), that version and role only | `check_oid4vp_verifier_boundary`, the OpenID4VP mutation drill, the scoreboard |
| Format bridges: ISO 18013-5 rendering, Verifiable Credential of a verdict | IMPLEMENTED | Formats, not trust models; no standard mDL reader can verify the ML-DSA issuer signature | the mdoc and VC drills |
| Relying-party verify API | IMPLEMENTED | Client-credentials only; mutual TLS deferred | `check_relying_party_api` |
| Signed status assertion, offline authorization | IMPLEMENTED | The revoked-but-valid window is the cost; its maximum rests partly on relying-party configuration | `check_offline_verification`, the status-freshness model |
| Epoch checkpoints, revocation feeds, status bundle, cache rules | IMPLEMENTED PROPOSED | Distribution at content-delivery scale is specified and drilled, not deployed | `check_epoch_revocation_propagation`, `check_federation_status_bundle`, `check_status_distribution` |
| Zero-knowledge membership proof, holder-side prover, scoped nullifier | IMPLEMENTED EXTERNAL VALIDATION REQUIRED | The issuer can compute any member's nullifier; no verdict reports the anonymity set's size above its floor (twenty unless set) | `check_zk_tree_depth_synced`, `check_holder_side_prover`, `check_scoped_nullifier`, the witness differential |
| Issuer-side unlinkable verification records | IMPLEMENTED | Scoped to zero-knowledge mode; adversary shaped so it cannot flatter the result | the C2 CHECK and model, the redaction suite, `check_redaction_adversary_is_not_flattered` |
| Cross-verifier correlation, measured | IMPLEMENTED | Bounded on the zero-knowledge path within the epoch's crowd against an equality adversary; exposed on plain and SD-JWT presentations | `check_pairwise_presentation`, the linkability lab |
| Wallet protocol, presentations, QR framing | IMPLEMENTED | No native clients, no browser bridge | `check_wallet_presentation`, `check_qr_resource_bounds` |
| Auth broker with registered policy | IMPLEMENTED | No session product, no consent screen | `check_auth_broker`, `check_broker_policy_bound` |
| Decisions about authorities, operators, quotas, ceilings and relying parties, recorded | IMPLEMENTED | A shell and the tool are recorded on the same terms | the audit-of-record checks, `check_every_audit_event_has_a_writer` |
| Identity proofing, trusted referee, enrolment code | IMPLEMENTED | Evidence authenticity is the operator's word; no proofing has been performed with it | `check_enrollment_code`, `check_trusted_referee`, the proofing drill |

*continued on the next page*

| Capability | Status | Limit stated by the repository | Pinned by |
|---|---|---|---|
| Federation: roots, attestations, manifests | IMPLEMENTED | Two instances by one author | `check_federation_topology`, `check_federation_two_instances` |
| Per-authority isolation of operators | IMPLEMENTED | A shared instance's owner, backup and schema are shared | `check_route_guards_are_derived_not_listed` |
| Cross-authority zero-knowledge | IMPLEMENTED | Abstains without the binary; issuer bound by root | `check_cross_authority_zk` |
| Transparency logs, monitor | IMPLEMENTED | A monitor alone cannot see a split view | `check_transparency_log` |
| Witnesses, gossip, equivocation proof | IMPLEMENTED | Witnesses are drill processes; independence is a deployment | `check_transparency_gossip` |
| External-ledger publication | IMPLEMENTED PROPOSED | File backend only; chain drivers declared | `check_transparency_publication` |
| Public transparency report | IMPLEMENTED | Cannot see an access that was never recorded | `check_transparency_program`, `check_audited_reads_are_logged` |
| Signed registry, exchange gateway and receipts, timestamp authority, document signing | IMPLEMENTED | Bare algorithm names in the registry; notarial signing; anchor verification in verifier and SDKs | `check_registry`, `check_exchange_gateway`, `check_exchange_receipt`, `check_timestamp_authority`, `check_document_signing`, `check_ltv_timestamp_trust` |
| HA, replicas, partitioning, DR, chaos, rolling deploys | IMPLEMENTED | Ephemeral CI hosts | `check_ha_automation` and its siblings |
| Second region as a standby cluster | IMPLEMENTED | Asynchronous; the recovery point is not zero and is measured | `check_multi_region_dr` |
| Capacity and cost models | IMPLEMENTED EXPERIMENTAL | Core counts extrapolated under a named assumption; nothing run at scale | `check_capacity_model` |
| Post-quantum TLS edge and application-to-pooler hop | IMPLEMENTED | The pooler-to-database hop is classical, gated on a PostgreSQL release | `check_edge_pq_kex`, `check_internal_kex_measured` |
| Abuse controls, session and origin hardening, WebAuthn MFA, logs without PII | IMPLEMENTED | Authenticators classical until hardware ships | `check_abuse_controls`, `check_session_origin_hardening`, `check_logs_exclude_pii` |
| Atlas | IMPLEMENTED | Every aggregate graded against its data's growth | `check_atlas_console`, `check_benchmark_measures_growth`, the C8 caps |
| Athena | IMPLEMENTED | Read-only; no person node, which one check refuses | `check_athena_no_person` |
| Card profile, software token, personalization, verifier device, duress timing | IMPLEMENTED PROPOSED | No card manufactured, no silicon certified | `check_card_profile`, `check_card_emulator`, `check_verifier_device`, `check_duress_on_card` |
| Duress code, in the application and on the card | IMPLEMENTED EXTERNAL VALIDATION REQUIRED | Duress-aware, not compulsion-resistant: resists the casual coercer only, and its record is indelible | `check_duress_alertable`, `check_duress_claims_are_aware` |
| Pilot wind-down; coexistence and sunset plan | IMPLEMENTED | The sunset verdict rests on operator attestations | `check_coexistence_plan`, the wind-down drill |
| NIST 800-63 mapping; accessibility | IMPLEMENTED | Not a conformance claim; the automated third only | the assurance-mapping and accessibility drills |

*continued on the next page*

| Capability | Status | Limit stated by the repository | Pinned by |
|---|---|---|---|
| Formal models: C1, C2, C3, status freshness | IMPLEMENTED | Models, not the code; each of three must fail a counterpart configuration | the TLA+ drill |
| Holder key bound to a credential | IMPLEMENTED | Optional; the proof deliberately does not cover the presented code | `check_holder_key_binding` |
| Agent delegation, revocable by the holder | IMPLEMENTED | A plain binding still shows the service the credential | `check_agent_grant` |
| Four packages on public registries | IMPLEMENTED | Distribution is not validation | the publish workflow, the boundary drill |
| Metis (learning the system) | PROPOSED | Assessment required before code, and an outside need before the assessment | |
| Myrmex (stigmergic debugging) | PROPOSED | Proposed in this paper | |
| Blinded or one-time presentations; a native anonymous predicate | PROPOSED | The residue: a plain or interoperable credential shows a verifier stable material | |
| Hardware token, enrolment kit | PROPOSED | Roadmap rows, held | |

## The checks behind each layer card

The layer cards in the body say what each layer is, trusts, refuses and sees. The invariant checks and drills that fail the build when a card's claims stop being true are listed here, card by card, so the body can be read without them.

**The credential core.** `check_c1c10_objects_resolve`, the C3 property tests and the C3 model, the concurrency suite, `check_card_profile`, `check_card_emulator`, `check_verifier_device`, `check_duress_on_card`, `check_duress_claims_are_aware`, the attack suite's forgery and tamper cases.

**The walls.** `check_aor_append_only_triggers`, `check_aor_privilege_boundary`, `check_no_fk_cascade`, `check_retention_engine`, `check_append_only_tables_are_tested_exhaustively`, `check_constraint_suite_is_mutation_tested`, `check_triggers_are_mutation_tested`, `check_unique_rules_are_tested_exhaustively`, `check_procedure_refusals_are_mutation_tested`, `check_seed_restart_resets_dependent_records`, the SQL self-tests.

**Cryptographic evidence.** `check_pqc_signing_wired`, `check_pqc_second_witness`, `check_verify_witness_sampling`, `check_real_pqc_default_boot`, `check_trust_lifecycle`, `check_algorithm_agility`, `check_quantum_event_readiness`, `check_zk_witnesses_are_mutation_tested`, `check_post_quantum_claims_are_agility`, the canonical-equivalence oracle, the attack suite.

**Independent verification.** `check_detached_verifier`, `check_conformance_suite`, `check_typescript_sdk`, `check_verifier_fuzz`, `check_attacks_run`, `check_protocol_versioning`, `check_sdk_refusals_are_mutation_tested`, `check_conformance_asks_the_relying_party_question`, `check_oid4vp_verifier_boundary`, `check_documented_verifier_commands_run`.

**Verification and status.** `check_relying_party_api`, `check_offline_verification`, `check_epoch_revocation_propagation`, `check_federation_status_bundle`, `check_status_distribution`, the status-freshness model, the verify-load drills under failover.

**The privacy boundary.** The C2 CHECK and the C2 model, `check_c6_atlas_redacts_zk_location`, the redaction property suite, `check_redaction_adversary_is_not_flattered`, `check_pairwise_presentation`, `check_scoped_nullifier`, `check_audited_reads_are_logged`, the ZK second-witness differential, `check_athena_no_person`.

**Use.** `check_holder_wallet`, `check_wallet_presentation`, `check_holder_verifier_flow`, `check_auth_broker`, `check_broker_policy_bound`, `check_relying_party_api`, `check_qr_resource_bounds`, `check_documented_verifier_commands_run`.

**Federation.** `check_federation_topology`, `check_inter_authority_protocol`, `check_epoch_revocation_propagation`, `check_cross_authority_zk`, `check_federation_two_instances`, `check_exchange_trust_directional`, `check_route_guards_are_derived_not_listed`.

**Transparency.** `check_transparency_log`, `check_transparency_gossip`, `check_transparency_publication`, `check_timestamp_transparency`, `check_transparency_program`, `check_audited_reads_are_logged`, the canonical-equivalence oracle for the signed head.

**Institutions.** `check_registry`, `check_exchange_gateway`, `check_exchange_receipt`, `check_timestamp_authority`, `check_document_signing`, `check_ltv_timestamp_trust`, `check_broker_policy_bound`.

**Operations.** `check_ha_automation`, `check_multi_region_dr`, `check_read_replica_routing`, `check_event_table_partitioning`, `check_dr_drill_scheduled`, `check_chaos_program`, `check_zero_downtime_deploy`, `check_capacity_model`, `check_abuse_controls`, `check_every_audit_event_has_a_writer`, `check_logs_exclude_pii`, `check_container_hardening`, `check_cve_scanning`, `check_drills_count_their_cases`, `check_coexistence_plan`.

**System cognition.** `check_athena_no_person`, `check_athena_read_only`, `check_athena_functions_bounded`, `check_athena_non_sovereign`, `check_athena_rule_enforcement_resolves`, `check_atlas_console`, `check_benchmark_measures_growth`, the C8 caps.

**The assurance layer.** Its own mutation drills, including one over the check layer and one over the conformance runner; the detection tests of every check.

## Operations: the drills behind Section 13

**High availability.** The database runs under Patroni, which elects the primary, with its leader lease held in a three-member etcd consensus store, and role-based routing. Under a live write stream the drill loses the leader (the replica promoted after the 20-second lease, in-flight queries failing fast and retried, none lost), cuts the leader off from the lease store (it stood down in 7 seconds), performs a planned switchover (3.3 seconds, no insert failed), and crashes an etcd member (a 0.3-second stall). The Kubernetes profile runs the same members with the cluster's API as the lease store and deletes the leader pod.

**A second region.** The HA profile survives a node dying; it does not survive the region, and the tempting fix, a third member in region B, is wrong three ways at once, because it puts the wide-area network inside the lease store's quorum, on every commit, and inside region A's blast radius. So region B is a standby cluster: its own lease store, a different scope, streaming asynchronously through region A's router so a failover inside A does not break replication, electing no primary of its own so two regions can never both accept writes. The recovery point is therefore not zero, and it is measured rather than asserted: the drill cuts region A the way a region goes dark, promotes B under a live write stream, and reports the time to serve (5 seconds) and the rows acknowledged in A and absent from B (zero at the drill's rate), against ceilings of 90 seconds and 50 rows. It asserts two things that matter more than either number: B holds no row A never acknowledged, and what crossed is a contiguous prefix, because a standby holding rows one to forty and forty-five to sixty has skipped rather than lagged. A returned region A is rebuilt as a standby and never brought back as a primary.

**Read replicas.** Read-only surfaces route to a streaming replica under an explicit staleness contract, at most ten seconds behind by default, falling back to the primary beyond it, with every routed response naming its source and lag. Correctness-critical reads stay on the primary, and the authorization half of a verification verdict is pinned there (Section 7).

**Partitioning.** The four append-only event tables are monthly range-partitioned, and a drill proves that the append-only trigger holds across a partition, an attach and a detach, because the archives' carve-out must not become a rewrite path.

**Disaster recovery.** Backups are encrypted and a restore fails closed without the key. A monthly drill kills the primary, restores it from the archive and commits the measured numbers against the targets of 300 seconds of data loss and four hours to recover: the last recorded rows show 36 to 42 seconds of loss and under five seconds to service, on the drill's ephemeral host, not production hardware.

**Chaos.** A weekly drill boots the blue-green stack under traffic, kills one colour with zero dropped requests, stops both until the outage page reaches a real alert manager and webhook, kills Redis and PostgreSQL, partitions the pooler, and commits every recovery time against a ceiling. Findings become checks.

**Rolling deploys.** A deploy under traffic drops zero requests; edge and database recreation are window operations measured on every push against ceilings (0.3 seconds for the edge on a single host, 0.6 seconds for a database crash), and the readiness ledger carries the remaining edge half openly.

## Performance numbers

**Table 6:** Performance numbers at the versions that measured them. Every row is single-node on the reference laptop unless marked as a simulation or a projection. The event-ingestion rate and the cryptographic verification rate differ by about 35 times, a gap the repository's own history once hid (Section 15). Rows the repository has not re-measured since Version 2 carry their original stamp.

| Quantity | Number | Kind | Source |
|---|---|---|---|
| ML-DSA-65 sign, one core | about 2,080 per second | measured | dyno, `1.0.0-rc.7` |
| ML-DSA-65 verify, single witness, one core | about 7,950 per second | measured | dyno, `1.0.0-rc.7` |
| ML-DSA-65 verify, two witnesses, one core | about 740 per second | measured | dyno, `1.0.0-rc.7` |
| Membership proof, depth 14 | about 25 ms to prove, 12 ms to verify, 76 KB | measured | dyno, `1.0.0-rc.7`; the soundness ledger |
| Closing an epoch at depth 24, one million members | 2.1 s, tens of megabytes (was 10.8 s and 2.9 GB for a thousand) | measured | epoch cadence, `v9.357` |
| Re-signing a population under ML-DSA-87, one runner | about 345 credentials per second, 91 percent of it signing | measured | quantum-event drill, `v9.365` |
| Issuance through the application | 40 per second sustained, p95 28 ms | measured | baseline, `v9.191` |
| Verification event through the application | 80 per second sustained, p95 18.5 ms | measured | baseline, `v9.191` |
| Verification latency on the two-member HA topology under failover | a p50 at about 2 requests per second; p95 and p99 withheld, the sample cannot carry them | measured | failover drill, `v9.387` |
| Enrolment with real signing, bulk pipeline | about 372 tokens per second | simulated, 1:10,000 | national simulation, `v9.257` |
| Verification-event ingestion (audit writes, no signature check) | about 25,970 per second | simulated, 1:10,000 | national simulation |
| Atlas aggregates, 8,010 to 80,010 events | every bounded aggregate grew slower than its data (0.72x to 0.89x of linear) | measured | benchmark, `v9.402` |
| Single-witness verification across an eight-core node | about 62,800 per second | projection, fan-out assumed | benchmark arithmetic |

# Appendix B Lineage: what is inherited, and what is not

This appendix separates invention from synthesis. It claims less than a reader might assume: Polaris contains no new cryptography, no new protocol and no new proof system. Every primitive below is someone else's, and what follows names whose.

## B.1 Inherited

- *Signatures.* ML-DSA and SLH-DSA are the NIST standards [6, 7]. The migration path and its audit trail are Polaris's engineering; the security of the primitives is not, and nothing in this repository bears on it.
- *Merkle commitments and transparency.* The epoch tree and the anchor batches are Merkle trees [12]. The transparency logs follow Certificate Transparency's append-only log, inclusion and consistency proofs [17], and the witness cosignature over a log head follows the idea of collective witness cosigning [22].
- *Anonymous membership.* Proving membership of a committed set without revealing which member, with a nullifier to refuse a second use, is the construction Zerocash made practical [25]; the circuit is built on Plonky2 [15]. The broader aim, credentials a holder shows without being traced, goes back to Chaum [3] and to anonymous credential systems [2], which Polaris does not implement: it is not a general anonymous-credential system (Section 8).
- *Presentation and wallets.* The wallet-facing verifier implements OpenID for Verifiable Presentations and its high-assurance profile [13, 8], over SD-JWT and SD-JWT VC [18, 19] with the token status list [21]. The mobile-document route follows ISO/IEC 18013-5 [10], and the credential vocabulary the W3C data model [23].
- *Relying parties.* Per-relying-party subject identifiers are OpenID Connect's pairwise identifiers [14]. The short-lived signed status assertion is the pattern of an OCSP response a server staples [16]. Operator sign-in uses WebAuthn [24].
- *Proofing.* The evidence strengths and the derivation of an assurance level follow NIST SP 800-63A [20]; Polaris records the reading, and says so where it is a reading.
- *Duress codes.* A second secret that signals coercion is long-standing practice in alarm systems and banking; it is not invented here. What Polaris adds is the measurement in Section 3 of the adversary against whom it is worse than nothing.
- *Testing the tests.* Mutation analysis is DeMillo, Lipton and Sayward's [4], surveyed by Jia and Harman [11].

## B.2 What may be Polaris's own

To the author's knowledge, and offered as a description, not a claim of priority:

- the *arrangement*: rights-level constraints held by the schema rather than the application, each paired with a machine check that has its own detection test;
- the *assurance method*: mutating the assurance layer itself (the checks, triggers, procedures, verifiers and witnesses) to find green checks that prove the wrong proposition, as described in Section 15, with its record in the companion *Evidence Record*;
- the *operating contract*: progress counted only as a surviving external dependency, with a kill line, rather than as internal growth.

None of these is a result a reader should accept on this report's word. Each is described so that it can be compared with prior work, and a reader who knows such work is asked to name it.

### B.3 What this appendix covers

It does not survey identity systems, digital-identity programmes or the anonymous-credential literature; it names only what Polaris's mechanisms directly derive from. A comparison with deployed national systems is the table in the repository's README, which states that every one of them is deployed and Polaris is not.

## Appendix C Glossary

Terms come first, in alphabetical order. The abbreviations the text uses follow them, each expanded, and for a standard or an organisation, what it is.

### Terms

**Anchor set.** The published issuer keys a relying party has chosen to trust. Trust in Polaris is always membership in the verifier's own anchor set, never a decision the issuer makes for it.

**Anonymity set.** The crowd a membership proof hides in: the members of the epoch it was made against. An epoch below the minimum anonymity set, twenty unless set, is not closed, and a proof against a smaller one is refused; above that floor no verdict reports its size.

**Append-only.** A table on which UPDATE and DELETE are refused by a database trigger, so history can only be added to. The audit of record is the named pattern of applying this per table.

**Assurance level.** What an enrolment can honestly claim about how the person was proven, derived from the evidence recorded beside it and never typed by an operator.

**Athena.** The read-only layer that holds the structure of authority (agencies, keys, contexts, trust and rules, and what enforces each) and the constitution as data. It may not model a person.

**Atlas.** The bounded operational console: aggregates of what the system is doing, capped by the application, never the location of a zero-knowledge event and never a person.

**Attestation.** A directional, per-context statement that one agency accepts another's credentials, recorded as a row. Never transitive.

**Authenticity pack.** A credential as a file: the token value, its signature and the issuer's public key. What a holder holds and a stranger verifies.

**C1–C10.** The ten hard constraints of the constitution: five constitutional rights guarantees (C1, C2, C3, C6, C10) and five engineering constraints that keep them honest under load and attack. Each is named in the section that enforces it.

**Canonicalisation.** Turning a structure into one exact sequence of bytes (sorted keys, no whitespace) so that a signature made on one machine can be checked on another that rebuilds the same bytes.

**Consistency proof.** A short proof that a smaller Merkle tree is a prefix of a larger one, so that a log only appended between two heads.

**Detached verifier.** A program that checks Polaris artifacts with no Polaris code, no database and no network: the `polaris-verify` package, and the script that is now a shim onto it.

**Disclosure level.** What a verification records: zero-knowledge (no token identifier), selective (named attributes), or full (the identity, logged).

**Duress-aware.** The word the front door uses for the duress code, because the mechanism notices duress and does not resist compulsion; the stronger word is refused by a check.

**Dyno.** The benchmark that runs in continuous integration and publishes the throughput and latency it measured on that machine; a number cited as "dyno" was measured there, not estimated.

**Epoch.** A closed, immutable commitment to the active-token set at a moment, as a Merkle root, against which membership proofs are made.

**Equivocation.** A log telling two observers two different histories. The proof of it is two signed heads of the same size with different roots.

**External noun.** A named party outside the repository that did something to it, on a date, with a result that can be pointed at: the only kind of entry the scoreboard accepts.

**Federation.** Independent authorities, each its own root, recognising one another through explicit attestations, with no central service.

**Gossip.** Witnesses sharing the log heads they were shown so that a split view is detected by comparison.

**High availability.** A database that survives the loss of its leader by promoting a replica under a lease, drilled and measured.

**Inclusion proof.** A short proof that one entry is in a Merkle tree with a given root.

**Lab.** The part of the repository whose job is to try to falsify the claims that make Polaris different (linkability, duress, the cryptographic migration). It carries no product version.

**Long-term validation.** Deciding that a signature was valid at the instant it was made, from evidence fixed at that instant, so that the decision survives the key's later retirement.

**Merkle tree.** A cryptographic fingerprint of a whole set, built by hashing pairs upward to one root, that lets a small proof show one item belongs to the set.

**Metis.** A proposed layer that would learn the behaviour of Polaris itself: anomalies, capacity, regressions, release risk.

**ML-DSA.** The lattice-based signature scheme standardised as FIPS 204; parameter sets 65 and 87 are accepted. Post-quantum in its assumptions; the repository claims agility, not settled security.

**Mosca's inequality.** The comparison of the years data must stay secret plus the years a migration takes against the years until a quantum computer can break today's signatures. When the first sum is larger, what is signed today is already at risk.

**Mutation drill.** A program that removes or inverts a guarantee, a constraint, a trigger, a refusal, a check's inputs, and requires the tests that supposedly prove it to go red, with a negative control so that zero survivors is distinguishable from a harness that ran nothing.

**Myrmex.** A proposed debugging system outside Polaris: specialised agents that attack it independently and leave reproducible evidence on a shared landscape, coordinated by stigmergy.

**Nullifier.** A value derived from a secret and a scope that lets a verifier detect a second use without linking uses across scopes. In Polaris `Poseidon(secret || scope || epoch)`, a public input of the membership proof, per relying party and per epoch, never global.

**OpenID4VP.** OpenID for Verifiable Presentations, the protocol by which a wallet presents a credential to a verifier; the path on which the first outside wallet spoke to Polaris.

**Operating contract.** The rule adopted on 13 September 2026 that the unit of progress is a surviving external dependency, that a product change needs a named outside reason, and that everything else goes to the lab.

**Pairwise identifier.** A different handle for each relying party, so that two parties cannot join their logs. In Polaris the login token's subject and the presentation handle are derived under the relying party's scope; it bounds what a verifier stores, not what it is shown.

**Partition.** A table split into monthly pieces by timestamp, transparent to queries, so that old months can be detached to the archive.

**PKCE.** A proof-of-possession extension to the authorization-code login flow, so that an intercepted code cannot be exchanged by an attacker.

**Poseidon.** A hash function designed to be cheap inside a zero-knowledge circuit. Polaris uses it for epoch leaves and nullifiers.

**Positive control.** The half of a measurement that proves the instrument can see anything: an adversary that must succeed on a linkable population, a mutated tree the check must fail on, a patched verifier the suite must notice.

**Post-quantum.** Cryptography whose security does not rest on problems a quantum computer solves efficiently. In this paper, a statement about algorithms, never about the system.

**Posture ledger.** The repository's record of each cryptographic dependency with its threat and its exposure, including the hash-based fallback, registered as rows.

**Readiness ledger.** The repository's bound on what it may claim: what a production deployment would need that the repository does not provide, and the decisions it leaves to a deploying organisation.

**Release candidate.** The version `1.0.0-rc.7`: every condition the operating contract names for 1.0.0 holds. The system moved to 1.0.0 on 11 October 2026, when IDFoundry/OID4VCgo's continuous integration began running `polaris-oid4vp` (Section 20.3).

**Replica.** A copy of the database that follows the primary with some lag; safe for reading immutable material, unsafe for deciding current authorization.

**Review packet.** The document handed to an outside reviewer: what each subsystem is supposed to guarantee, what holds it, and where to spend attention first.

**Revocation feed.** A signed list of the hashes of revoked credentials an authority issued, consumed offline.

**Scoreboard.** `lab/EXTERNAL-NOUNS.md`: the file that decides whether the project continues, holding only what named outside parties did, with zero and blank as valid values.

**SD-JWT VC.** A credential format in which the issuer signs digests of the claims and the holder discloses the ones a verifier asked for; the format the first outside wallet presented.

**Signed tree head.** A log's signed commitment to its whole history at a size.

**Soundness ledger.** The record of what the membership proof's soundness rests on and what was measured about it.

**Standby region.** A second cluster with its own lease store, streaming asynchronously from the first, electing no primary of its own until promoted; its recovery point is measured, not zero.

**Status assertion.** A short-lived, issuer-signed statement of a credential's current status, stapled to a presentation so that authorization is decidable offline.

**Stigmergy.** Coordination through traces left in a shared environment instead of a central plan, as in an ant colony. The proposed Myrmex would work this way.

**Sunset.** The day the old credential stops being accepted, treated as a decision with a floor under it because it is the moment the new one becomes compulsory.

**Trust list.** A signed statement of every key an instance knows, with each key's status and the instants those statuses took effect.

**Trusted referee.** A person whose vouching lets somebody with no documents be enrolled, bounded and co-signed past a threshold, and invisible on the credential.

**Two-witness principle.** No cryptographic verdict is trusted from a single implementation; a second, independently written one must agree or abstain explicitly.

**Version numbers.** The tree counted builds as `v9.N` until mid-September 2026 and then took release-candidate numbers, `1.0.0-rc.N`, which move only for a change seen from outside. This map is drawn at `1.0.0-rc.7`; a `v9.N` beside a measurement says which build it was taken on. The packages carry their own version numbers.

**Witness.** An independent party that cosigns log heads it found consistent and refuses a fork.

**Wycheproof.** Project Wycheproof, a public corpus of cryptographic test vectors, including inputs a correct implementation must reject. Polaris runs its ML-DSA vectors under both witnesses.

**Zero-knowledge proof.** A proof that a statement is true which reveals nothing beyond the statement's truth; in Polaris, membership of a token in an epoch.

## Abbreviations

Every abbreviation the text uses, spelled out, with what each standard or body is.

**API.** Application programming interface: the requests one program uses to call another.

**CBOR.** Concise Binary Object Representation, the binary data format ISO 18013-5 mobile documents use.

**CI.** Continuous integration: the build, tests and drills run automatically on every push.

**CSRF.** Cross-site request forgery: an attack in which another site makes a browser send a request as a signed-in user.

**ES256.** The classical ECDSA signature over curve P-256 that the HAIP profile pins.

**FIPS.** US Federal Information Processing Standards; FIPS 202 defines SHA-3, FIPS 204 ML-DSA and FIPS 205 SLH-DSA.

**HA.** High availability.

**HAIP.** The High Assurance Interoperability Profile of OpenID4VP, which pins the credential format, the signature algorithm and the response mode a wallet and a verifier must use.

**HTTP.** Hypertext Transfer Protocol, over which web services exchange requests.

**ISO/IEC 18013-5.** The international standard for the mobile driving licence.

**JSON.** JavaScript Object Notation, the text format every signed Polaris statement is written in.

**KMS.** Key management service: a cloud service that holds keys.

**NFC.** Near-field communication: the short-range radio a card answers a reader over.

**NIST SP 800-63.** The US National Institute of Standards and Technology's digital identity guidelines, which define assurance levels.

**npm.** The public package registry for JavaScript and TypeScript.

**PIN, PUK.** The personal code that unlocks a card, and the unblocking key that clears a card locked by too many wrong PINs.

**PKCS#11.** The standard programming interface to hardware security modules and tokens that generate and hold keys inside themselves.

**PyPI.** The Python Package Index, the public registry for Python packages.

**QR.** A two-dimensional barcode read by a camera; one way a presentation is transferred.

**RFC.** Request for Comments: the series in which internet standards are published; RFC 6962 describes Certificate Transparency.

**SDK.** Software development kit: a library a relying party builds into its own program.

**SHA3-256.** A cryptographic hash function of the SHA-3 family (FIPS 202) giving a 256-bit digest; Polaris signs the SHA3-256 digest of a token value.

**SLH-DSA.** The hash-based signature scheme standardised as FIPS 205, registered as the fallback should the lattice assumptions fall.

**SQL.** The query language of relational databases, in which the Polaris schema, triggers and procedures are written.

**TLA+.** A formal specification language whose model checker explores every interleaving of events to show a property holds.

**TLS.** Transport Layer Security, the protocol of the encrypted connection; the Polaris edge negotiates a hybrid post-quantum key exchange.

**W3C.** The World Wide Web Consortium, which publishes the Verifiable Credentials data model.

**WebAuthn.** The standard for proving possession of a hardware key or a device's built-in authenticator; operators use it as a second factor after the password.

**X.509.** The standard format of a public-key certificate.

**ZK.** Zero knowledge.